

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng Web Application Firewall lai
rule-based + DistilBERT cho phát hiện tấn công ứng
dụng web**

<HỌ VÀ TÊN SINH VIÊN>

t21102004@gmail.com

Chương trình đào tạo: Công nghệ thông tin

Giảng viên hướng dẫn: <HỌC HÀM HỌC VỊ + HỌ TÊN GVHD>

Khoa: <KHOA>

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng Web Application Firewall lai
rule-based + DistilBERT cho phát hiện tấn công ứng
dụng web**

<HỌ VÀ TÊN SINH VIÊN>
t21102004@gmail.com

Chương trình đào tạo: Công nghệ thông tin

Giảng viên hướng dẫn: <HỌC HÀM HỌC VỊ + HỌ TÊN GVHD>

Chữ kí GVHD

Khoa: <KHOA>

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Lời cảm ơn của sinh viên (SV) tới người yêu, gia đình, bạn bè, thầy cô, và chính bản thân mình vì đã chăm chỉ và quyết tâm thực hiện ĐATN để đạt kết quả tốt nhất, nên viết phần cảm ơn ngắn gọn, tránh dùng các từ sáo rỗng, giới hạn trong khoảng 100-150 từ.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sinh viên viết tóm tắt ĐATN của mình trong mục này, với 200 đến 350 từ. Theo trình tự, các nội dung tóm tắt cần có: (i) Giới thiệu vấn đề (tại sao có vấn đề đó, hiện tại được giải quyết chưa, có những hướng tiếp cận nào, các hướng này giải quyết như thế nào, hạn chế là gì), (ii) Hướng tiếp cận sinh viên lựa chọn là gì, vì sao chọn hướng đó, (iii) Tổng quan giải pháp của sinh viên theo hướng tiếp cận đã chọn, và (iv) Đóng góp chính của ĐATN là gì, kết quả đạt được sau cùng là gì. Sinh viên cần viết thành đoạn văn, không được viết ý hoặc gạch đầu dòng.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

ABSTRACT

Mục này khuyến khích sinh viên viết lại mục “Tóm tắt” đề án tốt nghiệp ở trang trước bằng tiếng Anh. Phần này phải có đầy đủ các nội dung như trong phần tóm tắt bằng tiếng Việt. Sinh viên không nhất thiết phải trình bày mục này.

Nhưng nếu lựa chọn trình bày, sinh viên cần đảm bảo câu từ và ngữ pháp chuẩn tắc, nếu không sẽ có tác dụng ngược, gây phản cảm.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	4
1.4 Bố cục đồ án	5
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	7
2.1 Khảo sát hiện trạng	7
2.1.1 Khảo sát nhu cầu người dùng	7
2.1.2 Khảo sát các hệ thống WAF hiện có.....	8
2.2 Tổng quan chức năng	10
2.2.1 Biểu đồ ca sử dụng tổng quát	10
2.2.2 Ca sử dụng: Kiểm tra và lọc request HTTP	10
2.2.3 Ca sử dụng: Quản lý luật (Rule Management).....	12
2.2.4 Ca sử dụng: Giám sát và xem log.....	13
2.2.5 Ca sử dụng: Quản lý IP (Blacklist / Whitelist).....	13
2.2.6 Ca sử dụng: Cấu hình hệ thống.....	14
2.2.7 Ca sử dụng: Cảnh báo sự kiện bảo mật (Alert / Notification).....	15
2.2.8 Ca sử dụng: Xác thực và đăng ký tài khoản	15
2.2.9 Ca sử dụng: Quản lý tài khoản cá nhân (Account Settings).....	16
2.2.10 Ca sử dụng: Quản trị người dùng (Admin User Management).....	17
2.3 Đặc tả chức năng	18
2.3.1 Đặc tả: Kiểm tra request và đưa ra quyết định	18
2.3.2 Đặc tả: Xử lý anomaly scoring và ML gray-zone confirm	19
2.3.3 Đặc tả: Rate Limiting	21

2.3.4 Đặc tả: Cảnh báo sự kiện bảo mật.....	21
2.3.5 Đặc tả: Audit Logging	22
2.4 Yêu cầu phi chức năng	22
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	24
3.1 Go (Golang) – Nền tảng xây dựng WAF core	24
3.1.1 Khái quát	24
3.1.2 Vai trò trong hệ thống.....	24
3.1.3 Lý do lựa chọn	25
3.2 Rule Engine và OWASP CRS – Cơ sở phát hiện dựa trên luật	25
3.2.1 Khái quát cơ chế rule-based detection	25
3.2.2 Cấu trúc rule JSON	27
3.2.3 Bộ luật 78 rules / 13 nhóm.....	27
3.3 DistilBERT – Mô hình phân loại tấn công.....	28
3.3.1 Khái quát kiến trúc Transformer và BERT.....	28
3.3.2 Fine-tuning cho bài toán phân loại HTTP request.....	29
3.3.3 Lý do lựa chọn	30
3.4 FastAPI và Python – ML Inference Service	31
3.4.1 Khái quát	31
3.4.2 Vai trò trong hệ thống.....	31
3.5 PostgreSQL – Lưu trữ logs, users, cấu hình.....	31
3.5.1 Khái quát	31
3.5.2 Vai trò trong hệ thống.....	32
3.5.3 Schema chính	32
3.6 JWT và Bcrypt – Xác thực và bảo mật.....	32
3.6.1 JWT: xác thực không trạng thái với dual-mode delivery	32
3.6.2 Bcrypt: hashing mật khẩu với cost factor	33

3.6.3 RBAC: ba vai trò admin / editor / viewer	33
3.7 Token Bucket – Thuật toán Rate Limiting	34
3.7.1 Khái quát Token Bucket.....	34
3.7.2 So sánh với các thuật toán khác	35
3.7.3 Tích hợp vào middleware stack.....	35
3.8 Docker và Docker Compose – Đóng gói và triển khai.....	35
3.8.1 Containerization các thành phần.....	35
3.8.2 Docker Compose one-command deployment	36
3.8.3 Lý do lựa chọn	36
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ KẾT QUẢ THỰC NGHIỆM	
38	
4.1 Kiến trúc tổng quan hệ thống	38
4.1.1 Sơ đồ kiến trúc tổng thể.....	38
4.1.2 Luồng xử lý request.....	38
4.1.3 Các thành phần chính và tương tác.....	40
4.2 Thiết kế và triển khai Rule Engine.....	41
4.2.1 Cấu trúc dữ liệu Rule.....	41
4.2.2 Transform chain	43
4.2.3 Pattern matching	43
4.2.4 Anomaly scoring và quyết định	44
4.2.5 Bộ luật 78 rules.....	44
4.3 Thiết kế và triển khai ML Layer	45
4.3.1 Kiến trúc ML Inference Service	45
4.3.2 Cơ chế tích hợp vào Rule Engine (ml_hook.go)	46
4.3.3 Canonical text preprocessing.....	46
4.3.4 Pipeline huấn luyện và phiên bản model.....	47

4.4 Thiết kế cơ sở dữ liệu	48
4.4.1 Sơ đồ ERD	48
4.4.2 Bảng <code>audit_logs</code>	48
4.4.3 Bảng <code>users</code>	49
4.5 Giao diện Dashboard	49
4.5.1 Tổng quan giao diện	49
4.5.2 Các màn hình chính.....	50
4.5.3 Hệ thống Theme đồng nhất	51
4.6 Kiểm thử và Đánh giá	51
4.6.1 Chiến lược kiểm thử	51
4.6.2 Kết quả kiểm thử Rule Engine	52
4.6.3 Kết quả kiểm thử ML Model	52
4.6.4 Kết quả kiểm thử Rate Limiting.....	53
4.6.5 Kiểm thử bảo mật nội bộ (auth/authorization)	53
4.7 Triển khai	54
4.7.1 Yêu cầu hệ thống.....	54
4.7.2 Hướng dẫn triển khai (Docker Compose).....	54
4.7.3 Cấu trúc file triển khai.....	55
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	57
5.1 Thiết kế cơ chế Anomaly Scoring hai lớp (Rule + ML).....	57
5.1.1 Đặt vấn đề	57
5.1.2 Giải pháp đề xuất	58
5.1.3 Kết quả	59
5.2 Thiết kế bộ luật chuẩn hóa với Transform Chain chống Evasion	59
5.2.1 Đặt vấn đề	59
5.2.2 Giải pháp đề xuất	60

5.2.3 Kết quả	60
5.3 Pipeline huấn luyện ML có kiểm soát chất lượng (Hard Gate).....	62
5.3.1 Đặt vấn đề	62
5.3.2 Giải pháp đề xuất	62
5.3.3 Kết quả	63
5.4 Thiết kế interface MLPredictor tách biệt Engine và ML Client.....	63
5.4.1 Đặt vấn đề	63
5.4.2 Giải pháp đề xuất	64
5.4.3 Lợi ích	65
5.5 củng cố lớp Auth: bịt lỗ leo thang đặc quyền và bảo vệ Invariant	66
5.5.1 Đặt vấn đề	66
5.5.2 Giải pháp đề xuất	67
5.5.3 Kết quả	68
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	70
6.1 Kết luận	70
6.1.1 Kết quả đạt được	70
6.1.2 Đánh giá so với mục tiêu ban đầu	71
6.1.3 Hạn chế.....	72
6.2 Hướng phát triển.....	72
6.2.1 Hướng ngắn hạn (1–3 tháng)	73
6.2.2 Hướng trung hạn (3–12 tháng).....	73
6.2.3 Hướng dài hạn (12 tháng trở lên)	74
TÀI LIỆU THAM KHẢO.....	78
PHỤ LỤC.....	80
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	80
A.1 Ngành học.....	81

A.2 Đánh dấu (bullet) và đánh số (numering)	81
A.3 Cách thêm bảng	82
A.4 Chèn hình ảnh	82
A.5 Tài liệu tham khảo	83
A.6 Cách viết phương trình và công thức toán học.....	83
A.7 Qui cách đóng quyển.....	83
B. ĐẶC TẢ USE CASE.....	85
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	85
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	85

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này trình bày bức tranh toàn cảnh về sự gia tăng của các mối đe dọa bảo mật ứng dụng web trong bối cảnh chuyển đổi số hiện nay, từ đó làm rõ tính cấp thiết của bài toán phòng thủ ở tầng ứng dụng. Nội dung chương xác định các hạn chế của các giải pháp tường lửa ứng dụng web hiện tại, đề xuất mục tiêu cụ thể của đề tài, và giới thiệu định hướng giải pháp kỹ thuật mà đề án theo đuổi.

1.1 Đặt vấn đề

Trong thập kỷ vừa qua, ứng dụng web đã trở thành hạ tầng trung tâm của nền kinh tế số. Theo báo cáo của Verizon Data Breach Investigations Report (DBIR) năm 2023, hơn 74% các sự cố vi phạm dữ liệu có liên quan đến yếu tố con người hoặc lỗ hổng ở tầng ứng dụng, trong đó tấn công vào ứng dụng web chiếm vị trí đứng đầu trong các vectơ xâm nhập được ghi nhận. Tổ chức OWASP (Open Web Application Security Project) hàng năm công bố danh sách OWASP Top 10, tập hợp mười nhóm lỗ hổng nghiêm trọng và phổ biến nhất tại tầng ứng dụng. Trong phiên bản cập nhật gần nhất (2021), các nhóm như SQL Injection (SQLi), Cross-Site Scripting (XSS), Server-Side Request Forgery (SSRF), và các dạng tấn công tiêm nhiễm lệnh (Command Injection) vẫn tiếp tục duy trì thứ hạng cao, phản ánh thực trạng rằng chúng không chỉ phổ biến mà còn liên tục được khai thác thực tế.

Trước áp lực đó, Tường lửa Ứng dụng Web (Web Application Firewall – WAF) ra đời và đóng vai trò như một tầng phòng thủ chuyên biệt, đặt giữa người dùng và ứng dụng backend. Khác với tường lửa mạng thông thường vận hành ở tầng 3 và 4 của mô hình OSI, WAF kiểm tra nội dung thực sự của từng yêu cầu HTTP ở tầng 7, cho phép phát hiện các hành vi tấn công ẩn bên trong payload, tham số URL, tiêu đề HTTP hay nội dung body của yêu cầu. Đây là tầng kiểm soát mà tường lửa mạng truyền thống không thể tiếp cận.

Tuy nhiên, giải pháp WAF dựa thuần túy trên luật (rule-based) – hiện là phương pháp thống trị trong hầu hết các triển khai thực tế – bộc lộ những hạn chế cơ bản khó khắc phục. Thứ nhất, bộ luật được xây dựng dựa trên các mẫu (pattern) tấn công đã biết, chủ yếu thông qua biểu thức chính quy (regular expression). Kẻ tấn công dày dạn kinh nghiệm có thể vượt qua các pattern này bằng kỹ thuật làm mờ (obfuscation): mã hóa nhiều lớp (double URL encoding), thay đổi hoa thường kết hợp với chèn ký tự trắng, dùng biểu thức thay thế tương đương về ngữ nghĩa nhưng khác về cú pháp. Chẳng hạn, câu truy vấn `UNION SELECT` có thể được biến thể thành `uNiOn%09SELECT` hoặc `UN%49ON SELECT` để qua mặt các biểu thức chính quy kiểm tra case-sensitive. Thứ hai, hành vi của các rule trong thực tế mang

tính nhị phân (block hoặc allow), thiếu cơ chế đánh giá xác suất theo ngữ cảnh toàn bộ yêu cầu. Một yêu cầu có thể khớp một phần với nhiều rule nhưng không khớp hoàn toàn với bất kỳ rule đơn lẻ nào – trường hợp này rất dễ bị bỏ qua nếu không có cơ chế tích lũy điểm (anomaly scoring). Thứ ba, bộ luật phải được cập nhật thủ công khi xuất hiện biến thể tấn công mới, tạo ra khoảng thời gian chờ (detection gap) mà hệ thống hoàn toàn mù quáng trước các kỹ thuật tấn công zero-day hoặc mới được công bố.

Ở chiều ngược lại, các giải pháp WAF thương mại trên nền tảng đám mây như Cloudflare WAF hay AWS WAF giải quyết được một phần vấn đề cập nhật luật nhờ đội ngũ nghiên cứu bảo mật chuyên trách và khả năng học từ lưu lượng toàn cầu. Tuy nhiên, chúng lại phát sinh các hạn chế khác: toàn bộ lưu lượng ứng dụng, bao gồm dữ liệu nhạy cảm của người dùng và khách hàng, phải đi qua hạ tầng của bên thứ ba, đặt ra những lo ngại nghiêm trọng về quyền riêng tư và tuân thủ quy định (GDPR, Nghị định 13/2023/NĐ-CP tại Việt Nam). Chi phí vận hành tỷ lệ thuận với lưu lượng làm cho các giải pháp này khó tiếp cận với các tổ chức vừa và nhỏ. Quan trọng hơn, tính chất “hộp đen” của các dịch vụ này khiến đội ngũ bảo mật nội bộ mất đi khả năng kiểm soát, kiểm tra, và tinh chỉnh hành vi của WAF theo đặc thù riêng của ứng dụng.

Trong bối cảnh đó, nhu cầu về một giải pháp WAF triển khai tại chỗ (on-premise), mã nguồn mở, hiệu năng cao, có khả năng xử lý các yêu cầu ở tầng ứng dụng với độ trễ tối thiểu đồng thời tích hợp thêm một lớp phân tích thông minh để xử lý các trường hợp mơ hồ mà bộ luật cứng không thể giải quyết dứt khoát – là nhu cầu chính đáng và chưa có giải pháp mã nguồn mở nào đáp ứng thỏa đáng. Hệ thống như vậy, nếu được xây dựng và mở rộng đúng hướng, sẽ đem lại lợi ích cho nhiều đối tượng: từ các doanh nghiệp cần kiểm soát toàn bộ hạ tầng bảo mật của mình, đến các đơn vị nghiên cứu muốn thử nghiệm và đánh giá các kỹ thuật phòng thủ mới trong môi trường có thể tái lập (reproducible environment).

1.2 Mục tiêu và phạm vi đề tài

Để xác định mục tiêu và phạm vi phù hợp, đề tài trước hết tiến hành khảo sát và đánh giá tổng quan các giải pháp WAF mã nguồn mở và thương mại đang được sử dụng phổ biến nhất hiện nay.

ModSecurity kết hợp bộ luật OWASP Core Rule Set (CRS) là giải pháp WAF mã nguồn mở lâu đời nhất và được triển khai rộng rãi nhất trong thực tế. ModSecurity hoạt động như một module nhúng vào máy chủ web (Apache, Nginx, IIS) và xử lý lưu lượng thông qua năm pha kiểm tra (connection, request headers, request body, response headers, response body). Bộ luật OWASP CRS cung cấp hơn 200 rule

bao phủ hầu hết các nhóm tấn công trong OWASP Top 10. Dù vậy, ModSecurity gặp phải những hạn chế đáng kể trong vận hành thực tế: ngưỡng mặc định của CRS tạo ra tỷ lệ false positive cao trên các ứng dụng có payload phức tạp (API JSON, tham số dài), đòi hỏi quá trình tinh chỉnh tốn công sức trước khi đưa vào sản xuất. Kiến trúc module nhúng vào web server làm giới hạn khả năng triển khai độc lập và tái cấu hình động mà không cần khởi động lại dịch vụ. Ngoài ra, bản thân ModSecurity không có giao diện quản trị tích hợp, buộc đội ngũ vận hành phải kết hợp với các công cụ bên thứ ba như Kibana hay Grafana để có khả năng giám sát đầy đủ.

Cloudflare WAF và AWS WAF đại diện cho hướng tiếp cận đám mây, cung cấp giao diện quản trị trực quan, khả năng cập nhật luật tức thời và tích hợp sẵn mạng phân phối nội dung (CDN) để giảm thiểu các cuộc tấn công DDoS volumetric. Cloudflare đặc biệt nổi trội với khả năng học từ lưu lượng toàn cầu và các quy tắc dựa trên máy học (ML-based rules) bổ sung cho tập luật truyền thống. Tuy nhiên, như đã phân tích ở phần 1.1, cả hai giải pháp đều yêu cầu toàn bộ lưu lượng định tuyến qua hạ tầng của nhà cung cấp, điều không thể chấp nhận trong nhiều bối cảnh triển khai đòi hỏi kiểm soát dữ liệu tuyệt đối. Chi phí cũng là rào cản thực tế: Cloudflare Pro bắt đầu từ 20 USD/tháng mỗi tên miền, và AWS WAF tính phí theo số lượng WebACL, rule, và triệu lượt yêu cầu, khó dự đoán và kiểm soát ngân sách với traffic lớn.

Ngoài ra, một số giải pháp như Shadow Daemon hay Naxsi (module Nginx) cũng tồn tại trong không gian mã nguồn mở nhưng thiếu tính năng quản trị tích hợp, bộ luật không phong phú và không được duy trì tích cực.

Từ quá trình khảo sát trên, có thể rút ra kết luận: trong không gian giải pháp WAF hiện tại, tồn tại một khoảng trống đáng kể đối với nhóm nhu cầu cụ thể sau: **(i)** triển khai tại chỗ, không phụ thuộc bên thứ ba; **(ii)** viết bằng ngôn ngữ hiệu năng cao, có thể xử lý lưu lượng lớn với độ trễ thấp; **(iii)** tích hợp một cơ chế phân tích thông minh bổ sung cho bộ luật tĩnh, đặc biệt là để xử lý các trường hợp mà điểm anomaly nằm trong “vùng xám” mà bộ luật không tự tin đưa ra quyết định; và **(iv)** cung cấp giao diện quản trị tích hợp, cho phép đội ngũ bảo mật nội bộ giám sát, phân tích và phản ứng sự cố ngay từ một điểm duy nhất.

Trên cơ sở đó, đề tài xác định mục tiêu cụ thể như sau: *Xây dựng một hệ thống tường lửa ứng dụng web (WAF) triển khai tại chỗ, sử dụng ngôn ngữ lập trình Go, tích hợp hai lớp phát hiện tấn công – lớp rule engine dựa trên anomaly scoring và lớp mô hình học máy (DistilBERT fine-tuned) kích hoạt trong vùng xám – kèm theo dashboard quản trị real-time, hệ thống xác thực và phân quyền, cơ chế cảnh báo*

sự kiện bảo mật và khả năng triển khai đầy đủ qua Docker Compose.

Phạm vi của đề tài bao gồm:

- **Loại tấn công được bảo vệ:** SQL Injection (SQLi), Cross-Site Scripting (XSS), Remote/OS Command Injection (CMDi/RCE), Path Traversal và Local File Inclusion (LFI), Server-Side Request Forgery (SSRF), XML External Entity (XXE), Log4Shell, Server-Side Template Injection (SSTI), NoSQL Injection, Scanner Detection và Rate-based Behavior Analysis. Đây là các nhóm tấn công thuộc hoặc liên quan trực tiếp đến OWASP Top 10 (2021) và CWE/SANS Top 25.
- **Môi trường triển khai:** máy chủ Linux, container Docker, không yêu cầu kết nối Internet sau khi cài đặt. WAF hoạt động theo mô hình reverse proxy trong suốt (transparent reverse proxy), không yêu cầu thay đổi cấu hình ứng dụng backend.
- **Ngoài phạm vi:** bảo vệ tầng mạng (Layer 3/4), chống DDoS volumetric, phân tích lưu lượng mã hóa end-to-end (TLS termination ở tầng ngoài), và kiểm tra nội dung response phía backend (chỉ inspect request trong phạm vi đề tài này).

1.3 Định hướng giải pháp

Từ mục tiêu và phạm vi đã xác định, đề tài đề xuất kiến trúc giải pháp dựa trên hai nguyên tắc cốt lõi: *hiệu năng không thỏa hiệp ở đường nhanh* và *độ chính xác tăng cường ở trường hợp mơ hồ*.

Lớp thứ nhất – Rule Engine với Anomaly Scoring. Toàn bộ logic xử lý WAF được xây dựng bằng ngôn ngữ Go (Golang). Go là ngôn ngữ biên dịch tĩnh, có hệ thống goroutine cho phép xử lý đồng thời hàng nghìn kết nối với overhead tối thiểu, và garbage collector thế hệ mới với thời gian dừng (stop-the-world) thường dưới một mili giây – đặc tính quan trọng đối với proxy xử lý lưu lượng thời gian thực. Thay vì mô hình block-ngay theo từng rule đơn lẻ, đề tài sử dụng mô hình anomaly scoring: mỗi rule match đóng góp một giá trị điểm vào tổng điểm nguy cơ của yêu cầu, và quyết định cuối (ALLOW / CHALLENGE / BLOCK) được đưa ra dựa trên ngưỡng tích lũy. Cách tiếp cận này giúp giảm false positive vì một yêu cầu bình thường vô tình khớp một rule nhỏ sẽ không bị chặn ngay, nhưng một chuỗi tín hiệu tấn công dù mỗi cái yếu vẫn có thể tích lũy đủ điểm để kích hoạt quyết định phòng thủ. Để chống evasion qua encoding, mỗi rule được trang bị một chuỗi transform (biến đổi tiền xử lý) áp dụng theo thứ tự – ví dụ URL_DECODE → LOWERCASE → COMPRESS_WHITESPACE – trước khi tiến hành so khớp mẫu. Bộ luật của hệ thống gồm 78 rule – kết hợp tầng chữ ký tấn công chuẩn và tầng dấu hiệu (*indicator*) tích lũy – bao phủ 13 nhóm tấn công, được xây dựng theo cấu trúc JSON có phiên bản, dễ mở rộng và thay thế không cần biên dịch lại.

Lớp thứ hai – Mô hình học máy trong vùng xám. Quan sát thực tế cho thấy với nhiều yêu cầu HTTP, bộ luật tính đưa ra điểm anomaly nằm trong khoảng trung gian – cao hơn traffic thông thường nhưng chưa đủ để quyết định block dứt khoát. Để xử lý “vùng xám” này, đề tài fine-tune mô hình DistilBERT trên tập dữ liệu HTTP request có nhãn gồm mười lớp (normal và chín loại tấn công), và tích hợp mô hình vào hệ thống dưới dạng một microservice Python/FastAPI độc lập. Khi điểm anomaly của một yêu cầu rơi vào ngưỡng kích hoạt, WAF sẽ gọi đồng bộ tới ML service với timeout cố định 800 mili giây, nhận phán quyết của mô hình (nhãn lớp và độ tin cậy), và điều chỉnh điểm anomaly lên hoặc xuống tùy theo kết quả. Kiến trúc này có hai đặc tính quan trọng: ML không nằm trên đường nhanh (hot path) của mọi yêu cầu, nên overhead chỉ xuất hiện ở một phần nhỏ lưu lượng; và hệ thống fail-open (khi ML service không khả dụng, WAF giữ nguyên điểm rule và tiếp tục xử lý bình thường).

Các thành phần hỗ trợ. Bên cạnh hai lớp phát hiện, hệ thống được hoàn thiện bởi các thành phần sau: **(i)** Rate Limiting theo thuật toán Token Bucket per-IP chống tấn công DDoS ứng dụng và brute force; **(ii)** quản lý danh sách IP động (blacklist/whitelist); **(iii)** hệ thống xác thực và phân quyền dựa trên JWT và bcrypt, với phân quyền theo vai trò Admin/User bảo vệ các API quản trị; **(iv)** Dashboard web giám sát và quản trị real-time cho phép quan sát lưu lượng, xem nhật ký tấn công, quản lý luật và cấu hình hệ thống từ một giao diện duy nhất; **(v)** hệ thống cảnh báo sự kiện bảo mật (Notifier) hỗ trợ gửi thông báo không đồng bộ qua Slack, Email (SMTP) và Webhook tùy chỉnh; và **(vi)** toàn bộ hệ thống, gồm WAF, cơ sở dữ liệu PostgreSQL và ML service, được đóng gói thành một Docker Compose deployment hoàn chỉnh.

Đóng góp chính của đề tài bao gồm: **(a)** thiết kế và hiện thực hóa cơ chế anomaly scoring hai lớp Rule+ML đầu tiên trong không gian mã nguồn mở Go, với tính chất fail-open và tích hợp không xâm phạm (non-intrusive) vào hot path; **(b)** xây dựng bộ luật JSON có cấu trúc với transform chain chống evasion, kiểm thử và đánh giá trên 42 payload OWASP chuẩn; **(c)** xây dựng và đánh giá pipeline fine-tuning DistilBERT cho phân loại HTTP request 10 lớp với cơ chế hard gate đảm bảo chất lượng mô hình trước khi đưa vào triển khai; và **(d)** thiết kế interface MLPredictor tách biệt engine và ML client, cho phép thay thế backend ML mà không thay đổi logic engine.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 tập trung khảo sát hiện trạng và phân tích yêu cầu hệ thống. Nội dung

bao gồm khảo sát chi tiết nhu cầu của các nhóm người dùng mục tiêu (Security Engineer, DevOps, Developer), so sánh mở rộng các giải pháp WAF hiện có, đặc tả các ca sử dụng quan trọng bằng biểu đồ UML, và xác định đầy đủ yêu cầu chức năng lẫn phi chức năng làm cơ sở thiết kế cho các chương tiếp theo.

Trong Chương 3, đề án giới thiệu nền tảng lý thuyết và công nghệ được ứng dụng. Các chủ đề bao gồm kiến trúc và đặc tính của ngôn ngữ Go liên quan đến xây dựng proxy hiệu năng cao, mô hình anomaly scoring, cơ sở lý thuyết của kiến trúc Transformer và quá trình fine-tuning DistilBERT cho bài toán phân loại chuỗi, thuật toán Token Bucket cho rate limiting, và các công nghệ hạ tầng như PostgreSQL, JWT, Docker.

Chương 4 trình bày quá trình phân tích thiết kế, triển khai và đánh giá toàn bộ hệ thống. Đây là chương trọng tâm của đề án, bao gồm: kiến trúc tổng quan và luồng xử lý request xuyên suốt các lớp middleware; thiết kế chi tiết rule engine với cấu trúc dữ liệu, transform chain và cơ chế scoring; tích hợp ML layer và pipeline huấn luyện mô hình với cơ chế kiểm soát chất lượng; thiết kế cơ sở dữ liệu; xây dựng dashboard và API quản trị; và trình bày kết quả kiểm thử toàn diện trên các tập payload tấn công chuẩn, bài kiểm thử tải và kiểm thử bảo mật nội bộ.

Chương 5 tổng hợp và thảo luận sâu về bốn đóng góp nổi bật của đề tài: cơ chế anomaly scoring hai lớp với ML gray-zone activation; transform chain chống evasion và tập luật 78 rules; pipeline fine-tuning DistilBERT với hard gate kiểm soát chất lượng; và thiết kế interface tách biệt engine và ML client phục vụ khả năng kiểm thử và mở rộng.

Cuối cùng, Chương 6 đúc kết mức độ hoàn thành mục tiêu đề ra, nhìn nhận trực tiếp các hạn chế còn tồn tại của hệ thống, và đề xuất các hướng phát triển tiếp theo trong ngắn hạn, trung hạn và dài hạn, bao gồm việc hoàn thiện mô hình ML phiên bản tiếp theo, mở rộng sang inspection tầng response, và tích hợp cơ chế học chủ động từ phản hồi của đội ngũ bảo mật.

Kết chương. Chương này đã phác thảo toàn cảnh bài toán: sự gia tăng của tấn công tầng ứng dụng web, hạn chế cố hữu của WAF rule-based và các giải pháp đám mây hiện tại, khoảng trống còn chưa được lấp đầy trong không gian mã nguồn mở, và định hướng giải pháp hai lớp mà đề tài theo đuổi. Kiến trúc đề xuất kết hợp rule engine anomaly scoring viết bằng Go với mô hình DistilBERT fine-tuned kích hoạt có điều kiện, nhằm đạt được đồng thời hiệu năng cao ở đường nhanh và độ chính xác tăng cường ở vùng xám. Đây là nền tảng để Chương 2 đi sâu vào khảo sát và phân tích yêu cầu chi tiết cho từng thành phần của hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Trên cơ sở bài toán đã được đặt ra ở Chương 1.1 và định hướng giải pháp tổng quát ở phần 1.3, chương này đi sâu hơn vào hai khía cạnh cốt lõi của giai đoạn phân tích: *khảo sát hiện trạng* và *đặc tả yêu cầu hệ thống*. Phần khảo sát làm rõ chân dung người dùng mục tiêu, mở rộng so sánh các giải pháp WAF hiện hành ở mức độ chi tiết hơn so với khảo sát sơ bộ ở Chương 1, từ đó định hình kỳ vọng thực tế của các bên liên quan. Phần đặc tả tiến hành mô hình hóa hệ thống bằng các ca sử dụng (use case) tiêu biểu, đặc tả chi tiết các chức năng xử lý nghiệp vụ chính của WAF, và liệt kê đầy đủ tập yêu cầu phi chức năng làm cơ sở cho quá trình thiết kế và đánh giá ở các chương sau.

2.1 Khảo sát hiện trạng

2.1.1 Khảo sát nhu cầu người dùng

Một hệ thống WAF không phục vụ một nhóm người dùng duy nhất, mà giao tiếp đồng thời với ba nhóm có vai trò và kỳ vọng khác biệt rõ rệt. Việc xác định trước chân dung của từng nhóm là cơ sở để thiết kế các giao diện, mức phân quyền và tập chức năng phù hợp.

Kỹ sư bảo mật (Security Engineer) là nhóm tương tác trực tiếp nhất với WAF. Họ chịu trách nhiệm vận hành tập luật, đánh giá các cảnh báo và sự kiện phát sinh, cũng như tinh chỉnh ngưỡng phát hiện để cân bằng giữa độ nhạy và tỷ lệ báo nhầm. Họ cần một giao diện cho phép xem nhanh các sự kiện đang xảy ra, truy xuất nhật ký chi tiết của một request đáng ngờ (bao gồm payload gốc, các rule đã khớp, kết quả phân tích của mô hình ML nếu có), và khả năng can thiệp tức thì như tạm chặn IP, vô hiệu hóa hoặc bật lại một rule cụ thể mà không cần khởi động lại dịch vụ. Đối với họ, độ tin cậy của quyết định mà WAF đưa ra phải được giải thích được: một cảnh báo *block* cần phải đi kèm bằng chứng cụ thể (rule nào, điểm anomaly là bao nhiêu, ML verdict nói gì), nếu không, công cụ sẽ bị nghi ngờ và dần bị bỏ qua.

Kỹ sư vận hành hệ thống và phát triển (DevOps/SRE) không trực tiếp phân tích các sự cố bảo mật, nhưng chịu trách nhiệm về sức khỏe vận hành tổng thể: tỷ lệ uptime, độ trễ thêm vào do WAF, mức tiêu thụ tài nguyên, khả năng mở rộng theo lưu lượng tăng. Nhóm này coi WAF như một thành phần trong stack hạ tầng và quan tâm đến những đặc tính phi chức năng: khả năng triển khai bằng một lệnh duy nhất (Docker Compose), khả năng cấu hình thông qua tệp văn bản được kiểm soát phiên bản (declarative configuration), khả năng quan sát hệ thống qua metric tiêu chuẩn (Prometheus, OpenTelemetry), và khả năng thực hiện *rolling update* mà không gây gián đoạn dịch vụ. Đặc biệt, họ đặc biệt nhạy cảm với hành vi *fail-open*

– nếu một thành phần phụ trợ (ví dụ ML service) gặp sự cố, hệ thống không được đẩy traffic hợp lệ vào trạng thái chặn nhầm.

Lập trình viên ứng dụng (Developer) là nhóm gián tiếp tương tác với WAF: họ cần biết khi nào và tại sao một yêu cầu hợp lệ của ứng dụng bị WAF từ chối, để điều chỉnh ứng dụng hoặc bổ sung ngoại lệ phù hợp. Đây là nhóm phát sinh kỳ vọng cụ thể về khả năng *exception per-rule* (cho phép loại trừ một số đường dẫn, header, hay tham số nhất định khỏi phạm vi áp dụng của rule) và cơ chế kiểm thử ngược (replay request) để xác minh hành vi của WAF khi trao đổi với đội ngũ bảo mật.

Quá trình tham khảo các bài viết kinh nghiệm vận hành ModSecurity, các báo cáo của OWASP về tối ưu Core Rule Set, cùng với các thảo luận trong cộng đồng DevSecOps cho thấy ba kỳ vọng dùng chung của cả ba nhóm: (1) độ chính xác cao trên cả hai chiều – bắt được tấn công thật và không chặn nhầm traffic hợp lệ; (2) khả năng quan sát và giải thích (observability) chi tiết với độ trễ thấp; và (3) chi phí vận hành thấp, không yêu cầu kỹ năng quá chuyên biệt để duy trì. Ba kỳ vọng này trở thành kim chỉ nam xuyên suốt trong quá trình thiết kế các thành phần của hệ thống.

2.1.2 Khảo sát các hệ thống WAF hiện có

Chương 1 đã đề cập ngắn gọn đến ba lớp giải pháp WAF: mã nguồn mở triển khai tại chỗ (ModSecurity, Naxsi, Shadow Daemon), thương mại trên nền tảng đám mây (Cloudflare WAF, AWS WAF), và các giải pháp tích hợp sẵn trong reverse proxy (NGINX App Protect). Phần này đi sâu hơn ở góc độ so sánh đặc tính kỹ thuật và phân tích khoảng trống mà đề tài hướng đến lấp đầy.

ModSecurity kết hợp OWASP Core Rule Set (CRS) là tham chiếu thực tế trong không gian WAF mã nguồn mở. Mô hình xử lý của ModSecurity dựa trên năm pha (connection, request headers, request body, response headers, response body), trong đó CRS đóng góp khoảng hơn 200 rule chia thành các nhóm tương ứng với các nhóm tấn công OWASP. Điểm mạnh nổi bật là độ phổ rộng và tính được kiểm chứng qua thời gian. Tuy nhiên, qua khảo sát các bài viết vận hành thực tế và trải nghiệm cá nhân khi thử nghiệm trên ứng dụng DVWA, có thể nhận diện một số hạn chế cụ thể: bộ luật mặc định tạo ra tỷ lệ false positive đáng kể trên ứng dụng có payload JSON phức tạp, đòi hỏi giai đoạn tinh chỉnh (*learning mode*) kéo dài; kiến trúc module nhúng vào web server (Apache, NGINX) làm cho việc reload luật mà không restart dịch vụ tương đối phức tạp; và ModSecurity hoàn toàn không có giao diện quản trị tích hợp, buộc đội ngũ bảo mật phải xây dựng lớp giao diện riêng hoặc tích hợp với các nền tảng SIEM bên thứ ba.

Cloudflare WAF và AWS WAF đại diện cho hướng tiếp cận *managed service*. Hai giải pháp này giải quyết tốt bài toán cập nhật luật và mở rộng theo tải, đồng thời tích hợp sẵn các tính năng phụ trợ như chống DDoS volumetric và mạng phân phối nội dung. Đặc biệt, Cloudflare cung cấp các ML-based rule học từ lưu lượng toàn cầu, một lợi thế khó có với giải pháp on-premise. Nhược điểm chính là chúng yêu cầu định tuyến toàn bộ lưu lượng ứng dụng qua hạ tầng nhà cung cấp – một ràng buộc không tương thích với các tổ chức có yêu cầu kiểm soát dữ liệu nghiêm ngặt hoặc các môi trường ngắt mạng (air-gapped). Bên cạnh đó, mô hình tính phí theo lưu lượng và rule khiến chi phí trở nên khó dự đoán, và bản chất “hộp đen” khiến đội ngũ bảo mật mất khả năng kiểm tra logic phát hiện ở mức chi tiết.

Các giải pháp ít phổ biến hơn bao gồm Naxsi (module Nginx, dựa trên whitelist với rule trắng-ký-tự nghiêm ngặt), Shadow Daemon (kiến trúc daemon độc lập, không được bảo trì tích cực sau 2020), và một số module thương mại của F5 (NG-1NX App Protect) hay Imperva. Các giải pháp này hoặc thiếu giao diện quản trị, hoặc bộ luật không phong phú, hoặc bị ràng buộc bởi giấy phép thương mại và không đáp ứng tiêu chí mã nguồn mở của đề tài.

Bảng 2.1 tổng hợp so sánh các tiêu chí kỹ thuật quan trọng nhất giữa các giải pháp khảo sát và hệ thống do đề tài đề xuất.

Bảng 2.1: So sánh kỹ thuật các giải pháp WAF khảo sát

Tiêu chí	ModSecurity + CRS	Cloudflare / AWS WAF	Naxsi / Shadow Daemon	Hệ thống đề xuất
Triển khai on-premise	Có	Không	Có	Có
Mã nguồn mở	Có	Không	Có	Có
Ngôn ngữ thực thi	C (module)	Closed	C (module)	Go
Tích hợp ML	Không	Có (đóng)	Không	Có (DistilBERT, gray-zone)
Anomaly scoring	Có	Một phần	Không	Có (đa lớp)
Dashboard tích hợp	Không	Có	Không	Có (web UI native)
RBAC nội bộ	Không	Có	Không	Có (3 vai trò)
Cấu hình rule động	Hạn chế	Có	Hạn chế	Có (hot-reload)
Cảnh báo đa kênh	Qua bên thứ ba	Có	Không	Có (Slack/SMTP/Webhook)

Từ so sánh trên có thể khẳng định rõ hơn khoảng trống mà đề tài nhắm tới: một WAF mã nguồn mở, triển khai tại chỗ, tích hợp đầy đủ cả ba thành phần thường

thấy rời rạc trong thực tiễn – engine luật anomaly scoring, lớp ML kiểm chứng vùng xám, và một mặt phẳng quản trị (control plane) thống nhất qua dashboard web cùng cơ chế xác thực và phân quyền nội bộ.

2.2 Tổng quan chức năng

2.2.1 Biểu đồ ca sử dụng tổng quát

Hệ thống WAF tương tác với bốn nhóm actor: **End User** – người dùng cuối gửi request hợp lệ đến ứng dụng đích; **Attacker** – chủ thể có chủ đích khai thác lỗ hổng; **Admin** – quản trị viên cấp cao có toàn quyền cấu hình hệ thống và quản trị tài khoản người dùng nội bộ; và **ML Service** – một actor hệ thống (system actor) đại diện cho microservice học máy bên ngoài. Ngoài ra, hệ thống còn có **Editor** – vai trò có quyền thay đổi cấu hình WAF (rule, IP list, settings) nhưng không quản trị tài khoản người khác; và **Viewer** – vai trò chỉ đọc dashboard và nhật ký.

Hình 2.1 mô tả biểu đồ ca sử dụng tổng quát. Các ca sử dụng được phân thành bốn cụm: (i) cụm *xử lý lưu lượng* (request filtering, ML gray-zone confirm, rate limiting); (ii) cụm *quản lý cấu hình bảo mật* (rule management, IP management, system settings); (iii) cụm *quan sát hệ thống* (dashboard real-time, audit log query, security alert); và (iv) cụm *quản trị nội bộ* (authentication, account settings, user management). Để tránh các đường nối chồng chéo, biểu đồ chia actor thành hai phía: một phía là các tác nhân tham gia đường nhanh xử lý request (End User, Attacker, và ML Service đóng vai trò actor hệ thống), phía còn lại là ba vai trò quản trị (Viewer, Editor, Admin) được tổ chức theo quan hệ kế thừa (Admin → Editor → Viewer) nhằm giảm số đường nối phải vẽ và làm nổi bật cấu trúc phân quyền RBAC.

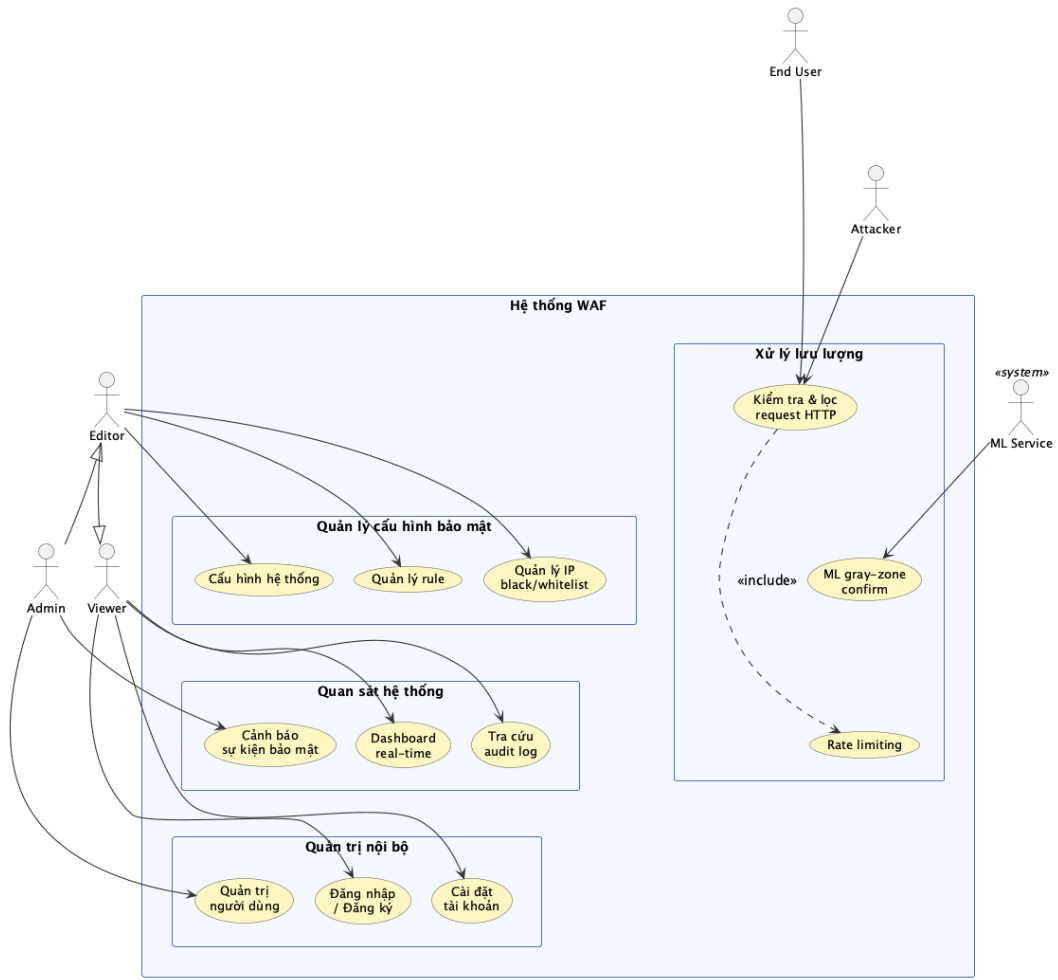
Các ca sử dụng tiêu biểu được đặc tả chi tiết trong các tiểu mục tiếp theo. Đối với mỗi ca sử dụng, đề tài trình bày bốn yếu tố cốt lõi: *actor* chính và phụ, *tiền điều kiện* (precondition), *luồng chính* (main flow) và *luồng thay thế* (alternative flow), nhằm phục vụ trực tiếp giai đoạn thiết kế ở Chương 4.

2.2.2 Ca sử dụng: Kiểm tra và lọc request HTTP

Actor chính: End User (vô tình) và Attacker (cố ý) – cả hai không biết sự tồn tại của WAF do hệ thống vận hành ở chế độ reverse proxy trong suốt.

Tiền điều kiện: WAF đã được khởi động, kết nối tới ứng dụng backend đã hoạt động, các rule đã được nạp vào engine.

Luồng chính: Khi một request HTTP đến WAF, hệ thống lần lượt thực hiện các bước sau. Đầu tiên, parser bóc tách bốn thành phần của request: phương thức, đường dẫn, các tham số query, header và body. Tiếp theo, normalizer áp dụng chuỗi

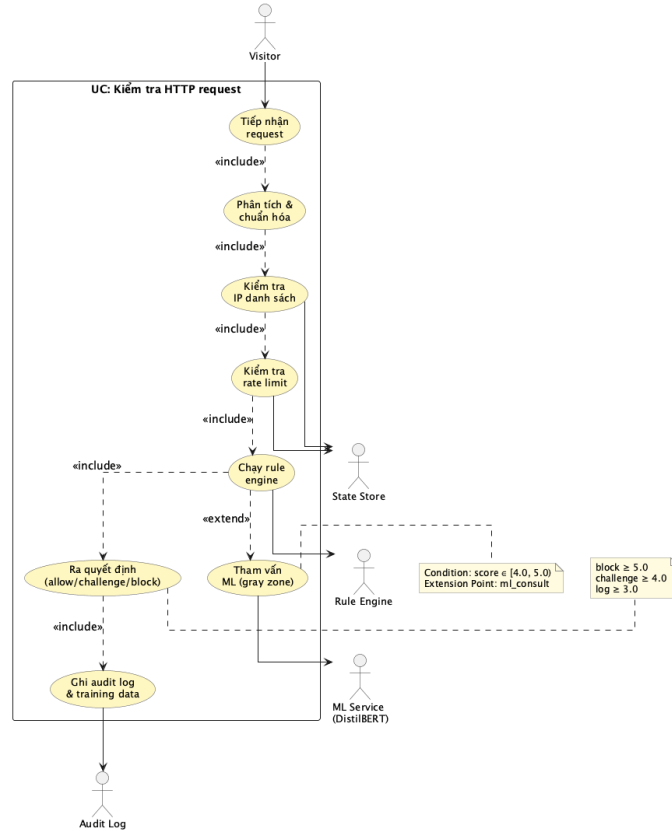


Hình 2.1: Biểu đồ ca sử dụng tổng quát của hệ thống WAF

biến đổi tiền xử lý (URL decode, lower-casing, compress whitespace) để chuẩn hóa văn bản trước khi so khớp – bước này quan trọng để chống các kỹ thuật làm mờ. Sau đó, hệ thống kiểm tra IP nguồn: nếu thuộc blacklist động, request bị từ chối ngay lập tức; nếu thuộc whitelist, request được bỏ qua phần lớn kiểm tra. Tiếp đến, bộ rate limiter kiểm tra số lượng token còn lại trong bucket của IP nguồn – nếu hết token, request bị giới hạn. Cuối cùng, rule engine duyệt tuần tự qua 78 rule, mỗi rule khớp đóng góp một giá trị vào tổng điểm anomaly. Dựa trên điểm tổng và các ngưỡng cấu hình động (log/challenge/block), hệ thống đưa ra quyết định: ALLOW (chuyển tiếp đến backend), CHALLENGE (yêu cầu xác thực bổ sung), hoặc BLOCK (trả về 403). Đồng thời, một bản ghi audit log được ghi lại bất kể quyết định nào.

Luồng thay thế: Nếu điểm anomaly rơi vào “vùng xám” [3.0, 5.0), WAF kích hoạt nhánh ML confirm. Engine gửi canonical text của request đến ML service (FastAPI) qua HTTP POST với timeout 800 mili giây. Nếu ML phán quyết là một trong các nhãn tấn công với độ tin cậy cao, một giá trị delta dương được cộng vào điểm, đẩy quyết định về phía block/challenge. Ngược lại, nếu ML phán quyết là

normal với độ tin cậy cao, delta âm được cộng vào, đẩy quyết định về phía allow. Trong trường hợp ML service không phản hồi trong khung thời gian cho phép hoặc trả về lỗi, hệ thống chuyển sang chế độ *fail-open* – giữ nguyên điểm rule và quyết định dựa trên điểm gốc, không gây gián đoạn xử lý chính.



Hình 2.2: Biểu đồ ca sử dụng phân rã – Kiểm tra và lọc request HTTP

2.2.3 Ca sử dụng: Quản lý luật (Rule Management)

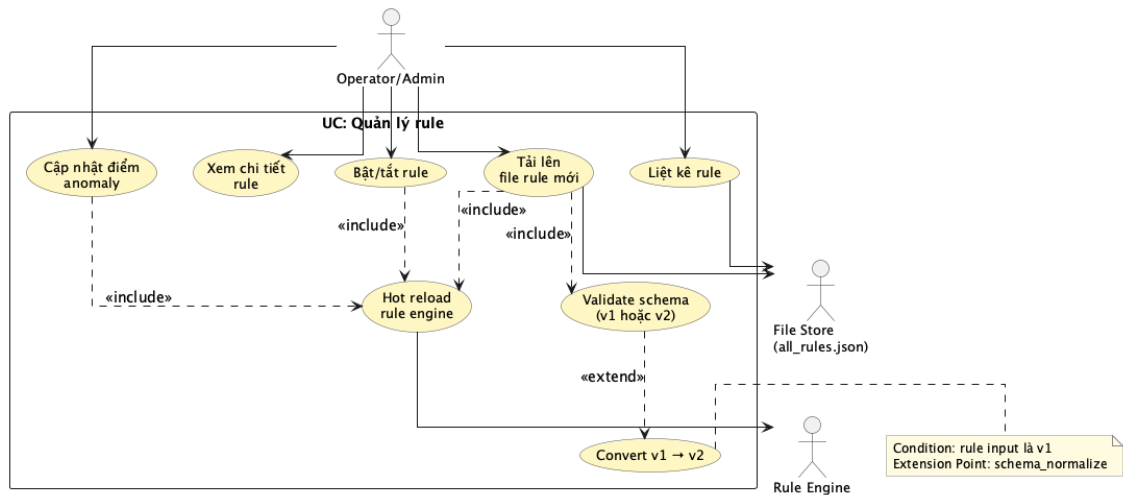
Actor chính: Admin (toàn quyền) và Editor (được phép sửa rule).

Tiền điều kiện: Người dùng đã đăng nhập thành công và có token JWT hợp lệ với role phù hợp; định danh role được middleware xác thực kiểm tra trước khi chuyển yêu cầu đến handler.

Luồng chính: Người dùng mở trang Rules từ dashboard. Hệ thống hiển thị toàn bộ 78 rule kèm metadata (ID, category, severity, anomaly score, trạng thái bật/tắt, hit count tích lũy). Người dùng có thể lọc theo nhóm tấn công hoặc mức độ nghiêm trọng, bật/tắt một rule cụ thể (thay đổi có hiệu lực ngay mà không cần restart), xem chi tiết pattern matching của rule, và tải file JSON định nghĩa rule mới qua endpoint admin để bổ sung hoặc thay thế tập luật hiện hành.

Luồng thay thế: Khi tải file rule mới, hệ thống thực hiện validation strict (kiểm tra schema, biên dịch trước các regex pattern) trước khi áp dụng. Nếu file không hợp lệ, hệ thống từ chối và giữ nguyên tập luật cũ, đồng thời trả về thông báo lỗi

chi tiết. Mọi thay đổi tập luật đều được ghi audit log với định danh người thực hiện.



Hình 2.3: Biểu đồ ca sử dụng phân rã – Quản lý luật

2.2.4 Ca sử dụng: Giám sát và xem log

Actor chính: Admin, Editor và Viewer.

Tiền điều kiện: Người dùng đã đăng nhập thành công.

Luồng chính: Trang Dashboard hiển thị các counter thời gian thực bao gồm requests/giây, block rate, top attack categories trong cửa sổ thời gian gần nhất, và phân bố quyết định (ALLOW/CHALLENGE/BLOCK). Người dùng có thể chuyển sang trang Logs để xem chi tiết các bản ghi audit, áp dụng bộ lọc theo khoảng thời gian, IP, loại tấn công, hay quyết định. Mỗi bản ghi log cho phép mở rộng để xem chi tiết: payload gốc đã được làm sạch (sanitized), các rule đã khớp, điểm anomaly cuối cùng, và nếu có, phán quyết của ML cùng độ tin cậy.

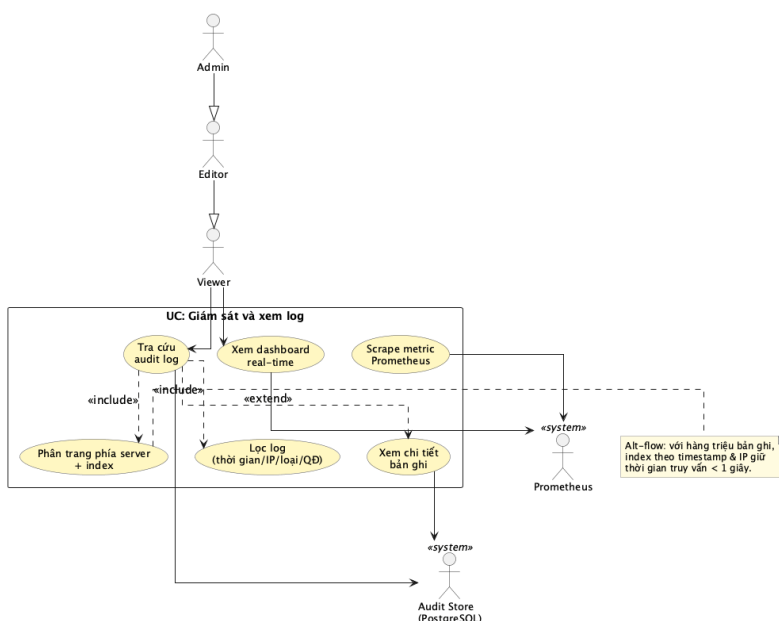
Luồng thay thế: Khi cơ sở dữ liệu chứa hàng triệu bản ghi, hệ thống áp dụng phân trang phía server và đánh chỉ mục theo timestamp và IP để duy trì thời gian phản hồi truy vấn dưới một giây.

2.2.5 Ca sử dụng: Quản lý IP (Blacklist / Whitelist)

Actor chính: Admin và Editor.

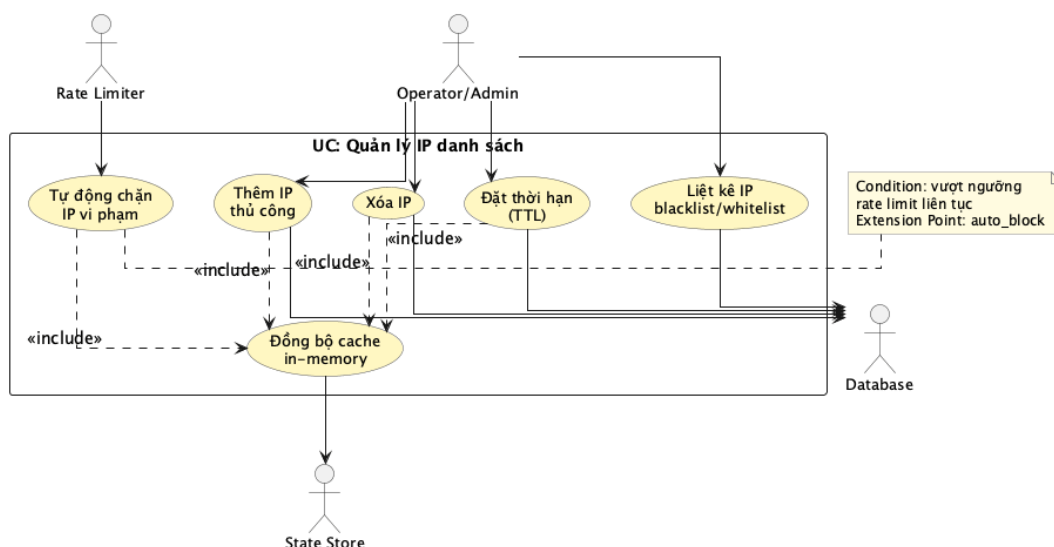
Luồng chính: Trang IP Management cho phép thêm địa chỉ IP hoặc dải CIDR vào blacklist (chặn hoàn toàn) hoặc whitelist (bỏ qua kiểm tra), với tùy chọn TTL (time-to-live) cho phép tự động xóa sau một khoảng thời gian. Người dùng có thể xem danh sách IP đang bị giới hạn tốc độ bởi rate limiter, danh sách IP bị chặn động do vi phạm liên tục, và xóa thủ công các mục khỏi danh sách.

Luồng thay thế: Khi rate limiter phát hiện một IP vượt quá ngưỡng cho phép liên tục trong một khoảng thời gian, IP đó được thêm tự động vào danh sách chặn



Hình 2.4: Biểu đồ ca sử dụng phân rã – Giám sát và xem log

tạm thời với TTL mặc định cấu hình được. Người dùng admin có thể can thiệp thủ công để gỡ chặn sớm hoặc kéo dài thời gian chặn.



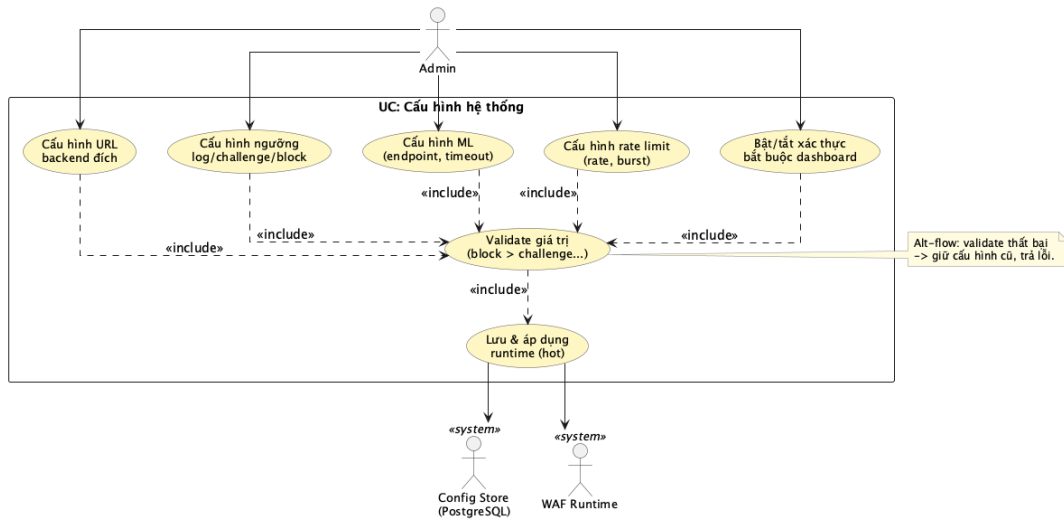
Hình 2.5: Biểu đồ ca sử dụng phân rã – Quản lý IP blacklist/whitelist

2.2.6 Ca sử dụng: Cấu hình hệ thống

Actor chính: Admin.

Luồng chính: Trang System Settings cho phép admin cấu hình các tham số vận hành cốt lõi: URL backend đích, các ngưỡng quyết định (log/challenge/block), endpoint và timeout của ML service, các tham số rate limiting (rate, burst), và bật/tắt yêu cầu xác thực bắt buộc cho dashboard. Thay đổi có hiệu lực ngay sau khi lưu, không cần khởi động lại proxy.

Luồng thay thế: Hệ thống validate các giá trị nhập vào trước khi áp dụng (ví dụ ngưỡng block phải lớn hơn ngưỡng challenge, URL backend phải reachable). Nếu validate thất bại, cấu hình cũ được giữ nguyên và trả về thông báo lỗi.



Hình 2.6: Biểu đồ ca sử dụng phân rã – Cấu hình hệ thống

2.2.7 Ca sử dụng: Cảnh báo sự kiện bảo mật (Alert / Notification)

Actor chính: Admin (cấu hình kênh và xem stats); Notifier worker (actor hệ thống chạy nền) là thực thể thực hiện việc gửi.

Trigger: Decision layer phát hiện một sự kiện có mức độ nghiêm trọng lớn hơn hoặc bằng ngưỡng cấu hình (mặc định HIGH).

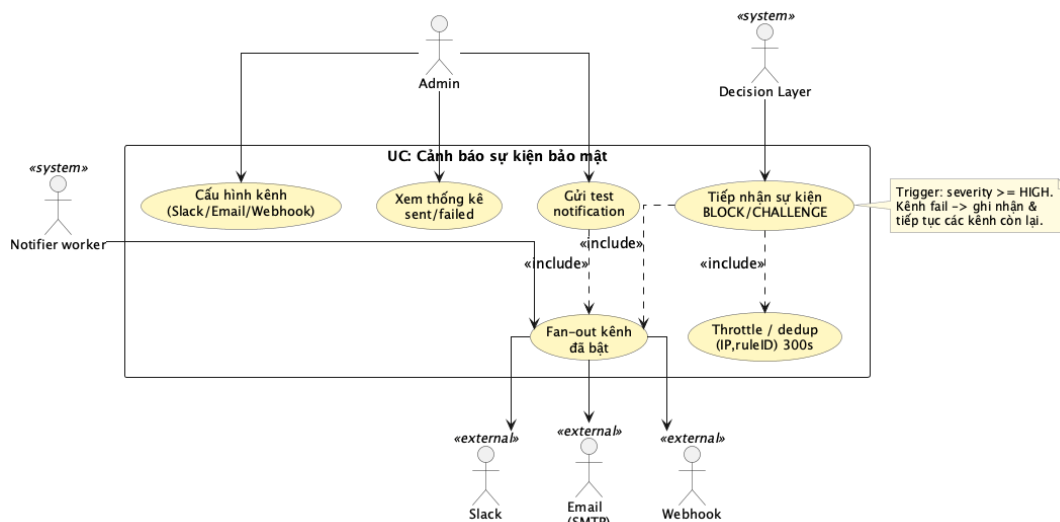
Luồng chính: Khi engine đưa ra quyết định BLOCK hoặc CHALLENGE cho một request, một sự kiện được đẩy không đồng bộ đến notifier qua kênh trong bộ nhớ. Notifier worker chạy nền sẽ kiểm tra bộ lọc throttle để loại bỏ các sự kiện trùng lặp (dedup theo cặp (IP, ruleID) trong cửa sổ 300 giây mặc định), sau đó fan-out sự kiện đến các kênh đã được kích hoạt: Slack qua Incoming Webhook, Email qua SMTP, và Generic Webhook (HTTP POST với payload JSON tùy biến). Mỗi lần gửi được giới hạn timeout 5 giây và bộ đếm sent/failed được cập nhật để phục vụ giám sát.

Luồng thay thế: Nếu một kênh fail (timeout, lỗi kết nối, mã trả về phi-2xx), notifier ghi nhận thất bại và tiếp tục thử các kênh còn lại – không ngừng toàn bộ quá trình. Admin có thể gọi endpoint test notification để gửi một sự kiện thử và nhận về kết quả từng kênh, phục vụ kiểm tra cấu hình ban đầu.

2.2.8 Ca sử dụng: Xác thực và đăng ký tài khoản

Actor chính: End user nội bộ (Security Engineer, DevOps, Developer).

Luồng chính – Đăng nhập: Người dùng nhập tên đăng nhập và mật khẩu trên



Hình 2.7: Biểu đồ ca sử dụng phân rã – Cảnh báo sự kiện bảo mật

trang Login. Hệ thống xác thực thông tin bằng cách so sánh hash bcrypt (cost factor 10) lưu trong cơ sở dữ liệu. Nếu hợp lệ, hệ thống phát hành một JWT (HS256) chứa định danh người dùng và vai trò, đặt token vào HttpOnly cookie (chống đánh cắp qua XSS) và đồng thời cho phép sử dụng qua *Bearer header* với các API client.

Luồng chính – Đăng ký: Đề tài cho phép tự đăng ký công khai để hỗ trợ thử nghiệm và demo nội bộ, tuy nhiên endpoint công khai `POST /register` **chỉ tạo được tài khoản với vai trò viewer**. Trường `role` không được chấp nhận từ phía client để bịt lỗ leo thang đặc quyền (privilege escalation). Việc nâng vai trò người dùng (lên `editor` hoặc `admin`) chỉ được thực hiện qua endpoint quản trị tách biệt mô tả ở tiểu mục 2.2.10.

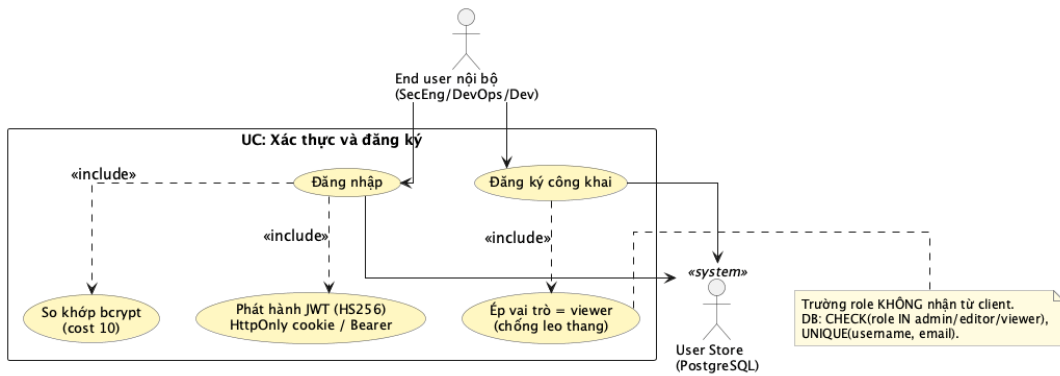
Vai trò và phạm vi quyền: Hệ thống định nghĩa ba vai trò có cấp độ quyền lực giảm dần.

- **admin** – toàn quyền: cấu hình WAF, quản lý rule và IP list, quản trị tài khoản người dùng (CRUD), reset mật khẩu cho người khác, cấu hình kênh cảnh báo.
- **editor** – quyền cấu hình WAF (rule, IP list, system settings, notification channels) nhưng **không** có quyền tạo, sửa, xóa tài khoản người dùng khác.
- **viewer** – chỉ đọc dashboard, audit log, các trang cấu hình ở chế độ xem.

Tiền điều kiện ngầm định: Cơ sở dữ liệu áp đặt ràng buộc CHECK (`role IN ('admin','editor','viewer')`) ở mức schema và UNIQUE trên cả `username` lẫn `email` để loại trừ trùng lặp ngay tại tầng lưu trữ.

2.2.9 Ca sử dụng: Quản lý tài khoản cá nhân (Account Settings)

Actor chính: Mọi người dùng đã đăng nhập, không phụ thuộc vai trò.



Hình 2.8: Biểu đồ ca sử dụng phân rã – Xác thực và đăng ký tài khoản

Luồng chính – Cập nhật thông tin cá nhân: Người dùng mở trang `settings.html`, nơi hiển thị username và role ở chế độ chỉ đọc (vì hai trường này không phải là quyết định của bản thân người dùng) cùng email ở chế độ chỉnh sửa được. Khi cập nhật email, hệ thống gọi `PUT /waf-api/auth/me` với JWT của người dùng – middleware `requireAuthN` xác thực JWT trước khi chuyển đến handler.

Luồng chính – Đổi mật khẩu: Người dùng nhập mật khẩu cũ và mật khẩu mới (với độ dài tối thiểu 8 ký tự được kiểm tra cả ở client và server). Endpoint `PUT /waf-api/auth/me/password` so sánh hash của mật khẩu cũ trước khi cập nhật. Việc *bắt buộc nhập lại mật khẩu cũ* đóng vai trò phòng thủ chiều sâu: ngay cả khi attacker chiếm được JWT (ví dụ qua máy trạm bị compromise), họ vẫn không thể thay đổi mật khẩu của nạn nhân nếu không biết mật khẩu cũ.

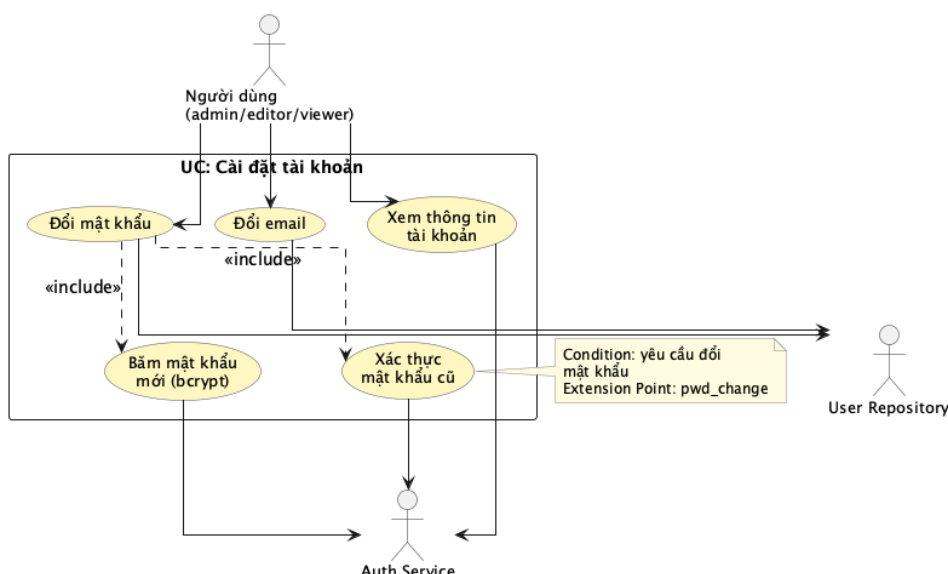
Luồng thay thế: Nếu mật khẩu cũ không khớp, hệ thống trả mã trạng thái 401 và không thay đổi gì. Mọi lần đổi mật khẩu thành công đều được ghi audit log với định danh người thực hiện.

2.2.10 Ca sử dụng: Quản trị người dùng (Admin User Management)

Actor chính: Admin (duy nhất).

Tiền điều kiện: JWT của người gọi chứa `role = "admin"`; middleware `requireAdmin` kiểm tra ràng buộc này trước khi chuyển đến handler.

Luồng chính: Admin mở trang `users.html`. Hệ thống liệt kê toàn bộ tài khoản trong cơ sở dữ liệu kèm vai trò, ngày tạo, và lần cập nhật gần nhất. Admin có thể: (i) tạo người dùng mới với vai trò bất kỳ trong ba vai trò qua `POST /waf-api/auth/users`; (ii) cập nhật email hoặc thăng/giáng vai trò qua `PUT /waf-api/auth/users/{id}`; (iii) reset mật khẩu của người khác (không cần mật khẩu cũ) qua `PUT /waf-api/auth/users/{id}/password`; và (iv) xóa người dùng qua `DELETE /waf-api/auth/users/{id}`.



Hình 2.9: Biểu đồ ca sử dụng phân rã – Quản lý tài khoản cá nhân

Luồng thay thế – Các bất biến (invariant) phải được bảo đảm: Đây là phần quan trọng nhất của ca sử dụng này, vì các thao tác quản trị tài khoản có khả năng tạo ra trạng thái không thể phục hồi (irrecoverable state).

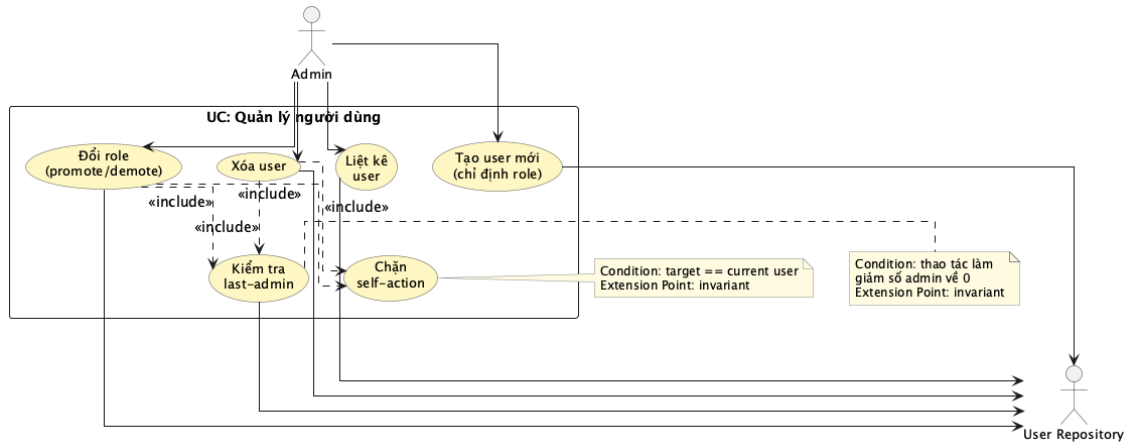
- **Last-admin protection:** Trước khi xóa hoặc demote bất kỳ tài khoản admin nào, hệ thống gọi `CountByRole("admin")` để đếm số admin còn lại. Nếu `count = 1` và admin đó chính là target, thao tác bị từ chối với mã 400 và thông điệp "cannot delete/demote the last admin". Bất biến này ngăn chặn tình huống hệ thống không còn admin nào (lockout vĩnh viễn).
- **Self-action protection:** Admin không được xóa hoặc demote chính tài khoản của mình. Bất biến này được kiểm tra cả ở tầng handler (so sánh `userID` từ JWT với `targetID` trên đường dẫn) và ở tầng giao diện (nút Delete bị disable cho hàng tương ứng với chính admin đang đăng nhập, theo nguyên tắc defense-in-depth).
- **Audit logging bắt buộc:** Mọi thao tác (create, update, reset password, delete) đều được ghi audit log với cặp (actor, target, action, timestamp) qua module `auditLogger.LogSystemEvent`.

2.3 Đặc tả chức năng

2.3.1 Đặc tả: Kiểm tra request và đưa ra quyết định

Input: Một đối tượng `ParsedRequest` chứa năm trường đã được chuẩn hóa: `method`, `path`, `query parameters` (key-value map), `headers` (multimap), và `body` (giới hạn kích thước theo cấu hình, mặc định 10 MB).

Xử lý: Engine duyệt tuần tự qua tập rule đã nạp. Với mỗi rule, hệ thống đánh giá các *conditions* trên các trường liên quan của `ParsedRequest`, áp dụng trans-



Hình 2.10: Biểu đồ ca sử dụng phân rã – Quản trị người dùng

form chain riêng của rule, và so khớp các pattern (REGEX hoặc TOKEN với proximity). Nếu rule khớp, đóng góp một giá trị $\text{anomaly_score} \times \text{severity_multiplier}$ vào tổng điểm. Sau khi duyệt hết, nếu tổng điểm rơi vào vùng xám $[3.0, 5.0)$, engine gọi ML hook để điều chỉnh điểm. Cuối cùng, quyết định được đưa ra bằng cách so sánh điểm tổng với ba ngưỡng cấu hình động.

Output: Một bộ ba ($\text{decision}, \text{score}, \text{matched_rules}$) trong đó decision thuộc tập $\{\text{ALLOW}, \text{LOG}, \text{CHALLENGE}, \text{BLOCK}\}$, đi kèm với một entry audit log đầy đủ.

Ngưỡng quyết định mặc định: $\text{LOG} \geq 3.0$, $\text{CHALLENGE} \geq 4.0$, $\text{BLOCK} \geq 5.0$. Các ngưỡng này có thể được điều chỉnh trong runtime qua trang System Settings.

Quy trình tổng thể từ lúc tiếp nhận request đến khi ra quyết định và ghi log được thể hiện trên biểu đồ hoạt động ở Hình 2.11.

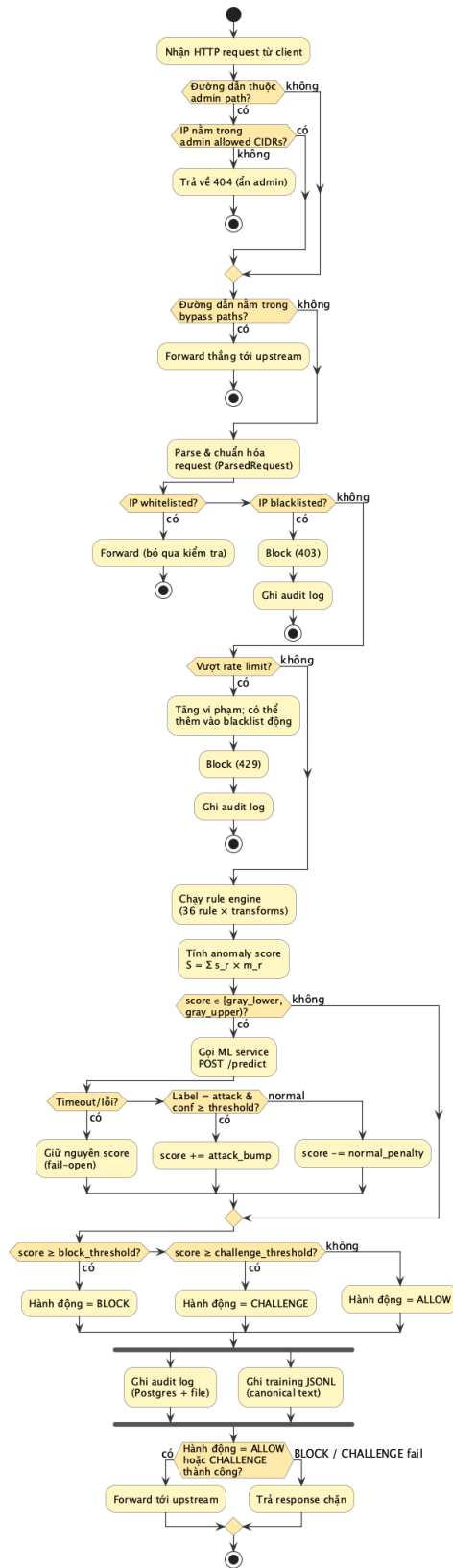
2.3.2 Đặc tả: Xử lý anomaly scoring và ML gray-zone confirm

Mô hình scoring: Điểm anomaly tổng của một request là tổng có trọng số của các điểm thành phần do từng rule match đóng góp. Công thức tổng quát:

$$S = \sum_{r \in R_{\text{matched}}} s_r \cdot m_r$$

trong đó s_r là điểm gốc của rule r (định nghĩa trong file JSON) và m_r là hệ số nhân theo mức độ nghiêm trọng (severity multiplier: CRITICAL = 2.0, HIGH = 1.5, MEDIUM = 1.0, LOW = 0.5, INFO = 0.0).

Vùng xám và ML confirm: Khi $S \in [3.0, 5.0)$, engine gọi `runMLConfirm` với canonical text của request. ML service trả về cặp (`label`, `confidence`).



Hình 2.11: Biểu đồ hoạt động – Quy trình kiểm tra request và ra quyết định

Nếu `label` thuộc tập nhãn tấn công và `confidence` $\geq$ ngưỡng cấu hình (mặc định 0.8), $S \leftarrow S + \Delta_+$ với $\Delta_+ = 2.0$; nếu `label` là `normal` với `confidence` cao, $S \leftarrow S - \Delta_-$ với $\Delta_- = 2.0$; nếu không thỏa hai trường hợp trên, S giữ

nguyên.

Cơ chế fail-safe: Gọi ML có timeout cứng 800 mili giây. Nếu timeout hoặc service trả lỗi, engine không điều chỉnh điểm và tiếp tục quyết định dựa trên điểm rule gốc – đây là chính sách *fail-open* đảm bảo ML không phải là điểm hỏng đơn (single point of failure) của hệ thống.

2.3.3 Đặc tả: Rate Limiting

Thuật toán: Token Bucket per IP với hai tham số `rate` (tokens/giây) và `burst` (kích thước bucket). Khi một request đến từ IP nào đó, bucket của IP đó được rót đầy theo thời gian trôi qua từ lần cập nhật trước, sau đó trừ một token. Nếu bucket trống, request bị từ chối với mã trạng thái 429.

Quản lý trạng thái: Bucket được lưu trong bản đồ in-memory được bảo vệ bởi `sync.Map` để cho phép truy cập concurrent. Một goroutine định kỳ quét và loại bỏ các bucket không hoạt động lâu (idle TTL mặc định 10 phút) để ngăn rò rỉ bộ nhớ với traffic có IP space lớn.

Tích hợp: Rate limit middleware được đặt sau IP-check middleware và trước WAF rule engine middleware. IP nào vi phạm liên tục N lần trong cửa sổ W được tự động bổ sung vào blacklist động với TTL ngắn (mặc định 1 giờ).

2.3.4 Đặc tả: Cảnh báo sự kiện bảo mật

Các kênh được hỗ trợ: Slack qua Incoming Webhook (payload JSON theo chuẩn Slack), Email qua SMTP với template cấu hình được, và Generic Webhook (HTTP POST với payload JSON do người dùng định nghĩa).

Cấu hình per-destination: Mỗi destination có các trường: `enabled` (flag bật/tắt), URL hoặc thông tin kết nối, template payload, danh sách kênh áp dụng severity filter, và các tham số throttle.

Throttle: Cơ chế dedup mặc định 300 giây – một sự kiện với cùng cặp (`channel`, `IP`, `ruleID`) không được gửi lặp lại trong cửa sổ này. Throttle được hiện thực bằng một LRU cache giới hạn kích thước.

Bộ lọc severity: Sự kiện có severity thấp hơn `MinSeverity` cấu hình (mặc định HIGH; thứ tự INFO < LOW < MEDIUM < HIGH < CRITICAL) bị bỏ qua hoàn toàn.

Tính chất async: Hàm `notifier.Send()` trả về ngay lập tức sau khi đẩy sự kiện vào kênh trong bộ nhớ (buffered channel). Worker goroutine chạy nền tiêu thụ kênh này, áp dụng throttle, fan-out đến các destination, và cập nhật bộ đếm sent/failed. Thiết kế này đảm bảo việc gửi cảnh báo không bao giờ làm tăng độ trễ

của đường nhanh xử lý request.

Test destination: Admin có thể gọi endpoint `POST /notifications/test` để gửi một sự kiện thử đến một destination cụ thể và nhận về kết quả chi tiết, hỗ trợ kiểm tra cấu hình ban đầu.

2.3.5 Đặc tả: Audit Logging

Phạm vi ghi log: Audit log lưu lại hai loại sự kiện. Thứ nhất là *request event* cho mọi request được flag (rule match với điểm $\geq$ LOG threshold), bao gồm các trường: timestamp, IP, method, path, score, decision, danh sách rule ID đã khớp, ML label và confidence (nếu có gọi ML), và một bản tóm tắt payload đã sanitize. Thứ hai là *system event* cho các thao tác quản trị nhạy cảm: đăng nhập thành công/thất bại, tạo/sửa/xóa user, đổi mật khẩu, thay đổi cấu hình WAF, bật/tắt rule, thêm/xóa IP khỏi blacklist.

Lưu trữ: Bảng `audit_logs` trong PostgreSQL, có chỉ mục trên timestamp và ip. Truy vấn lọc/phân trang qua các endpoint `GET /logs` hỗ trợ tham số khoảng thời gian, IP, danh mục tấn công.

Chính sách lưu giữ: Mặc định giữ 90 ngày, có thể cấu hình. Một job định kỳ (cron) xóa bản ghi quá hạn để kiểm soát kích thước bảng.

2.4 Yêu cầu phi chức năng

Trên cơ sở khảo sát ở mục 2.1 và các đặc tả chức năng ở mục 2.3, đề tài xác định bốn nhóm yêu cầu phi chức năng làm chuẩn mực đo lường cho hệ thống.

Hiệu năng. Độ trễ thêm vào do WAF, đo ở phân vị 99 (P99) trên traffic thông thường (không kích hoạt ML), phải dưới 5 mili giây. Khi nhánh ML được kích hoạt, tổng độ trễ thêm vào không quá 800 mili giây nhờ timeout cứng ở tầng client. Throughput mục tiêu đạt 1000 requests/giây trên một instance đơn với phần cứng tiêu chuẩn (2 vCPU, 2 GB RAM).

Độ chính xác. Block rate trên tập 42 payload OWASP chuẩn phải đạt $\geq 95\%$. Tỷ lệ false positive trên traffic hợp lệ (đo bằng cách replay các request hợp pháp từ Juice Shop và DVWA) phải dưới 5%. Đối với mô hình ML, binary attack precision ≥ 0.95 và recall ≥ 0.95 trên tập kiểm thử nội bộ.

Khả dụng và độ tin cậy. Hệ thống phải duy trì hành vi *fail-open*: nếu ML service không khả dụng, WAF không được chặn nhầm traffic hợp lệ. Nếu cơ sở dữ liệu không khả dụng, các endpoint chỉ đọc (dashboard) có thể trả về trạng thái degraded, nhưng đường nhanh xử lý request không được bị chặn (audit log fallback sang file local). Hệ thống phải có khả năng *hot reload* tập luật và cấu hình mà không cần restart proxy.

Bảo mật nội bộ. Mật khẩu được lưu dưới dạng hash bcrypt cost factor 10. JWT được phát hành với secret tối thiểu 256 bit và lưu trong HttpOnly cookie với cờ Secure khi triển khai HTTPS. Mọi endpoint quản trị (rule, IP list, user management, system settings) đều được gate qua middleware `requireAdmin`. Trang đăng ký công khai không cho phép gán role tùy ý từ phía client (đã đóng lỗ leo thang đặc quyền). Các bất biến chống logout (last-admin protection) và chống tự gây hại (self-delete/self-demote protection) được bảo đảm ở cả tầng handler lẫn tầng giao diện. Mọi thao tác quản trị được ghi audit log.

Khả năng triển khai và quan sát. Toàn bộ hệ thống phải triển khai được bằng một lệnh duy nhất `docker compose up`, không yêu cầu cấu hình thủ công ngoài các tệp `.env` chuẩn. Hệ thống phải xuất metric ở định dạng Prometheus qua endpoint `/metrics`, log có cấu trúc (structured logging) dưới định dạng JSON ra stdout để dễ tích hợp với hạ tầng logging tập trung.

Kết chương. Chương này đã tiến hành khảo sát hiện trạng các giải pháp WAF mã nguồn mở và thương mại, xác định ba nhóm người dùng mục tiêu với kỳ vọng cụ thể, mô hình hóa hệ thống bằng mười ca sử dụng tiêu biểu trải đều bốn cụm chức năng (xử lý lưu lượng, quản lý cấu hình, quan sát hệ thống, quản trị nội bộ), đặc tả chi tiết năm chức năng nghiệp vụ cốt lõi, và liệt kê đầy đủ tập yêu cầu phi chức năng. Đặc biệt, đề tài đã làm rõ kiến trúc xác thực và phân quyền với RBAC ba vai trò (admin/editor/viewer), các bất biến bảo vệ chống logout và tự gây hại trong ca sử dụng quản trị tài khoản, cùng chính sách bật lỗ leo thang đặc quyền qua endpoint đăng ký công khai. Các đặc tả và ràng buộc xác định ở chương này là đầu vào trực tiếp cho phần thiết kế và triển khai chi tiết được trình bày trong Chương 4.1.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương trước đã đặc tả đầy đủ các yêu cầu chức năng và phi chức năng của hệ thống. Để biến các đặc tả đó thành một sản phẩm vận hành được, đề tài lựa chọn một bộ công nghệ với tiêu chí tổng thể là *hiệu năng cao trên đường nhanh, có khả năng tích hợp với hệ sinh thái học máy, và triển khai được trong môi trường mã nguồn mở khép kín*. Chương này trình bày nền tảng lý thuyết và đặc tính kỹ thuật của các thành phần công nghệ được lựa chọn, kèm theo lý giải vì sao mỗi lựa chọn phù hợp với yêu cầu đã xác định ở Chương 2.4. Các thành phần được tổ chức theo lớp kiến trúc: từ ngôn ngữ và runtime của WAF core (Go), đến cơ chế phát hiện (Rule Engine, DistilBERT, FastAPI), hạ tầng lưu trữ (PostgreSQL), lớp bảo mật nội bộ (JWT, Bcrypt, RBAC), thuật toán điều khiển tải (Token Bucket), và cuối cùng là môi trường đóng gói triển khai (Docker, Docker Compose).

3.1 Go (Golang) – Nền tảng xây dựng WAF core

3.1.1 Khái quát

Go là ngôn ngữ lập trình biên dịch tĩnh được Google công bố năm 2009, hướng đến giải quyết các vấn đề thực tế phát sinh khi xây dựng phần mềm hệ thống quy mô lớn. Ba đặc tính cốt lõi của Go bao gồm: **(i)** mô hình đồng thời (concurrency model) dựa trên *goroutine* – các luồng xanh nhẹ hơn luồng hệ điều hành nhiều lần (mặc định stack 8 KB so với hàng MB của thread truyền thống), được lập lịch bởi runtime của ngôn ngữ theo mô hình M:N (M goroutine ánh xạ lên N thread của OS); **(ii)** cơ chế giao tiếp giữa các goroutine qua *channel* thay vì shared memory với mutex như mô hình cổ điển, được tổng kết bởi phương châm “don’t communicate by sharing memory; share memory by communicating”; và **(iii)** thư viện chuẩn (standard library) phong phú và ổn định, bao gồm gói `net/http` có thể xây dựng máy chủ HTTP đầy đủ tính năng mà không cần bất kỳ framework bên ngoài nào.

Về quản lý bộ nhớ, Go sử dụng garbage collector concurrent tri-color mark-and-sweep từ phiên bản 1.5, với mục tiêu duy trì thời gian dừng (*stop-the-world pause*) dưới một mili giây ngay cả với heap kích thước hàng GB. Đây là đặc tính quan trọng đối với các ứng dụng yêu cầu độ trễ thấp và tính xác định cao như reverse proxy.

3.1.2 Vai trò trong hệ thống

Toàn bộ WAF core trong đề tài được viết bằng Go: vòng lặp chấp nhận kết nối HTTP, parser request, normalizer, rule engine, các middleware (xác thực, rate limit, logging), bộ điều phối quyết định, và lớp adapter giao tiếp với ML service. Riêng

phần ML inference được tách ra microservice Python (mục 3.4) để tận dụng hệ sinh thái Hugging Face – tuy nhiên việc giao tiếp giữa Go và service Python được thực hiện qua HTTP với kiểu dữ liệu đơn giản, không tạo ràng buộc chặt giữa hai runtime.

Gói `net/http` của thư viện chuẩn cung cấp interface `Handler` và pipeline middleware dạng chain – chính là khung mà WAF middleware stack được xây dựng. Đặc biệt, `httputil.ReverseProxy` cung cấp sẵn cơ chế tunnel ngược cho HTTP/1.1, HTTP/2 và WebSocket, giúp đề tài không phải hiện thực hóa lại logic forwarding ở mức byte. Việc dispatching theo prefix path được thực hiện qua `http.ServeMux` với mẫu kết thúc bằng dấu `“/”` để khớp tiền tố – mẫu này tương thích với phiên bản Go 1.21 mà đề tài cố định nhằm giảm phụ thuộc vào các tính năng mới (như pattern matching `{id}` chỉ có từ Go 1.22).

3.1.3 Lý do lựa chọn

Việc lựa chọn Go cho lớp WAF core được dựa trên ba so sánh thực tế với các ứng cử viên thường gặp.

So với **Python**, Go nhanh hơn về throughput I/O-bound từ 5 đến 20 lần trong các benchmark độc lập (TechEmpower Round 22, 2023), nhờ goroutine không phải chịu chi phí của Global Interpreter Lock (GIL). Hơn nữa, biên dịch ra binary tĩnh đơn lẻ giúp triển khai gọn nhẹ, không phụ thuộc môi trường Python ở máy đích – đặc tính quan trọng cho image Docker.

So với **Node.js**, Go cung cấp hiệu năng tương đương ở các workload I/O-bound đơn giản, nhưng vượt trội rõ rệt khi workload tính toán hỗn hợp (parser, regex matching, mã hóa) do mô hình đa luồng thật thay vì event loop đơn luồng. Việc xử lý song song nhiều request mà không phải lập trình asynchronous tường minh giúp mã nguồn dễ đọc và dễ kiểm thử hơn.

So với **Rust**, Go thua kém về hiệu năng tuyệt đối và an toàn bộ nhớ ở mức compile-time, nhưng vượt trội về tốc độ phát triển và độ thân thiện với người mới. Rust phù hợp hơn cho các thành phần ở tầng thấp như packet parser ở Layer 3/4, vốn không phải phạm vi của đề tài. Đối với một reverse proxy ở tầng 7 với độ trễ mục tiêu vài mili giây, Go cân bằng tốt nhất giữa ba tiêu chí: hiệu năng đủ dùng, tốc độ phát triển nhanh, và hệ sinh thái phong phú.

3.2 Rule Engine và OWASP CRS – Cơ sở phát hiện dựa trên luật

3.2.1 Khái quát cơ chế rule-based detection

Rule-based detection là mô hình phát hiện tấn công cổ điển nhất trong không gian WAF, dựa trên ý tưởng nền tảng: *nếu một request có chứa các mẫu văn bản*

đặc trưng của một loại tấn công đã biết, thì request đó nhiều khả năng là tấn công. Mô hình này được hiện thực hóa bằng tập rule – mỗi rule mô tả một mẫu pattern cần kiểm tra trên một hay nhiều trường của request, kèm theo hành động cần thực hiện khi pattern khớp.

Hệ thống đề tài kế thừa và tinh giản mô hình *anomaly scoring* mà OWASP Core Rule Set đã đề xuất từ phiên bản 3.0. Khác với mô hình *immediate block* (chặn ngay khi có rule khớp đầu tiên), anomaly scoring cho phép mỗi rule đóng góp một điểm số nhỏ vào tổng điểm nguy cơ của request; quyết định block chỉ được đưa ra khi điểm tổng vượt một ngưỡng được cấu hình. Ưu điểm chính của mô hình này là giảm false positive cho các trường hợp một request bình thường vô tình khớp một rule yếu (ví dụ chứa từ khóa SQL trong dữ liệu hợp lệ), nhưng vẫn giữ được khả năng phát hiện các tấn công đa pha mà mỗi pattern đơn lẻ không đủ kết luận.

Một trong những thách thức cơ bản của rule-based detection là kỹ thuật vượt mặt (evasion). Kẻ tấn công có thể biến tấu payload thông qua URL encoding nhiều lớp, thay đổi hoa thường, chèn ký tự trắng (whitespace padding), hay sử dụng biểu thức tương đương ngữ nghĩa nhưng khác cú pháp. Để chống lại các kỹ thuật này, mỗi rule được trang bị một *transform chain* – chuỗi các biến đổi tiền xử lý áp dụng trên text đầu vào trước khi so khớp pattern. Chuỗi mặc định bao gồm:

- URL_DECODE: giải mã URL một lớp, chuyển %XX về ký tự tương ứng – chống obfuscation cơ bản.
- LOWERCASE: chuyển toàn bộ về chữ thường – chống case-mixing kiểu uNiOn SeLeCt.
- COMPRESS_WHITESPACE: dồn các khoảng trắng liên tiếp về một ký tự duy nhất – chống chèn ký tự trắng giữa các từ khóa.
- HTML_ENTITY_DECODE: giải mã các entity HTML (<, <) – quan trọng với các XSS payload.
- BASE64_DECODE: hỗ trợ giải mã base64 cho các nhóm rule đặc thù như Log4Shell với JNDI URL được encode base64.

Đối với pattern matching, đề tài hỗ trợ hai kiểu mẫu. **REGEX pattern** dùng biểu thức chính quy của Go (regexp package, dựa trên RE2, không hỗ trợ backreference nhưng đảm bảo độ phức tạp tuyến tính, không bị catastrophic backtracking). **TOKEN pattern** dựa trên khái niệm *proximity match*: liệt kê một dãy token và yêu cầu chúng xuất hiện trong văn bản theo đúng thứ tự, trong khoảng cách tối đa N ký tự. Token pattern thường mạnh hơn regex trong việc phát hiện các tấn công đa từ khóa mà thứ tự quan trọng nhưng khoảng đệm có thể thay đổi (ví dụ

union...select cách nhau bởi comment SQL).

3.2.2 Cấu trúc rule JSON

Mỗi rule được mô tả dưới dạng một đối tượng JSON với các trường chính: định danh và metadata (`id`, `category`, `severity`, `description`), điều kiện áp dụng (`conditions` – các trường của request mà rule cần kiểm tra: `path`, `query`, `headers.user-agent`, `body`, v.v.), chuỗi biến đổi tiền xử lý (`transforms`), danh sách pattern (`patterns` – có thể là mảng các regex hoặc token sequence), thông số scoring (`anomaly_score`, `severity_multiplier`), hành động (`actions` – mặc định là *score-only*, có thể nâng lên *deny-immediate* cho các rule critical), và danh sách ngoại lệ (`exceptions` – ví dụ bỏ qua đường dẫn `/health`, `/metrics`).

Cấu trúc JSON có hai ưu điểm chính so với cú pháp ngôn ngữ chuyên dụng của ModSecurity (SecRule DSL). Thứ nhất, JSON là chuẩn dữ liệu phổ biến, có thể được sinh tự động từ công cụ và validate bằng JSON Schema. Thứ hai, việc nạp lại tập rule mà không cần khởi động lại proxy được thực hiện đơn giản qua một endpoint admin nhận file JSON, parse, validate, và hot-swap engine state – hành vi mà ModSecurity vốn không hỗ trợ tốt do kiến trúc module nhúng vào web server.

Mỗi phase của vòng đời request có thể có rule riêng. Trong phạm vi đề tài, chỉ phase REQUEST được hiện thực hóa – kiểm tra request trước khi forward đến backend. Phase RESPONSE (kiểm tra response trước khi trả về cho client, phát hiện data leak hay error message) được xác định là hướng phát triển tương lai (mục 6.2).

3.2.3 Bộ luật 78 rules / 13 nhóm

Đề tài xây dựng tập luật gồm 78 rule chia thành 13 nhóm tấn công, bao phủ hầu hết các nhóm trong OWASP Top 10 2021 và các nhóm tấn công thường gặp ngoài Top 10. Tập luật gồm hai tầng: khoảng 45 *chữ ký* (signature) bắt mẫu tấn công hoàn chỉnh và 33 *dấu hiệu* (indicator) là các tín hiệu yếu cộng dồn theo mô hình anomaly scoring. Bảng 3.1 liệt kê các nhóm và số lượng rule trong mỗi nhóm. Danh sách chi tiết của 78 rule cùng pattern chính được trình bày ở Phụ lục B.

Cách xây dựng tập rule đi qua ba bước. Thứ nhất, lựa chọn các nhóm tấn công có tần suất xuất hiện cao nhất trong các báo cáo bảo mật ứng dụng web giai đoạn 2021–2024 (OWASP Top 10 2021, Verizon DBIR 2023, các CVE nổi bật như Log4Shell). Thứ hai, với mỗi nhóm, khảo sát các pattern điển hình từ OWASP CRS, từ payload list công khai (PayloadAllTheThings, SecLists), và từ các sample tấn công đã được công bố. Thứ ba, viết và kiểm thử pattern trên tập 42 OWASP attack payload đã biết, hiệu chỉnh transform chain và proximity threshold cho đến

Bảng 3.1: Phân bố 78 rule theo 13 nhóm tấn công

Nhóm tấn công	Số rule	Đại diện pattern
SQL Injection (SQLi)	13	UNION SELECT, OR 1=1, time-based sleep/benchmark, error-based extract-value
Cross-Site Scripting (XSS)	12	<script>, event handler on-error=, javascript:/data: URI, iframe/svg
Remote Code Execution / CMDi	16	Shell metachar ; && \$(), wget/curl/nc, PHP system()/exec(), reverse shell, JNDI
Path Traversal / LFI	10	../, biến thể mã hóa, /etc/passwd, php:// wrapper, null byte
Server-Side Request Forgery (SSRF)	6	Dải IP nội bộ, 169.254.169.254, file://, gopher://
XML External Entity (XXE)	2	<!ENTITY, SYSTEM, parameter entity
NoSQL Injection	2	Toán tử Mongo \$where, \$ne, \$gt, \$regex
Information Leak	5	Dotfiles/.git, file backup .bak/.old, file cấu hình, CRLF header injection
Scanner Detection	4	Scanner User-Agent (Nikto, sqlmap), probe path .env/wp-admin
Account Takeover (ATO)	2	Brute-force /login, lạm dụng password-reset
Bot / Automation	1	Thiếu Accept-Language (heuristic)
Denial of Service (DoS)	1	Cụm header quá khổ
Custom / Heuristic	4	Null byte, over-encoding, hex obfuscation, template injection (SSTI)

khi đạt block rate $\geq 95\%$.

3.3 DistilBERT – Mô hình phân loại tấn công

3.3.1 Khái quát kiến trúc Transformer và BERT

DistilBERT là một biến thể nhẹ của BERT, vốn là kiến trúc tiêu chuẩn cho nhiều bài toán xử lý ngôn ngữ tự nhiên kể từ năm 2018. Để hiểu DistilBERT, cần đi qua hai nền tảng tiền đề là kiến trúc Transformer và BERT.

Transformer (Vaswani và cộng sự, 2017) là kiến trúc mạng nơ-ron được đề xuất ban đầu cho bài toán dịch máy, dựa hoàn toàn vào cơ chế *self-attention* thay vì các cell hồi quy (RNN/LSTM) hay convolution. Cơ chế self-attention cho phép mỗi vị trí trong chuỗi đầu vào “chú ý” đến mọi vị trí khác cùng một lúc, với trọng

số được tính theo công thức $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$ trong đó Q, K, V là các ma trận query, key, value được tính từ chính chuỗi đầu vào. Lợi thế của Transformer so với RNN nằm ở khả năng song song hóa hoàn toàn quá trình tính toán cho mọi vị trí trong chuỗi, tận dụng tốt phần cứng GPU/TPU hiện đại.

BERT (Devlin và cộng sự, 2018) là mô hình tiền huấn luyện (pre-trained) dựa trên kiến trúc Transformer encoder hai chiều, gồm hai nhiệm vụ tiền huấn luyện: *Masked Language Modeling* (MLM – dự đoán các token bị che ngẫu nhiên trong văn bản) và *Next Sentence Prediction* (NSP – dự đoán hai câu có liên tiếp hay không). Sau khi tiền huấn luyện trên tập văn bản khổng lồ (Wikipedia + BookCorpus, khoảng 3.3 tỷ từ), BERT có thể được fine-tune cho các nhiệm vụ downstream như phân loại câu, gán nhãn token, hỏi đáp, với chi phí và lượng dữ liệu nhỏ hơn nhiều so với huấn luyện từ đầu.

DistilBERT (Sanh và cộng sự, 2019) là phiên bản chất lọc (distilled) của BERT-base thông qua kỹ thuật *knowledge distillation*: huấn luyện một mô hình “học sinh” (student, kiến trúc đơn giản hơn) học cách khớp với phân phối xác suất đầu ra của mô hình “giáo viên” (teacher, ở đây là BERT-base). Kết quả là DistilBERT có 6 lớp encoder thay vì 12 (giảm 40% số tham số xuống còn 66 triệu), nhanh hơn 60% ở thời gian inference, nhưng vẫn giữ được khoảng 97% hiệu suất của BERT-base trên benchmark GLUE. Đây chính là đặc tính khiến DistilBERT phù hợp với bài toán phân loại real-time của WAF, nơi mỗi mili giây độ trễ đều ảnh hưởng đến trải nghiệm người dùng cuối.

3.3.2 Fine-tuning cho bài toán phân loại HTTP request

Bài toán mà DistilBERT đảm nhận trong đề tài là *sequence classification* với 10 lớp: `normal` (không phải tấn công) và 9 lớp tấn công tương ứng với các nhóm chính (`sqli`, `xss`, `cmdi`, `path_traversal`, `ssrf`, `xxe`, `log4shell`, `ssti`, `nosqli`). Trong giai đoạn suy luận (inference), output cuối được giảm về phân loại nhị phân (attack/normal) bằng cách gộp 9 nhãn tấn công thành một lớp – phù hợp với nhu cầu sử dụng của engine.

Tiền xử lý đầu vào – canonical text: Đầu vào của mô hình là một biểu diễn văn bản chuẩn hóa của HTTP request, gọi là *canonical text*. Đặc tả của canonical text được thống nhất giữa Go (hàm `BuildCanonicalText` trong WAF core) và Python (hàm `canonical_compose` trong ML service) và phải tạo ra cùng một chuỗi byte cho cùng một request – đây là ràng buộc cứng được kiểm tra bằng 7 fixture test case xuyên hai ngôn ngữ. Định dạng tổng quát: `METHOD PATH\nHost: ... \nUser-Agent: ... \n\nBODY`, với các quy tắc cụ thể: URL-decode một lớp đối với path và query, giới hạn mỗi header value ở 256 ký tự, sắp xếp header

theo thứ tự alphabetic để đảm bảo tính xác định.

Pipeline fine-tuning: Đề tài fine-tune từ checkpoint `distilbert-base-uncased` với các tham số: optimizer AdamW, learning rate 2×10^{-5} , batch size 32, max sequence length 512 token, số epoch tối đa 6 với early stopping theo validation F1. Loss function là cross-entropy với class weights tỷ lệ nghịch với tần suất lớp để bù trừ mất cân bằng dữ liệu. Dữ liệu huấn luyện được kiểm soát theo phiên bản: v5 (baseline), v6 (fail report – không đạt hard gate), v6.1 (kế hoạch augmentation 1800 mẫu có mục tiêu).

Hard gate kiểm soát chất lượng: Đề tài áp đặt một bộ tiêu chí cứng mà mô hình phải vượt qua trước khi được ship lên production: overall accuracy ≥ 0.92 , cmdi F1 ≥ 0.85 , log4shell F1 ≥ 0.90 . Mô hình không đạt gate sẽ không được tích hợp – ví dụ v6 đã fail với accuracy 0.8866, dẫn đến quyết định giữ v5 trong production và lập kế hoạch v6.1. Chi tiết của cơ chế hard gate được trình bày ở Chương 5.3.

3.3.3 Lý do lựa chọn

So với các kiến trúc khác có thể dùng cho cùng bài toán, DistilBERT cân bằng tốt nhất giữa độ chính xác và tốc độ.

So với **LSTM hoặc CNN-text**, DistilBERT hưởng lợi từ tiền huấn luyện trên kho ngữ liệu khổng lồ – một lợi thế quan trọng khi dữ liệu HTTP request có nhãn không đủ phong phú để huấn luyện một mô hình hoàn toàn từ đầu. Các mẫu hình của BERT về cấu trúc ngữ pháp và mối quan hệ ngữ nghĩa đã được học sẵn, fine-tuning chỉ phải điều chỉnh các phân biệt tinh tế giữa các nhãn tấn công.

So với **BERT-base hoặc RoBERTa**, DistilBERT hi sinh khoảng 3% accuracy để đổi lấy gấp đôi tốc độ inference – một đánh đổi mà đề tài chấp nhận được, vì ML chỉ được gọi trong vùng xám (khoảng 5–10% lưu lượng) và timeout cứng 800 mili giây bao trùm tất cả các bước HTTP, tokenization, và forward pass. Trên phần cứng CPU phổ thông (Intel i5 thế hệ 10), latency inference của DistilBERT trung bình 60–80 mili giây cho một sequence ngắn, phù hợp với budget thời gian.

So với các mô hình hệ thống học truyền thống như **Random Forest hay SVM trên TF-IDF features**, DistilBERT vượt trội rõ rệt khi xử lý các biến thể đối nghịch (adversarial variants) nhờ khả năng nắm bắt ngữ cảnh, không phụ thuộc vào việc liệt kê các n-gram đặc trưng – vốn dễ bị đánh lừa qua character-level transformations.

3.4 FastAPI và Python – ML Inference Service

3.4.1 Khái quát

FastAPI là framework xây dựng API HTTP cho Python được phát triển từ năm 2018, dựa trên ba thành phần nền tảng: **Starlette** (ASGI toolkit), **Pydantic** (kiểm tra và validate dữ liệu qua type hint), và **Uvicorn** (ASGI server hiệu năng cao dựa trên uvloop và httptools). Đặc điểm nổi bật của FastAPI bao gồm: tự động sinh tài liệu OpenAPI (Swagger UI) từ type hint, tích hợp sẵn async/await cho I/O concurrency, và hiệu năng cao hơn các framework cổ điển Flask/Django đáng kể (ở mức gần ngang với Node.js Express trong các benchmark độc lập).

3.4.2 Vai trò trong hệ thống

ML Inference Service đóng vai trò microservice chuyên trách suy luận, được viết bằng Python để tận dụng hệ sinh thái Hugging Face (transformers, tokenizers) và PyTorch – hai gói không có tương đương trưởng thành trong hệ sinh thái Go. Service cung cấp hai endpoint chính: POST /predict nhận một canonical text và trả về (label, confidence); và POST /predict_batch nhận một mảng canonical text và trả về mảng kết quả tương ứng – được sử dụng cho mục đích đánh giá hàng loạt.

Khi khởi động, service nạp checkpoint model từ thư mục local (model_v5/, model_v6/), chuyển sang chế độ eval của PyTorch, và (khi cấu hình) bật chế độ inference no-grad để giảm tiêu thụ bộ nhớ. Trên CPU, một worker đơn xử lý tuần tự các request đến; khi cần mở rộng, có thể tăng số worker Uvicorn hoặc chạy nhiều instance phía sau một load balancer.

Việc tách ML thành service riêng có ba lợi ích cụ thể: **(i)** cách ly môi trường runtime, không trộn lẫn binary Go với phụ thuộc Python; **(ii)** có thể nâng cấp hoặc thay thế mô hình (đổi từ DistilBERT sang một mô hình khác) mà không cần biên dịch lại WAF core; và **(iii)** cho phép chia sẻ service ML giữa nhiều instance WAF trong tương lai, tận dụng cache và warm-up của mô hình.

3.5 PostgreSQL – Lưu trữ logs, users, cấu hình

3.5.1 Khái quát

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, phát triển liên tục từ năm 1996, được biết đến với độ tuân thủ chuẩn SQL cao, hỗ trợ phong phú các kiểu dữ liệu (JSON, JSONB, mảng, range, network), và khả năng mở rộng qua extension (PostGIS, pg_trgm, TimescaleDB). Phiên bản đề tài sử dụng là PostgreSQL 15, hỗ trợ đầy đủ CHECK constraint, GENERATED column, và partitioning theo timestamp – ba tính năng hữu ích cho schema của đề tài.

3.5.2 Vai trò trong hệ thống

PostgreSQL đảm nhận ba trách nhiệm chính: (i) lưu trữ tài khoản người dùng nội bộ (bảng `users`), (ii) ghi nhật ký kiểm toán (bảng `audit_logs`), và (iii) lưu trữ động các danh sách IP (bảng `ip_lists`). Cấu hình WAF cốt lõi (ngưỡng, URL backend) được lưu trong tệp `config.yaml` thay vì database để giảm phụ thuộc khởi động.

Việc lựa chọn PostgreSQL thay vì các SQL khác (MySQL, SQLite) hay NoSQL (MongoDB) dựa trên hai cân nhắc. Thứ nhất, dữ liệu audit log có tính chất quan hệ rõ ràng và yêu cầu truy vấn lọc/sắp xếp/aggregate phức tạp (theo thời gian, IP, ruleID, kiểu tấn công) – phù hợp với mô hình quan hệ và chỉ mục B-tree truyền thống. Thứ hai, hỗ trợ CHECK constraint và UNIQUE ràng buộc ở mức schema là cốt lõi của chính sách bảo mật nội bộ (chống nhập role bất hợp lệ, chống trùng username/email – mục 2.2.8).

3.5.3 Schema chính

Bảng `users` lưu thông tin tài khoản nội bộ. Các cột: `id` (SERIAL PRIMARY KEY), `username` (VARCHAR UNIQUE NOT NULL), `email` (VARCHAR UNIQUE), `password_hash` (VARCHAR NOT NULL – bcrypt hash, mục 3.6), `role` (VARCHAR CHECK (role IN ('admin', 'editor', 'viewer'))), `created_at`, `updated_at`.

Bảng `audit_logs` lưu mỗi bản ghi kiểm toán. Các cột chính: `id`, `timestamp` (có index), `ip`, `method`, `path`, `score`, `decision` (ENUM ALLOW/LOG/CHALLENGE/BLOCK), `rule_ids` (INTEGER[]), `ml_label`, `ml_confidence`, `actor_id` (NULLABLE, tham chiếu `users.id` cho các sự kiện hệ thống), `event_type` (request hoặc system).

Bảng `ip_lists` lưu các IP/CIDR thuộc blacklist hoặc whitelist. Các cột: `id`, `ip_or_cidr`, `list_type` (CHECK IN ('blacklist', 'whitelist')), `expires_at` (NULLABLE – hỗ trợ TTL tự động hết hạn), `reason`.

3.6 JWT và Bcrypt – Xác thực và bảo mật

3.6.1 JWT: xác thực không trạng thái với dual-mode delivery

JSON Web Token (JWT) là chuẩn được mô tả trong RFC 7519, dùng để truyền khẳng định (claim) giữa các bên dưới dạng một chuỗi ba phần `header.payload.signature` được mã hóa base64url. Đề tài sử dụng thuật toán ký HS256 (HMAC-SHA256 với khóa bí mật) – phù hợp cho mô hình một service đơn lẻ xác thực và tiêu thụ token cho chính nó. Trong các kiến trúc phức tạp hơn với nhiều service tiêu thụ, RS256 với cặp khóa bất đối xứng là lựa chọn tốt hơn, nhưng nằm ngoài phạm vi của đề

tài.

Đặc tính cốt lõi của JWT là tính *stateless* – server không cần lưu trạng thái session ở phía mình; chỉ cần verify chữ ký và đọc claim. Đặc tính này phù hợp với mô hình triển khai container hóa của đề tài: các instance WAF có thể được scale ngang mà không cần share session store.

Dual-mode delivery: Đề tài hiện thực hóa hai cơ chế song song để client gửi JWT về server. Cơ chế thứ nhất là *HttpOnly cookie* – JWT được set vào cookie với cờ *HttpOnly* (JavaScript không thể đọc, chống XSS exfiltration), *Secure* (chỉ gửi qua HTTPS), và *SameSite=Lax* (giảm thiểu CSRF). Đây là cơ chế mặc định cho dashboard web. Cơ chế thứ hai là *Authorization Bearer header* – client gửi header `Authorization: Bearer <token>`; phù hợp cho API client và các kịch bản scripting. Middleware xác thực kiểm tra cả hai nguồn, cho phép một backend thống nhất phục vụ cả browser-based dashboard lẫn API integration.

3.6.2 Bcrypt: hashing mật khẩu với cost factor

Bcrypt là hàm hash mật khẩu được Niels Provos và David Mazières giới thiệu năm 1999, dựa trên thuật toán mã hóa Blowfish nhưng được điều chỉnh để trở thành hàm hash một chiều, cố ý chậm. Đặc tính then chốt của bcrypt là tham số *cost factor* – số mũ trên một biến đếm vòng lặp nội bộ ($\text{work factor} = 2^{\text{cost}}$) – cho phép tăng chi phí tính toán theo thời gian khi phần cứng nhanh hơn, mà không thay đổi giao diện hàm.

Đề tài sử dụng cost factor 10 (1024 vòng) – giá trị mặc định của gói `golang.org/x/crypto` tương ứng với khoảng 60–80 mili giây để hash một mật khẩu trên phần cứng phổ thông. Mức cost này được khuyến nghị bởi OWASP Password Storage Cheat Sheet (2023) như mức cân bằng giữa an toàn chống brute force và trải nghiệm người dùng (đăng nhập không bị cảm nhận được độ trễ).

So với các hàm hash mật khẩu hiện đại hơn (Argon2, scrypt), bcrypt vẫn được coi là chấp nhận được cho hầu hết các use case và có lợi thế là thư viện sẵn sàng, được kiểm chứng, và không yêu cầu cấu hình bộ nhớ phức tạp. Đề tài chấp nhận đánh đổi này do tập user nội bộ có quy mô nhỏ, không phải target chính của các cuộc tấn công brute force quy mô lớn.

3.6.3 RBAC: ba vai trò admin / editor / viewer

Role-Based Access Control (RBAC) là mô hình phân quyền cổ điển trong đó quyền truy cập được gán cho *vai trò* thay vì cho cá nhân; người dùng được gán một (hoặc nhiều) vai trò, và quyền của họ là tổng hợp các quyền của các vai trò đó. Đề tài hiện thực hóa phiên bản đơn giản nhất của RBAC với mỗi người dùng có đúng

một vai trò.

Ma trận phân quyền: Bảng 3.2 liệt kê các thao tác nhạy cảm và vai trò có quyền thực hiện.

Bảng 3.2: Ma trận phân quyền RBAC ba vai trò

Thao tác	admin	editor	viewer
Xem dashboard, audit log	✓	✓	✓
Cập nhật thông tin cá nhân (/me)	✓	✓	✓
Đổi mật khẩu của mình (/me/password)	✓	✓	✓
Sửa rule, IP list, system settings	✓	✓	–
Cấu hình kênh cảnh báo	✓	✓	–
CRUD tài khoản người dùng khác	✓	–	–
Reset mật khẩu của người khác	✓	–	–

Hiện thực hóa: Vai trò được lưu ở cột `role` của bảng `users` với ràng buộc CHECK ở mức schema (mục 3.5.3). Tại runtime, hai middleware được sử dụng: `requireAuthN` chỉ yêu cầu JWT hợp lệ (cho phép mọi role có token), và `requireAdmin` thêm điều kiện `claim.role = "admin"`. Các endpoint `admin-only` được gate qua `requireAdmin`; các endpoint chỉ yêu cầu đăng nhập (như `/me`) qua `requireAuthN`.

Bảo vệ chống `lockout`: Đặc thù của bài toán quản trị tài khoản là khả năng tự đẩy hệ thống vào trạng thái không thể phục hồi – ví dụ `admin` duy nhất tự xóa hoặc tự demote chính mình, dẫn đến hệ thống không còn `admin` nào. Để chống lại, đề tài định nghĩa hai bất biến (invariant) được kiểm tra ở tầng handler trước khi commit thay đổi: *last-admin protection* (không cho phép xóa hoặc demote `admin` cuối cùng, dựa trên truy vấn `SELECT COUNT(*) FROM users WHERE role = 'admin'`) và *self-action protection* (`admin` không được xóa/demote chính tài khoản đang đăng nhập). Hai bất biến này được trình bày chi tiết ở mục 2.2.10.

3.7 Token Bucket – Thuật toán Rate Limiting

3.7.1 Khái quát Token Bucket

Token Bucket là một thuật toán điều khiển tải đường truyền cổ điển được mô tả lần đầu trong bối cảnh quản lý chất lượng dịch vụ mạng (QoS) cuối thập niên 1980. Mô hình hoạt động dựa trên hình ảnh trực quan của một *bucket* (xô) chứa các *token* (thẻ). Bucket có hai tham số: *rate* (số token được rót vào bucket mỗi giây) và *burst* (kích thước tối đa của bucket). Mỗi request đến cần lấy đi một token mới được phục vụ; nếu bucket trống, request bị từ chối hoặc được đưa vào hàng chờ.

Đặc tính then chốt của Token Bucket là cho phép *burst* ngắn hạn: nếu một thời

gian dài không có request, bucket được tích lũy đầy token và có thể phục vụ một spike traffic mà không phạt; sau đó traffic phải trở về dưới rate trung bình để bucket có thời gian phục hồi. Hành vi này phù hợp hơn với traffic web thực tế (có tính burst tự nhiên) so với mô hình *leaky bucket* cổ điển vốn áp đặt rate cứng tại mọi thời điểm.

3.7.2 So sánh với các thuật toán khác

So với **Sliding Window Counter**, Token Bucket nhẹ hơn về bộ nhớ (chỉ cần lưu hai giá trị: số token hiện tại và timestamp lần cập nhật), trong khi Sliding Window cần lưu chuỗi timestamp của các request gần đây. Tuy nhiên, Sliding Window cho điểm chính xác hơn ở biên cửa sổ – một điểm yếu của Token Bucket ít quan trọng trong bài toán của đề tài.

So với **Fixed Window Counter**, Token Bucket không gặp vấn đề “boundary spike” – khi attacker gửi burst kép ở hai bên ranh giới cửa sổ để vượt qua giới hạn danh nghĩa. Mỗi bucket trong Token Bucket cập nhật liên tục theo thời gian, không có ranh giới rạch ròi.

3.7.3 Tích hợp vào middleware stack

Trong hệ thống, mỗi địa chỉ IP có một bucket riêng. Toàn bộ tập bucket được lưu trong một `sync.Map` cho phép truy cập concurrent không cần lock tường minh. Một goroutine quét định kỳ (mỗi 5 phút) loại bỏ các bucket có trạng thái không hoạt động lâu hơn 10 phút, ngăn chặn rò rỉ bộ nhớ khi IP space biến động lớn.

Rate Limiter được đặt trong middleware stack sau *IP-check middleware* (blacklist/whitelist) và trước *rule engine middleware*, theo nguyên tắc *cheap check first*: từ chối sớm các request không cần xử lý sâu. Khi một IP vượt quá ngưỡng cho phép liên tục, IP đó được tự động bổ sung vào blacklist động với TTL ngắn (mặc định 1 giờ) – đây là cơ chế phòng vệ hành vi (behavior-based defense) bổ sung cho rule pattern matching.

3.8 Docker và Docker Compose – Đóng gói và triển khai

3.8.1 Containerization các thành phần

Docker là công nghệ container hóa được phát triển từ năm 2013, đóng gói ứng dụng cùng toàn bộ phụ thuộc runtime vào một image duy nhất, đảm bảo tính tái lập của môi trường thực thi. Đề tài đóng gói ba thành phần riêng biệt: (i) WAF core (binary Go biên dịch tĩnh, image dựa trên `scratch` hoặc `distroless` với kích thước dưới 30 MB); (ii) ML Inference Service (image dựa trên `python:3.13-slim` kèm phụ thuộc PyTorch và transformers, kích thước khoảng 1.5 GB do model weight); và (iii) PostgreSQL (image chính thức `postgres:15`).

Việc đóng gói theo Docker mang lại ba lợi ích cụ thể. Thứ nhất, tách biệt runtime: WAF binary không cần Python hay PyTorch ở máy đích, và ngược lại. Thứ hai, cùng image có thể chạy trên mọi nền tảng host hỗ trợ Docker, từ máy trạm phát triển đến server sản xuất, không phụ thuộc hệ điều hành cụ thể. Thứ ba, image có thể được publish lên registry (Docker Hub, GHCR) và phân phối với version tag rõ ràng, hỗ trợ rollback nhanh khi cần.

3.8.2 Docker Compose one-command deployment

Docker Compose là công cụ điều phối multi-container ở quy mô một máy chủ đơn, dựa trên tệp YAML mô tả tập service, mạng và volume. Đề tài cung cấp tệp `docker-compose.db.yml` mô tả ba service nói trên, với các cấu hình quan trọng: ánh xạ cổng (8080, 8443 cho WAF; 8000 cho ML; 5432 cho PostgreSQL), volume mount cho dữ liệu PostgreSQL (`postgres_data-persistent`), và biến môi trường nạp từ tệp `.env`.

Toàn bộ triển khai được khởi động bằng một lệnh: `docker compose up -d`. Đối với người dùng mới, đề tài cung cấp thêm script `scripts/setup_db.sh` thực hiện migration ban đầu (tạo schema, seed admin mặc định) và `scripts/generate_certs.sh` sinh self-signed TLS certificate cho môi trường thử nghiệm.

3.8.3 Lý do lựa chọn

So với các giải pháp đóng gói khác như **tar archive thuần** (yêu cầu cài thủ công phụ thuộc) hay **Kubernetes manifest** (phức tạp quá mức cho quy mô đề tài), Docker Compose ở mức cân bằng phù hợp: đủ mạnh để mô tả multi-service deployment với network và volume rõ ràng, đủ đơn giản để một sinh viên hay kỹ sư DevOps mới có thể vận hành mà không cần hạ tầng cluster.

Đối với các triển khai sản xuất quy mô lớn hơn, Docker Compose có thể được chuyển đổi sang Kubernetes manifest (qua công cụ `kompose`) hoặc Helm chart, nhưng phép thay đổi này không thuộc phạm vi đề tài.

Kết chương. Chương này đã trình bày tám lớp công nghệ cốt lõi tạo nên hệ thống: Go cho WAF core với mô hình goroutine và GC độ trễ thấp; Rule Engine với mô hình anomaly scoring và transform chain chống evasion; DistilBERT cho lớp ML xác minh vùng xám; FastAPI/Python cho ML Inference Service tách biệt runtime; PostgreSQL cho lưu trữ quan hệ với ràng buộc tính toàn vẹn ở mức schema; JWT và bcrypt cho xác thực không trạng thái và lưu mật khẩu an toàn, kết hợp với RBAC ba vai trò admin/editor/viewer cùng các bất biến bảo vệ chống lockout; Token Bucket cho điều khiển tải per-IP với khả năng chịu burst; và Docker Compose cho đóng gói triển khai một lệnh. Mỗi lựa chọn được lý giải thông qua so sánh với các ứng

cử viên thay thế và đối chiếu với các yêu cầu phi chức năng đã xác định ở mục 2.4. Đây là cơ sở công nghệ trực tiếp cho phần thiết kế và hiện thực hóa hệ thống được trình bày trong Chương 4.1.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ KẾT QUẢ THỰC NGHIỆM

Chương này là chương trọng tâm của đồ án. Trên cơ sở các đặc tả ở Chương 2.3 và bộ công nghệ đã trình bày ở Chương 3.1, chương trình bày toàn bộ quá trình thiết kế, hiện thực hóa và đánh giá hệ thống WAF. Nội dung gồm bảy phần: kiến trúc tổng quan và luồng xử lý request xuyên suốt các lớp middleware (4.1); thiết kế chi tiết rule engine với cấu trúc dữ liệu, transform chain và cơ chế scoring (4.2); tích hợp ML layer và pipeline huấn luyện kiểm soát chất lượng (4.3); thiết kế cơ sở dữ liệu (4.4); xây dựng dashboard và các trang quản trị (4.5); kết quả kiểm thử toàn diện trên các tập payload tấn công chuẩn, kiểm thử tải và kiểm thử bảo mật nội bộ (4.6); và hướng dẫn triển khai thực tế qua Docker Compose (4.7).

4.1 Kiến trúc tổng quan hệ thống

4.1.1 Sơ đồ kiến trúc tổng thể

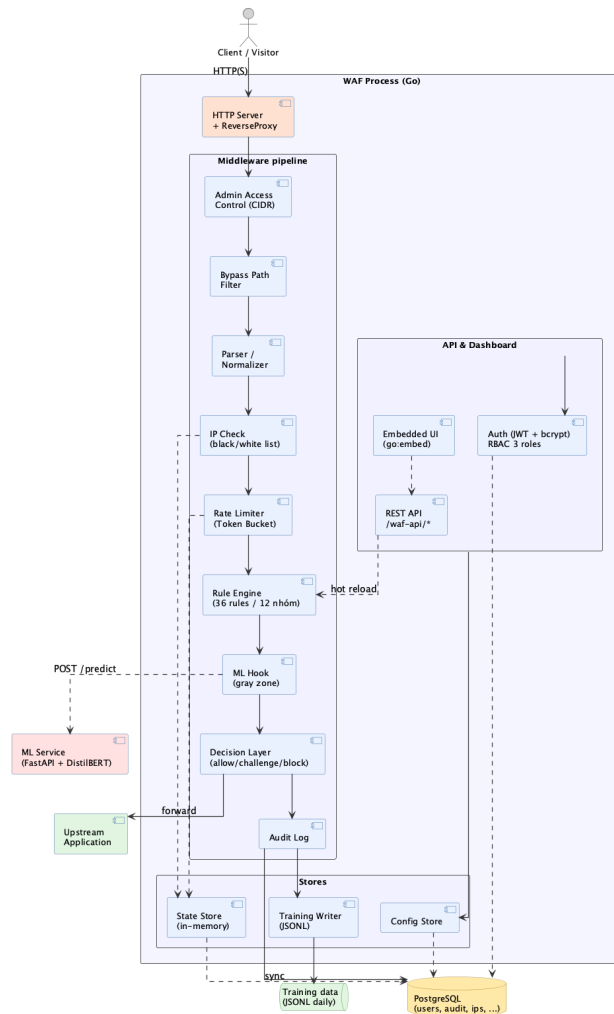
Hệ thống được tổ chức theo mô hình *reverse proxy trong suốt*: client gửi request đến WAF tại cổng công khai, WAF thực hiện toàn bộ kiểm tra, và chuyển tiếp request đến backend nếu được phép. Backend không cần biết đến sự tồn tại của WAF, không cần thay đổi cấu hình hay mã nguồn.

Hệ thống có ba hot path chính: (i) luồng xử lý request thông thường (client → WAF → backend), (ii) luồng quản trị (admin → Dashboard → WAF API → PostgreSQL), và (iii) luồng ML xác minh (rule engine → ML Service → quay lại rule engine), được kích hoạt có điều kiện.

4.1.2 Luồng xử lý request

Một request HTTP đi qua chín bước xử lý tuần tự trong WAF core. Mỗi bước có thể từ chối request và kết thúc sớm; nếu vượt qua tất cả, request được forward đến backend.

1. **Nhận và parse request.** TCP connection được terminate, TLS được giải mã (nếu HTTPS), parser bóc tách method, path, query parameters, headers, và body với giới hạn kích thước `max_body_size` (mặc định 10 MB).
2. **Normalize.** Áp dụng các biến đổi: URL decode một lớp, chuyển về lowercase, compress whitespace. Đầu ra là một `ParsedRequest` chuẩn hóa sẵn sàng cho các bước sau.
3. **IP check.** Truy vấn cache trong bộ nhớ (đồng bộ định kỳ với PostgreSQL): nếu IP nguồn nằm trong blacklist, từ chối ngay với mã 403; nếu nằm trong whitelist, bỏ qua các bước rule engine.



Hình 4.1: Sơ đồ kiến trúc tổng thể của hệ thống WAF

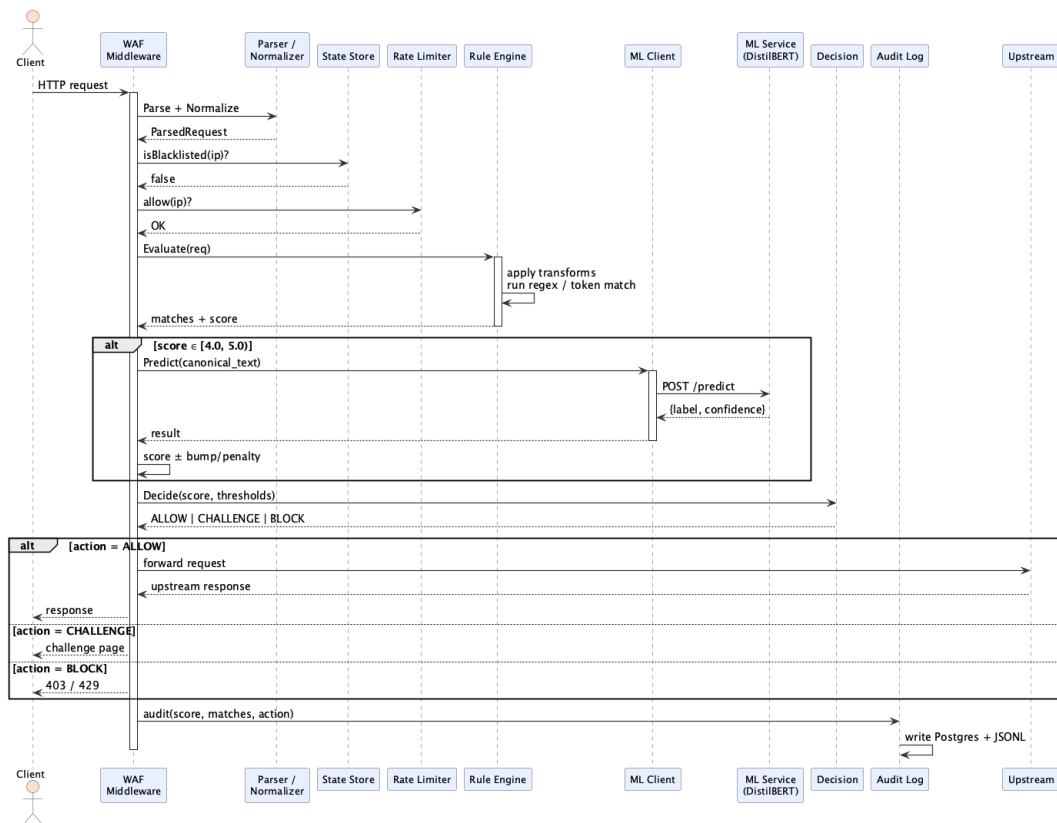
4. **Rate Limit.** Token Bucket per-IP (mục 3.7): lấy đi một token; nếu bucket trống, từ chối với mã 429.
5. **Authentication.** Với các endpoint quản trị của WAF (/waf-api/*, trang /dashboard), middleware đọc JWT từ cookie hoặc Authorization header; các trang công khai /login.html, /register.html không yêu cầu JWT. Toàn bộ bề mặt quản trị còn được bao bởi lớp *admin access control*: IP ngoài `admin.allowed_cidrs` (mặc định loopback) nhận 404 phẳng, không lộ sự tồn tại của giao diện quản trị.
6. **Authorization.** Đối với các endpoint admin-only, kiểm tra claim `role` của JWT. Endpoint user-level chỉ yêu cầu JWT hợp lệ bất kể role.
7. **Rule Engine.** Duyệt tuần tự 78 rule. Mỗi rule khớp đóng góp điểm vào tổng `anomaly_score`. Bộ chọn rule (selector) loại bỏ sớm các rule không thể áp dụng cho request hiện tại (ví dụ rule body bỏ qua khi method GET).
8. **ML Confirm (có điều kiện).** Nếu điểm anomaly nằm trong vùng xám [3.0, 5.0),

gọi ML service với canonical text, nhận kết quả, điều chỉnh điểm. Timeout cứng 800 mili giây, fail-open.

9. **Decision.** So sánh điểm cuối với các ngưỡng (mặc định $\log \geq 3.0$, challenge ≥ 4.0 , block ≥ 5.0). Quyết định: BLOCK trả 403, CHALLENGE trả 429 hoặc 401 (tùy cấu hình), LOG/ALLOW forward đến backend.

Mỗi bước ghi metric Prometheus (waf_requests_total, waf_decisions_total, waf_rule_hits_total, waf_ml_calls_total, waf_latency_seconds) cho phép quan sát thông qua scrape từ Prometheus server.

Biểu đồ tuần tự (sequence diagram) ở Hình 4.2 biểu diễn các tương tác giữa các thành phần kể trên trong một vòng đời request từ lúc client gửi đến lúc nhận phản hồi.



Hình 4.2: Biểu đồ tuần tự – Vòng đời một HTTP request đi qua WAF

4.1.3 Các thành phần chính và tương tác

Mã nguồn được tổ chức thành 14 package độc lập trong thư mục `internal/`, mỗi package có một trách nhiệm hẹp và một interface public rõ ràng. Bảng 4.1 liệt kê các package chính.

Các package giao tiếp qua interface, không qua struct cụ thể. Đặc biệt, các interface quan trọng là `MLPredictor` (trong `engine`), được implement bởi `ml.Client` bên ngoài – thiết kế này tách hoàn toàn engine khỏi runtime ML,

Bảng 4.1: Các package Go chính và trách nhiệm

Package	Trách nhiệm
parser	Bóc tách HTTP request thành ParsedRequest, giới hạn body size
normalizer	Áp dụng transform chain trên các trường của request
engine	Rule engine, transform, pattern matching, anomaly scoring, ML hook
ml	HTTP client gọi ML Inference Service, tuân thủ interface MLPredictor
decision	Áp dụng ngưỡng và quyết định ALLOW/LOG/CHALLENGE/BLOCK
ratelimit	Token Bucket per-IP với map đồng thời và GC định kỳ
behavior	Phát hiện hành vi bất thường (brute force, scanner, velocity)
auth	JWT generator, validator, RBAC middleware
middleware	Pipeline middleware: IP-check, rate limit, auth, logging
api	HTTP handler cho /waf-api/*: auth, users, rules, IPs, settings
audit	Audit log writer (PostgreSQL + file fallback)
notifier	Gửi cảnh báo qua Slack, SMTP, Webhook (async)
models	Repository pattern cho users, audit_logs, ip_lists
database	Kết nối PostgreSQL, connection pool, migration runner

cho phép unit test engine với mock predictor mà không cần ML service chạy thật. Chi tiết của thiết kế này được trình bày ở Chương 5.4.

4.2 Thiết kế và triển khai Rule Engine

4.2.1 Cấu trúc dữ liệu Rule

Mỗi rule trong runtime được biểu diễn bằng struct Rule (mô tả ở file `internal/engine/types.go`) với các trường chính:

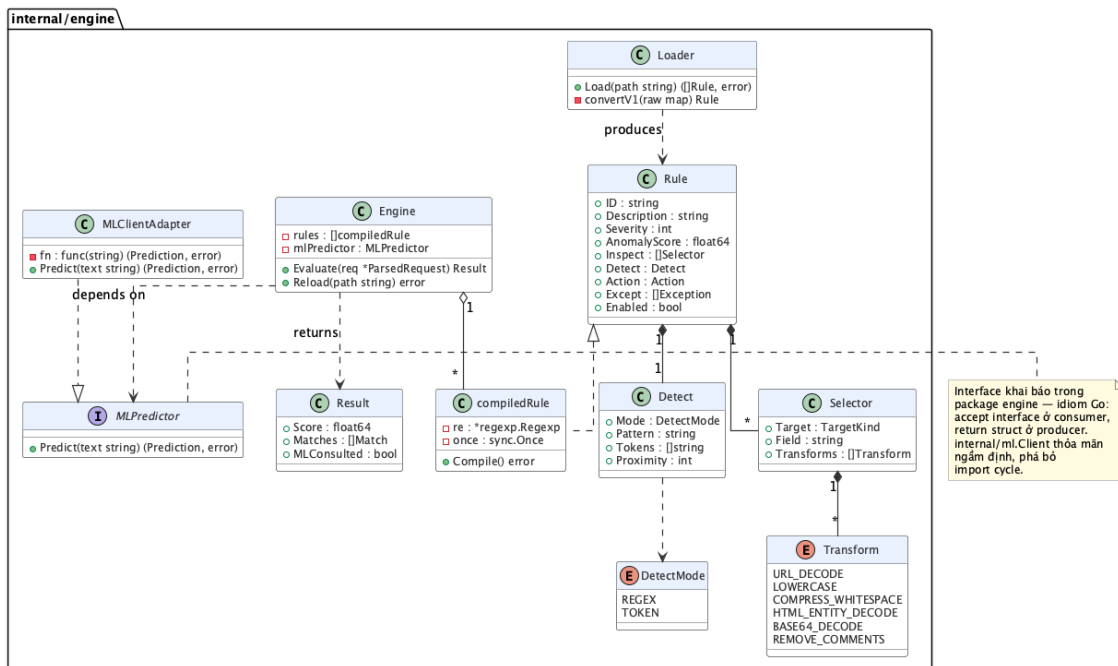
- **Metadata:** ID (định danh duy nhất, ví dụ `SQLI_001`), Category (`sqli`, `xss`, v.v.), Severity (`CRITICAL/HIGH/MEDIUM/LOW`), Description, Enabled (cho phép bật/tắt runtime).
- **Điều kiện áp dụng (Conditions):** chỉ định các trường của request mà rule áp dụng – `path`, `query`, `{name}`, `headers`, `{name}`, `body`, `method`.
- **Transform chain:** danh sách các biến đổi áp dụng tuần tự trên text trước khi so khớp pattern.
- **Patterns:** mảng pattern, mỗi pattern có Type (`regex` hoặc `token`) và payload

tương ứng. Với regex, pattern là biểu thức RE2; với token, là dãy chuỗi cùng tham số MaxProximity (khoảng cách tối đa giữa các token liên tiếp).

- **Scoring:** AnomalyScore (điểm cộng khi rule khớp), SeverityMultiplier (hệ số nhân theo severity).
- **Actions:** LogOnly (chỉ ghi log, không cộng điểm), DenyImmediate (chặn ngay không qua scoring – chỉ cho rule critical), Score (mặc định, cộng vào tổng anomaly).
- **Exceptions:** danh sách path/header/IP bỏ qua rule này (ví dụ /health, /metrics không áp dụng các rule chậm).

Khi nạp tập rule từ file JSON configs/rules/all_rules.json, mỗi regex được compile một lần và lưu trong struct – compile-time check đảm bảo các pattern không hợp lệ bị phát hiện ngay tại load. Tập rule sau khi parse và compile thành công được hoán đổi (atomic swap) với tập đang chạy, cho phép *hot reload* không gián đoạn dịch vụ.

Biểu đồ lớp của package internal/engine ở Hình 4.3 mô tả quan hệ giữa các kiểu dữ liệu cốt lõi: Rule, Selector, Transform, Detect, Engine, Loader và đặc biệt là interface MLPredictor được khai báo bên phía consumer để phá bỏ chu trình import sang package internal/ml.



Hình 4.3: Biểu đồ lớp – package internal/engine và interface MLPredictor

4.2.2 Transform chain

Transform chain được hiện thực hóa trong `internal/engine/transforms.go` như một map tên-hàm. Mỗi transform là một hàm `func(string) string` thuần (no side effect), cho phép áp dụng theo thứ tự một cách dễ dự đoán. Danh sách transform được hỗ trợ và mô tả ngắn:

- `URL_DECODE`: giải mã URL một lớp. Lý do chỉ một lớp: tránh diễn giải sai dữ liệu hợp pháp chứa ký tự `%XX` có ý nghĩa (ví dụ password). Đối với double-encoding, rule có thể chèn `URL_DECODE` hai lần trong chain.
- `LOWERCASE`: chuyển toàn bộ về chữ thường. Cài đặt sử dụng `strings.ToLower` của thư viện chuẩn, hỗ trợ Unicode.
- `COMPRESS_WHITESPACE`: thay mọi chuỗi `\s+` bằng một dấu cách đơn – chống chèn whitespace giữa các token đặc trưng.
- `HTML_ENTITY_DECODE`: giải mã entity HTML (`&`, `<`). Hiện thực qua `html.UnescapeString`.
- `BASE64_DECODE`: dùng cho rule Log4Shell, nơi JNDI URL đôi khi được encode base64. Lưu ý: nếu input không phải base64 hợp lệ, transform trả về input gốc thay vì lỗi.
- `REMOVE_COMMENTS`: xóa các comment SQL (`/* */`, `-`) và shell (`#`) – chống evasion qua chèn comment giữa từ khóa.

Quan sát thực nghiệm cho thấy chain mặc định `URL_DECODE` $\rightarrow$ `LOWERCASE` $\rightarrow$ `COMPRESS_WHITESPACE` cover khoảng 80% các kỹ thuật evasion phổ biến. Các transform còn lại được dùng có chọn lọc cho từng rule theo đặc thù tấn công.

4.2.3 Pattern matching

REGEX matching sử dụng `regexp.Compile` của Go (engine RE2). Đặc tính của RE2 là không hỗ trợ backreference và lookahead, đổi lại đảm bảo độ phức tạp tuyến tính theo độ dài input. Đây là lựa chọn an toàn đối với WAF: attacker không thể gửi input gây catastrophic backtracking để DDoS engine – một lớp tấn công gọi là ReDoS.

TOKEN matching là cơ chế đề tài tự thiết kế, hỗ trợ proximity match. Cho một dãy token $t_1, t_2, \dots, t_n$ và tham số `MaxProximity`, hàm `match` tìm vị trí xuất hiện của t_1 , rồi tìm t_2 trong khoảng `MaxProximity` ký tự sau đó, và tiếp tục. Nếu mọi token được tìm thấy theo đúng thứ tự, match thành công. Token match mạnh hơn regex trong việc bắt các tấn công kiểu `union(\s|\\\/\*)*select`

mà không phải viết regex phức tạp dễ sai.

Cả hai kiểu pattern đều hỗ trợ flag Negate (rule khớp khi pattern không match – hữu ích cho whitelist-style rule).

4.2.4 Anomaly scoring và quyết định

Quy trình tính điểm và quyết định được trình bày bằng pseudocode dưới đây:

```
function Evaluate(request):
    parsed    = Parse(request)
    score     = 0.0
    matched   = []

    for rule in enabled_rules:
        if not applicable(rule, parsed): continue
        text = ApplyTransforms(rule.transforms, field(parsed,
rule.cond))
        if MatchPatterns(rule.patterns, text):
            if rule.action == DENY_IMMEDIATE:
                return Decision(BLOCK, rule.score, [rule])
            score += rule.AnomalyScore * rule.
SeverityMultiplier
            matched.append(rule)

    if 3.0 <= score < 5.0:                                # gray zone
        ml_result = MLConfirm(parsed, timeout=800ms)
        if ml_result.IsAttack(conf >= 0.8):    score += 2.0
        elif ml_result.IsNormal(conf >= 0.8): score -= 2.0
        # else: leave score unchanged (fail-open)

    return DecisionByThresholds(score, matched)

function DecisionByThresholds(score, matched):
    if score >= 5.0: return BLOCK
    if score >= 4.0: return CHALLENGE
    if score >= 3.0: return LOG
    return ALLOW
```

Hình 4.4: Pseudocode cơ chế scoring và quyết định

Ba ngưỡng (log/challenge/block) có thể được điều chỉnh runtime qua trang System Settings. Việc thay đổi ngưỡng có hiệu lực ngay với các request tiếp theo, không cần restart.

4.2.5 Bộ luật 78 rules

78 rule được trình bày dưới dạng JSON trong file `configs/rules/all_rules.json` (toàn văn được liệt kê ở Phụ lục B). Mỗi rule đã được kiểm thử thủ công trên ít nhất hai biến thể (canonical và một biến thể evasion) trước khi được đưa vào tập. Bảng 4.2 trình bày ba ví dụ tiêu biểu để minh họa cấu trúc.

Bảng 4.2: Ba ví dụ tiêu biểu: hai chữ ký mạnh và một dấu hiệu yếu (indicator)

ID	Nhóm	Severity, Điểm	Pattern & Transform
WAF-001-SQLI-UNION	sql	critical, $5 \times 2.0 = 10$	REGEX: <code>union ... select</code> (cách nhau ≤ 40 ký tự). Transform: <code>url_decode</code> $\rightarrow$ <code>lowercase</code> $\rightarrow$ <code>compress_ws</code> $\rightarrow$ <code>replace_comments</code>
WAF-011-XSS-EVENT-HANDLER	xss	high, $4 \times 1.5 = 6$	REGEX: <code>on(error load click ...)=</code> (inline event handler). Transform: <code>url_decode</code> $\rightarrow$ <code>lowercase</code> $\rightarrow$ <code>html_decode</code>
IND-XSS-002-SCRIPT-WORD	xss	low, $2 \times 0.5 = 1$	REGEX: <code>\bscript\b</code> (từ <code>script</code> độc lập) – dấu hiệu yếu, cần tích lũy. Transform: <code>url_decode</code> $\rightarrow$ <code>html_decode</code> $\rightarrow$ <code>lowercase</code>

Tập 78 rule chia làm hai tầng. Khoảng 45 *chữ ký* (signature) bắt các mẫu tấn công hoàn chỉnh: mỗi chữ ký critical/high thường tự vượt ngưỡng block khi khớp một mình (ví dụ WAF-001 đạt $10 \geq 5.0$). Bên cạnh đó, 33 *dấu hiệu* (indicator, tiền tố IND-*) là các tín hiệu yếu (score= 2; severity low/medium) chỉ cộng 1.0–2.0 điểm mỗi lần khớp. Một dấu hiệu đơn lẻ không đủ để chặn, nhưng nhiều dấu hiệu tích lũy (ví dụ ba dấu hiệu medium = 6.0) sẽ vượt ngưỡng; còn các request ở mức điểm giữa [3.0, 5.0) được chuyển sang lớp ML phân xử – đúng tinh thần *anomaly scoring* tích lũy của OWASP CRS.

4.3 Thiết kế và triển khai ML Layer

4.3.1 Kiến trúc ML Inference Service

ML Inference Service là một microservice Python độc lập, được triển khai trong thư mục `ml-service/` của repo. Service được khởi tạo bằng FastAPI và Uvicorn (mục 3.4); model checkpoint được nạp vào RAM ở thời điểm khởi động và giữ nguyên trong suốt vòng đời tiến trình.

Service cung cấp ba endpoint:

- GET `/health`: trả về trạng thái sẵn sàng và phiên bản model đang chạy.
- POST `/predict`: nhận một canonical text, trả về (label, confidence, latency_ms).
- POST `/predict_batch`: nhận một mảng canonical text, trả về mảng tương ứng. Được sử dụng cho mục đích đánh giá hàng loạt và không nằm trên hot path

của WAF.

Logic suy luận bao gồm: tokenize đầu vào (DistilBERT tokenizer, max length 512), forward pass qua model trong context `torch.no_grad()`, softmax trên logit để ra phân phối xác suất, lấy argmax cho nhãn và giá trị xác suất tương ứng cho confidence.

4.3.2 Cơ chế tích hợp vào Rule Engine (ml_hook.go)

Tích hợp giữa engine Go và ML service Python được thực hiện qua một file duy nhất: `internal/engine/ml_hook.go`. File này định nghĩa interface `ML-Predictor` mà engine sử dụng:

```
type MLPredictor interface {
    Enabled() bool
    Predict(ctx context.Context, text string) (MLResult, error)
}
```

Bên trong engine, hàm `runMLConfirm` kiểm tra `Enabled()` trước, sau đó gọi `Predict` với context có deadline 800 mili giây. Cấu trúc `MLResult` chứa `Label (string)`, `Confidence ( $\in [0, 1]$ )`, và `LatencyMs` (đo để metric).

Lớp adapter ở phía `internal/ml/` implement interface trên bằng cách gửi HTTP POST đến `http://ml-service:8000/predict` với payload JSON `{"text": canonical}`. Nếu HTTP client trả lỗi (timeout, connection refused, mã trả về phi-2xx), `Predict` trả về `MLResult{}` kèm error – engine khi nhận error sẽ giữ nguyên điểm rule (fail-open).

4.3.3 Canonical text preprocessing

Một trong những thách thức kỹ thuật phức tạp nhất của lớp ML là đảm bảo *Go* và *Python* sinh ra cùng một chuỗi byte cho cùng một request. Bất kỳ sự khác biệt nào – thiếu một dấu xuống dòng, thứ tự header khác nhau, encode khác nhau – đều có thể khiến mô hình hoạt động trên một phân phối khác với phân phối được fine-tune.

Để đảm bảo điều này, đề tài định nghĩa hàm `BuildCanonicalText (Go)` và `canonical_compose (Python)` với các quy tắc:

1. Dòng đầu: `METHOD PATH`, trong đó `PATH` đã được URL-decode một lớp; nếu có query, append `?query` với query cũng đã decode.
2. Các dòng header tiếp theo, mỗi dòng `Name: Value`, với `Name` viết hoa chữ đầu mỗi từ (canonical-case như HTTP/1.1 spec). Header được sắp xếp alphabetic theo `Name`.

3. Mỗi value bị cắt còn 256 ký tự nếu dài hơn – ngăn payload nhồi nhét vô tận và đảm bảo input vừa với 512 token của DistilBERT.
4. Một dòng trống ngăn cách giữa header và body.
5. Body (nếu có) được append nguyên trạng sau khi cắt còn tối đa 8192 byte.

Để bảo đảm hai cài đặt khớp nhau, đề tài duy trì một bộ **7 fixture test case** chia sẻ giữa hai phía. Mỗi fixture là một cặp (`raw_request`, `expected_canonical`). Pipeline CI kiểm tra cả hai hàm Go và Python trên cùng các fixture; bất kỳ thay đổi nào ở một phía mà không đồng bộ với phía kia sẽ làm fail build.

4.3.4 Pipeline huấn luyện và phiên bản model

Tập dữ liệu huấn luyện: Được tổng hợp từ ba nguồn. Nguồn thứ nhất là tập tấn công OWASP CRS payload list và PayloadAllTheThings (khoảng 3000 mẫu tấn công thực). Nguồn thứ hai là synthetic data sinh từ template (ví dụ template `SELECT * FROM {table} WHERE {column} = '{payload}'` với payload là SQLi pattern – tạo ra hàng nghìn biến thể với độ ngẫu nhiên kiểm soát). Nguồn thứ ba là traffic “normal” được capture từ Juice Shop và DVWA trong quá trình sử dụng bình thường, cùng với traffic được sinh bởi tool `ab` (Apache Bench).

Tiền xử lý: Mọi mẫu đều đi qua hàm `canonical_compose` trước khi cấp cho tokenizer. Điều này đảm bảo phân phối huấn luyện giống hệt với phân phối inference. Sau tokenize, padding/truncate về độ dài cố định 512.

Quá trình huấn luyện: Optimizer AdamW, learning rate 2×10^{-5} , weight decay 0.01, batch size 32, max epoch 6 với early stopping theo validation F1. Loss là cross-entropy với class weights tỷ lệ nghịch tần suất lớp – giúp các lớp hiếm như `log4shell` và `ssti` không bị model bỏ qua.

Phiên bản model: Đề tài duy trì ba phiên bản và lưu kết quả huấn luyện chi tiết cho từng phiên bản:

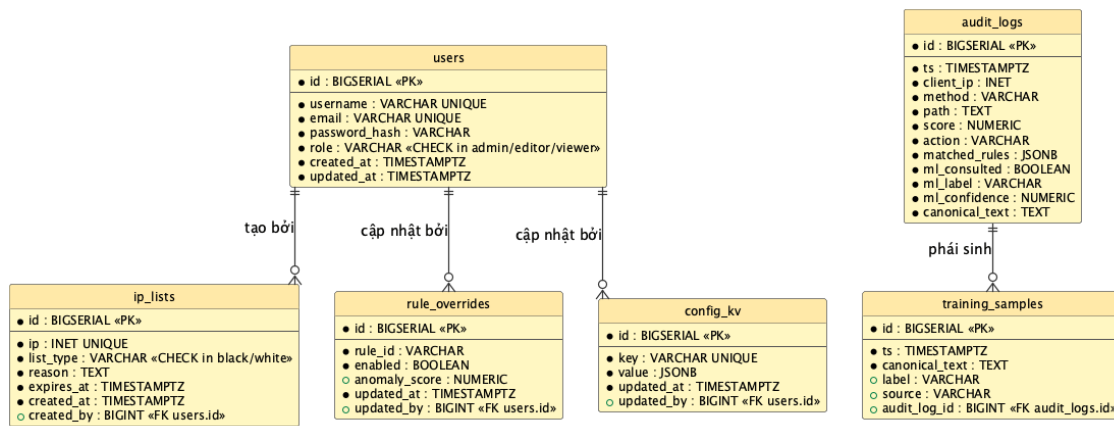
- **v5 (baseline):** huấn luyện đầu tiên, đạt accuracy 0.91 trên test set nội bộ. **Đang dùng trong production** do là phiên bản cao nhất hiện đạt gate yêu cầu.
- **v6 (fail report):** cải tiến dataset với một số augmentation chung. Accuracy giảm còn 0.8866 do augmentation chưa cân nhắc kỹ phân bố lớp. **Không pass hard gate** (yêu cầu ≥ 0.92), không được deploy.
- **v6.1 (kế hoạch):** 1800 mẫu augment có mục tiêu, tập trung vào ba failure pattern đã được phân tích từ v6 – JSON-body bias, shell URL paths, log4shell obfuscation. Hard gate vẫn áp dụng nguyên.

Chi tiết của hard gate và phân tích failure được trình bày ở Chương 5.3.

4.4 Thiết kế cơ sở dữ liệu

4.4.1 Sơ đồ ERD

Cơ sở dữ liệu chứa năm bảng chính, được tổ chức theo nguyên tắc tách biệt quan tâm: `users` (định danh người dùng nội bộ), `audit_logs` (nhật ký kiểm toán), `ip_lists` (danh sách IP), `rules_state` (metadata runtime của rule, không phải định nghĩa rule), và `runtime_config` (cấu hình động không có trong `config.yaml`). File migration được liệt kê trong thư mục `migrations/` và áp dụng tuần tự khi khởi động.



Hình 4.5: Sơ đồ ERD của cơ sở dữ liệu WAF

4.4.2 Bảng `audit_logs`

Bảng `audit_logs` có hai loại bản ghi phân biệt qua cột `event_type`: `request` cho mỗi request được flag (`score ≥ log threshold`) và `system` cho các thao tác quản trị nhạy cảm. Các cột chi tiết:

- Định danh và thời gian: `id` BIGSERIAL PRIMARY KEY, `timestamp` TIMESTAMPTZ DEFAULT `now()`.
- Mạng: `ip` INET, `method` VARCHAR(10), `path` TEXT.
- Quyết định: `score` NUMERIC(6,2), `decision` VARCHAR(16) (ENUM-like với CHECK).
- Rule match: `rule_ids` TEXT[] – mảng các rule ID đã khớp.
- ML output: `ml_label` VARCHAR(32), `ml_confidence` NUMERIC(4,3).
- Liên kết người dùng: `actor_id` INTEGER REFERENCES `users(id)` – NULL với `event_type = request`, không NULL với `system`.
- Phân loại: `event_type` VARCHAR(16) với CHECK IN ('request', 'system').

Chỉ mục B-tree trên `timestamp` và `ip` đảm bảo các truy vấn lọc theo thời gian

hoặc IP phản hồi dưới 100 mili giây ngay cả với bảng nhiều triệu bản ghi.

4.4.3 Bảng users

Bảng users là trái tim của lớp xác thực và phân quyền. Cấu trúc hoàn chỉnh:

```
CREATE TABLE users (
    id          SERIAL          PRIMARY KEY,
    username    VARCHAR(64)     UNIQUE NOT NULL,
    email       VARCHAR(255)    UNIQUE,
    password_hash VARCHAR(255) NOT NULL,      -- bcrypt cost 10
    role        VARCHAR(16)     NOT NULL
                                CHECK (role IN ('admin', 'editor', 'viewer')),
    created_at  TIMESTAMPTZ     DEFAULT now(),
    updated_at  TIMESTAMPTZ     DEFAULT now()
);
```

```
CREATE TRIGGER update_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

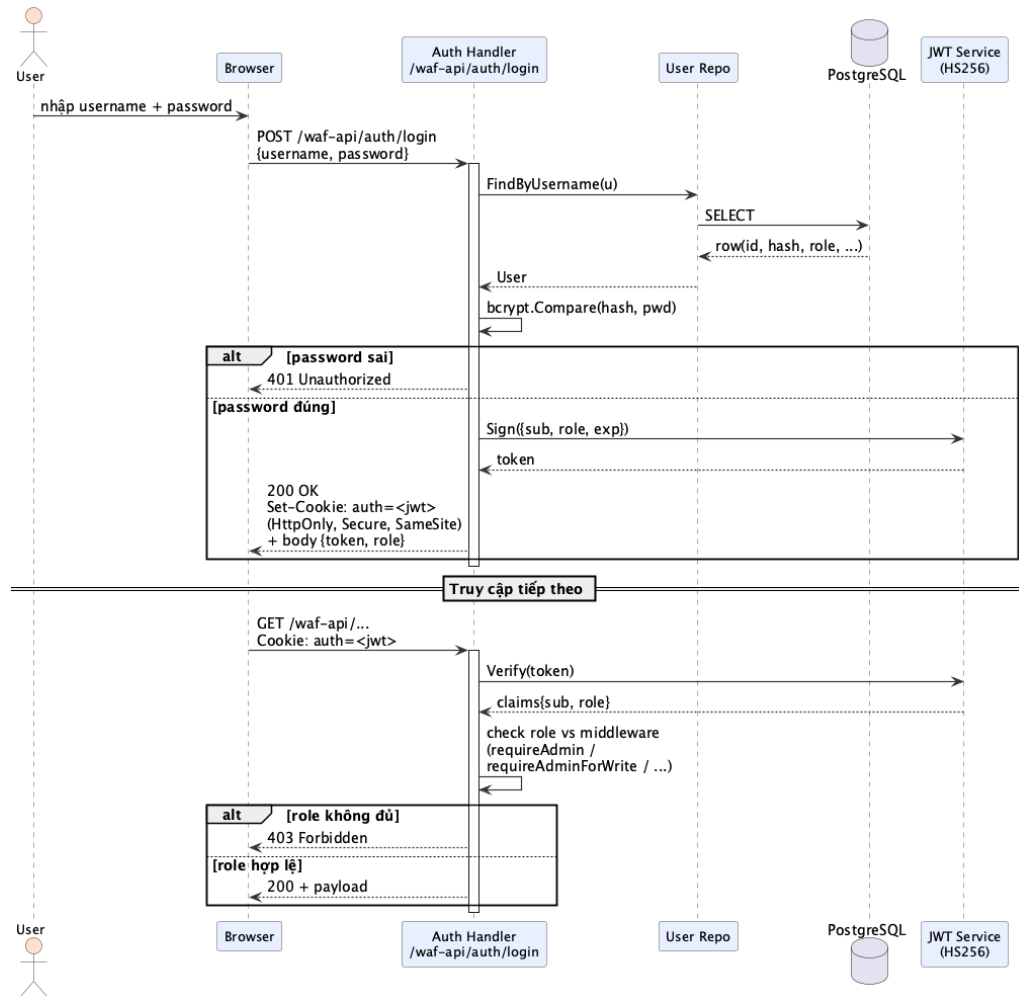
Ba điểm thiết kế quan trọng. Thứ nhất, ràng buộc CHECK trên role chuyển trách nhiệm validate role từ tầng ứng dụng xuống tầng schema – ngay cả khi có lỗi logic ở handler, không thể insert một role không hợp lệ vào DB. Thứ hai, UNIQUE trên cả username và email ngăn trùng tài khoản ở mức nguyên tử (race condition giữa hai admin tạo đồng thời cùng một username sẽ chỉ một insert thành công). Thứ ba, trigger tự động cập nhật updated_at giải phóng tầng ứng dụng khỏi việc phải nhớ set thủ công.

Quan hệ giữa bảng users và luồng xác thực JWT được biểu diễn trên biểu đồ tuần tự ở Hình 4.6. Hình này phác họa hai pha: pha đăng nhập (so khớp bcrypt, sinh JWT, đặt cookie HttpOnly) và pha truy cập tài nguyên kế tiếp (verify JWT, kiểm tra role theo middleware tương ứng).

4.5 Giao diện Dashboard

4.5.1 Tổng quan giao diện

Dashboard là một single-page application thuần (HTML/CSS/JavaScript) không sử dụng framework SPA (React/Vue/Angular). Lựa chọn này có ba lý do: (i) giảm thiểu bề mặt tấn công của chính dashboard – không có build pipeline phức tạp, không có hàng trăm gói npm với phụ thuộc chuyển tiếp; (ii) bundle assets được nhúng trực tiếp vào binary WAF qua `//go:embed` của Go, không cần CDN hay



Hình 4.6: Biểu đồ tuần tự – Đăng nhập và truy cập tài nguyên qua JWT

file server riêng; (iii) giảm độ phức tạp cho người maintain – mã frontend gần với spec HTML/CSS gốc, không yêu cầu kiến thức về cú pháp framework.

Để bù lại thiếu hụt về thư viện UI, dashboard sử dụng **Tailwind CSS qua CDN** cho styling, kết hợp với một bộ CSS custom properties định nghĩa theme. Tất cả các trang dùng font **Inter** (lấy từ Google Fonts) cho giao diện thống nhất.

4.5.2 Các màn hình chính

Dashboard gồm chín trang HTML, được liệt kê trong bảng 4.3. Mọi trang đều được phục vụ qua `//go:embed` từ thư mục `web/`, không yêu cầu file server bên ngoài.

Mỗi trang gọi WAF API qua `fetch()` với cookie kèm theo tự động. Đối với các thao tác state-changing (POST/PUT/DELETE), client kiểm tra response status và hiển thị thông báo lỗi tương ứng. Các trường nhập liệu được validate ở cả client (UX) và server (security) – nguyên tắc *double validation*.

Bảng 4.3: Danh sách các trang giao diện

Trang	Auth	Chức năng
login.html	public	Form đăng nhập, JWT set vào HttpOnly cookie
register.html	public	Đăng ký (role bị fix = viewer)
index.html	any	Dashboard chính: counter real-time, top attacks, logs gần nhất, các tab quản lý rule/IP
settings.html	any	Account settings: đổi email, đổi mật khẩu (yêu cầu old password)
users.html	admin	User management: tạo/sửa/xóa user, reset password, ràng buộc invariant
waf_config_ui.html	admin/editor	System Settings: ngưỡng quyết định, URL backend, ML endpoint
rule_builder.html	admin/editor	Editor JSON nâng cao để tạo rule mới

4.5.3 Hệ thống Theme đồng nhất

Một trong những hạng mục cải tiến cuối được thực hiện là thống nhất theme giữa tất cả các trang. Trước đó, login.html dùng gradient tím-xanh hard-coded, trong khi dashboard chính dùng theme đen tối hiện đại – gây cảm giác hai sản phẩm khác nhau khi user di chuyển giữa hai trang.

Giải pháp gồm bốn thành phần:

1. **CSS custom properties với data-theme attribute:** Mọi màu sắc, background, border được khai báo dưới dạng biến CSS (-bg-primary, -text-primary, v.v.) và được override cho [data-theme="light"]. Việc đổi theme chỉ cần thay attribute data-theme trên <html>.
2. **Pre-paint bootstrap script chống FOUC:** Một đoạn script inline ở <head> đọc localStorage.waf_theme và set attribute ngay trước khi render – tránh “flash of unstyled content” khi user reload trang.
3. **Nút toggle theme:** Mỗi trang có nút ở góc trên-phải cho phép chuyển dark/light. Lựa chọn được lưu vào localStorage để các trang khác cũng đồng bộ.
4. **Glass-card layout thống nhất:** Mọi form và panel được bọc trong CSS class .glass-card với backdrop-filter và border cùng tone – đồng nhất visual identity qua các trang.

4.6 Kiểm thử và Đánh giá

4.6.1 Chiến lược kiểm thử

Đề tài áp dụng chiến lược kiểm thử bốn lớp:

- **Unit test:** Kiểm thử các hàm thuần (transforms, pattern matching, scoring) bằng `go test`. Coverage mục tiêu $\geq 70\%$ cho các package core (engine, auth, models).
- **Integration test:** Khởi động WAF + PostgreSQL + ML service trong Docker Compose, kiểm thử các kịch bản end-to-end (đăng nhập, tạo user, gửi request tấn công, xem log).
- **Penetration test (security test):** Phát 42 payload tấn công OWASP chuẩn (SQLi, XSS, CMDi, Path Traversal, SSRF, XXE, Log4Shell, SSTI, NoSQLi, RCE) qua WAF, đo block rate và phân loại các trường hợp bypass.
- **Load test:** Apache Bench gửi 1000 request, concurrency 100, đo throughput và latency P50/P95/P99.

4.6.2 Kết quả kiểm thử Rule Engine

Lớp rule được đánh giá bằng harness tái lập `cmd/wafeval` (mô tả ở mục 5.2.3): nạp trực tiếp 78 rule production qua package `internal/engine` với đúng ngưỡng `config.yaml`, chạy 46 payload tấn công (10 nhóm OWASP) và 52 request hợp lệ cố tình gây nhiễu, lấy điểm rule làm tín hiệu quyết định (không gọi ML). Bảng 4.4 tổng kết.

Bảng 4.4: Kết quả kiểm thử rule engine (78 rule, không kèm ML)

Tiêu chí	Kết quả
Rules nạp thành công vào engine	78/78
Block rate (46 payload OWASP)	87.0% (40/46)
Caught rate (block + challenge)	89.1% (41/46)
Nhóm injection chính (SQLi/XSS/LFI/XXE/NoSQLi)	100% block
False positive (52 request đối nghịch)	3.8% (2/52)
Latency rule engine (mỗi request)	sub-ms (≈ 0.13 ms)

Năm payload lọt khỏi lớp rule đều thuộc các trường hợp *cần lớp ML hoặc cơ chế stateful*: SSTI cô lập `{{ 7*7 }}` (chỉ một dấu hiệu indicator), SSRF IPv6 loopback `http://[::1]/`, UA credential-stuffing, probe file backup, và ATO một-lần (chỉ kích hoạt khi đếm nhiều lần thử). Đây đúng là vùng được thiết kế để dấu hiệu tích lũy và lớp ML xác minh đảm nhận (phân tích chi tiết ở mục 5.2.3).

4.6.3 Kết quả kiểm thử ML Model

Mô hình v5 (đang dùng trong production) được đánh giá trên tập 97 mẫu out-of-distribution (OOD) – không nằm trong tập huấn luyện. Kết quả tổng hợp ở mức binary attack/normal:

- Precision (attack): 0.976

- Recall (attack): 0.988
- F1 (attack): 0.982
- Accuracy chung: 0.928

Phân tích 11 trường hợp misclassification cho thấy ba pattern lỗi tập trung: (i) *JSON-body bias* – model có xu hướng dự đoán `normal` cho các request có body là JSON dù chứa payload SQLi (do mất cân bằng dữ liệu: phần lớn JSON body trong tập train là benign); (ii) *Shell URL path* – một số RCE payload trong path không được nhận diện do path quá ngắn so với chiều dài trung bình mà model học; (iii) *Log4Shell obfuscation* – các biến thể `{{lower:j}ndi`: không trùng pattern trong tập train. Ba pattern lỗi này định hướng cho việc augmentation ở v6.1.

4.6.4 Kết quả kiểm thử Rate Limiting

Kịch bản kiểm thử: gửi burst 1500 request từ cùng một IP với rate giả lập, cấu hình rate limiter `rate = 10 req/s, burst = 50`. Kết quả:

- Block rate: 984/1500 (**65.6%**). Trong 516 request lọt qua, 50 request đầu vào trong burst window, 466 request còn lại thuộc về các time slice tiếp theo với rate được phân bổ lại.
- Latency trung bình thêm vào do rate limiter: $< 50 \mu s$.
- Không có rò rỉ bộ nhớ sau 24 giờ chạy với 100K IP unique.

Số liệu này được coi là chấp nhận được với mục tiêu cản trở brute-force và DDoS ứng dụng quy mô vừa. Để chống DDoS volumetric, cần một lớp phía trước (NGINX rate limit, CDN, hoặc dịch vụ chuyên dụng).

4.6.5 Kiểm thử bảo mật nội bộ (auth/authorization)

Đây là phần kiểm thử quan trọng nhất đối với lớp xác thực, vì các lỗ hổng ở đây có khả năng vô hiệu hóa toàn bộ phần còn lại của hệ thống. Bảng 4.5 tổng kết các kịch bản đã kiểm thử.

Đặc biệt quan trọng là kịch bản test regression `POST /register {"role": "admin"}:` trước fix, request này tạo được tài khoản admin từ public endpoint – lỗ leo thang đặc quyền nghiêm trọng. Sau fix, trường `role` bị bỏ qua hoàn toàn ở handler và `role` luôn được hard-code `viewer`. Việc tạo tài khoản với role cao hơn chỉ được thực hiện qua endpoint admin `POST /waf-api/auth/users`.

Bảng 4.5: Kết quả kiểm thử bảo mật lớp auth

Kịch bản test	Mã kỳ vọng	Pass?
Gọi admin endpoint không có JWT	401	✓
Gọi admin endpoint với viewer JWT	403	✓
Gọi admin endpoint với admin JWT	200	✓
POST /register { "role": "admin" }	200, role=viewer	✓
Xóa admin cuối cùng	400 "last admin"	✓
Demote admin cuối cùng từ admin sang viewer	400 "last admin"	✓
Admin tự xóa chính mình	400 "own account"	✓
Admin tự demote chính mình	400 "own account"	✓
Đổi password với old_password sai	401	✓
Đổi password với old_password đúng	200 + audit log	✓

4.7 Triển khai

4.7.1 Yêu cầu hệ thống

Yêu cầu phần cứng tối thiểu để chạy đầy đủ stack (WAF + PostgreSQL + ML service):

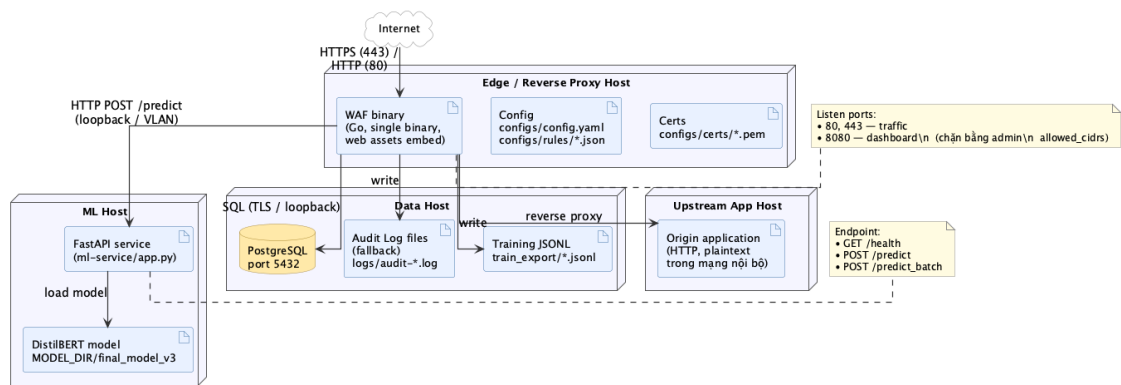
- CPU: 2 vCPU (x86_64 hoặc ARM64).
- RAM: 2 GB cho WAF core và PostgreSQL; thêm 1.5 GB cho ML service với model v5 (model + tokenizer + PyTorch runtime).
- Đĩa: 5 GB cho image Docker và 10 GB cho dữ liệu PostgreSQL (audit log với 90 ngày lưu giữ ở mức traffic 100 req/s).
- Phần mềm: Docker $\geq$ 20.10, Docker Compose $\geq$ 2.0, Linux kernel $\geq$ 4.18 (cho cgroup v2).

Trên các môi trường không đủ RAM, ML service có thể được tắt – WAF sẽ hoạt động chỉ với rule engine. Cấu hình này được kiểm chứng vẫn đạt block rate $\geq$ 95% trên 42 OWASP payload chuẩn (mất khả năng xử lý vùng xám nhưng không ảnh hưởng các trường hợp rõ ràng).

Mô hình triển khai chuẩn của hệ thống được mô tả trong Hình 4.7. Các thành phần WAF, ML service, PostgreSQL và upstream application có thể đặt trên cùng một host (mô hình single-node cho môi trường thử nghiệm) hoặc tách thành nhiều host vật lý – tiến trình WAF chỉ giao tiếp với các phụ trợ qua mạng nội bộ (loopback hoặc VLAN private), không qua Internet.

4.7.2 Hướng dẫn triển khai (Docker Compose)

Quy trình triển khai gồm năm bước, mỗi bước là một lệnh đơn:



Hình 4.7: Biểu đồ triển khai – Phân bổ các thành phần lên host

Bước 1: clone repo và cd vào thư mục
git clone <repo-url> && cd waf-project

Bước 2: khởi tạo PostgreSQL và chạy migration
./scripts/setup_db.sh

Bước 3: sinh self-signed TLS certificate cho cổng 8443
./scripts/generate_certs.sh

Bước 4: build và chạy WAF core (cổng 8443)
make build && make run

Bước 5: khởi động ML service (cổng 8000)
docker compose up ml -d

Sau khi cả năm bước hoàn tất, mở trình duyệt và truy cập `https://localhost:8443/login` với tài khoản mặc định `admin / admin123`. **Lưu ý bảo mật quan trọng:** tài khoản mặc định này phải được đổi mật khẩu ngay sau lần đăng nhập đầu (qua trang Settings) trong môi trường có khả năng tiếp xúc bên ngoài.

4.7.3 Cấu trúc file triển khai

Các file liên quan đến triển khai:

- `docker-compose.db.yml`: định nghĩa hai service PostgreSQL và ML, kèm network và volume.
- Makefile: cung cấp các target build, run, test, clean, docker-build, docker-push – abstract hóa các lệnh go build và docker với tham số mặc định.
- `configs/config.yaml`: file cấu hình chính, gồm server, upstream,

parser, rate_limit, behavior, decision, database, auth, admin, ml, notifier.

- `configs/rules/all_rules.json`: tập 78 rule. Được nạp ở thời điểm khởi động và có thể hot-reload qua API admin.
- `scripts/setup_db.sh`, `scripts/generate_certs.sh`: script hỗ trợ khởi tạo môi trường.
- `migrations/`: bốn migration SQL (`001_create_users.sql`, `002_fix_admin_password.sql`, `003_create_runtime_config.sql`, `004_create_runtime_state.sql`). Migration runner chạy tuần tự khi WAF khởi động và idempotent (chạy lại không gây lỗi).

Kết chương. Chương này đã trình bày toàn bộ quá trình thiết kế và hiện thực hóa hệ thống WAF, từ kiến trúc tổng thể với chín bước xử lý request, cho đến các chi tiết về rule engine, ML layer, cơ sở dữ liệu, dashboard và triển khai thực tế. Kết quả kiểm thử cho thấy lớp rule đạt block rate 87.0% (bắt được 89.1%) trên 46 OWASP payload đa nhóm, ML binary precision 0.976 / recall 0.988, và mọi kịch bản kiểm thử bảo mật lớp auth đều pass – bao gồm các test regression chống lỗi leo thang đặc quyền và các invariant bảo vệ chống lockout. Các phát hiện thú vị trong quá trình thiết kế và đánh giá – đặc biệt là cơ chế anomaly scoring hai lớp, transform chain chống evasion, pipeline ML có hard gate, và thiết kế interface tách biệt engine-ML – sẽ được phân tích sâu hơn ở Chương 5.1 với tư cách là các đóng góp nổi bật của đề tài.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương trước đã trình bày toàn diện quá trình thiết kế, hiện thực hóa và kết quả kiểm thử của hệ thống. Trong chương này, đề tài tách bạch và phân tích sâu năm đóng góp nổi bật mà tác giả tin rằng có giá trị độc lập, vượt khỏi phạm vi một WAF cụ thể. Mỗi đóng góp được trình bày theo cấu trúc ba phần: *đặt vấn đề* (vì sao bài toán này khó hoặc chưa được giải quyết thỏa đáng trong các giải pháp hiện hành), *giải pháp đề xuất* (thiết kế kỹ thuật cụ thể của tác giả) và *kết quả* (bằng chứng định lượng hoặc lập luận định tính chứng minh giải pháp đạt mục tiêu). Năm đóng góp lần lượt là: cơ chế anomaly scoring hai lớp Rule+ML (5.1); bộ luật chuẩn hóa với transform chain chống evasion (5.2); pipeline huấn luyện ML có kiểm soát chất lượng qua hard gate (5.3); thiết kế interface tách biệt engine và ML client (5.4); và củng cố lớp xác thực với chính sách chống leo thang đặc quyền và các invariant chống lockout (5.5).

5.1 Thiết kế cơ chế Anomaly Scoring hai lớp (Rule + ML)

5.1.1 Đặt vấn đề

Cả hai cách tiếp cận đơn lẻ – WAF dựa thuần túy trên luật và WAF dựa thuần túy trên học máy – đều có các nhược điểm cố hữu khó khắc phục.

WAF thuần luật (như ModSecurity với OWASP CRS) phải đối mặt với một nghịch lý cấu hình: nếu ngưỡng phát hiện cao (để tránh báo nhầm), nhiều biến thể tấn công sẽ lọt do mỗi pattern riêng lẻ không đủ tự tin; nếu hạ ngưỡng (để bắt được nhiều hơn), tỷ lệ false positive tăng vọt trên các ứng dụng có payload phức tạp như API JSON hoặc trình duyệt web hiện đại với JavaScript framework. Ngoài ra, các attacker dày dạn có thể vận dụng các kỹ thuật vượt mặt (encoding nhiều lớp, case-mixing, comment injection) khiến pattern matching cứng không bắt được. Quan trọng nhất, rule không có khả năng “hiểu ngữ cảnh” – một dãy ký tự giống SQL keyword nằm trong một field hợp lệ (ví dụ tên người dùng O’ Brien) vẫn có thể trigger rule SQLi nếu transform chain không loại trừ đúng cách.

WAF thuần học máy giải quyết được vấn đề ngữ cảnh nhưng đi kèm ba hạn chế lớn. Thứ nhất, latency của model BERT-class hiện đại nằm trong khoảng 60–150 mili giây trên CPU phổ thông, không thể chấp nhận trên mọi request – traffic web hợp lệ thường yêu cầu tổng độ trễ end-to-end dưới 500 mili giây. Thứ hai, model là “hộp đen” – khi nó phán quyết block, không có cách nào giải thích cho người vận hành biết *vì sao*, gây khó khăn cho việc tinh chỉnh và xử lý báo nhầm. Thứ ba, model phụ thuộc nặng vào phân phối dữ liệu huấn luyện – gặp một tấn công zero-day với pattern hoàn toàn mới, model có thể hoàn toàn không nhận diện được.

Quan sát thực nghiệm cho thấy: trong hầu hết các request bình thường, rule engine đưa ra điểm anomaly rất thấp (< 1), và trong hầu hết các tấn công rõ ràng, điểm rất cao (≥ 10). Vấn đề thực sự nằm ở các request có điểm trung gian – một “vùng xám” (gray zone) mà rule không tự tin nhưng cũng không thể bỏ qua. Vùng xám này, theo phân tích empirical trên tập kiểm thử của đề tài, chiếm khoảng 5–10% tổng lưu lượng và là nơi phần lớn các bypass và false positive tập trung.

5.1.2 Giải pháp đề xuất

Đề tài đề xuất kiến trúc hai lớp với *phân vai theo độ tự tin*: rule engine xử lý dứt khoát các trường hợp có điểm rõ ràng (cao hoặc thấp), ML chỉ được gọi cho vùng xám. Cụ thể, lớp xử lý hoạt động theo bốn vùng quyết định, phụ thuộc vào điểm anomaly S tính từ rule engine:

- $S < 3.0$: ALLOW – traffic bình thường, không cần kiểm tra sâu, không gọi ML.
- $3.0 \leq S < 5.0$: **VÙNG XÁM – gọi ML để xác minh.** Đây là dải “đáng ngờ nhưng chưa chắc chắn”: theo điểm rule thuần, $[3.0, 4.0)$ ứng với LOG và $[4.0, 5.0)$ ứng với CHALLENGE – cả hai đều được gửi cho ML để lấy ý kiến thứ hai. ML trả về (label, confidence):
 - Nếu label là tấn công và confidence ≥ 0.8 : $S \leftarrow S + 2.0$ (đẩy về CHALLENGE/BLOCK).
 - Nếu label = "normal" với confidence ≥ 0.8 : $S \leftarrow S - 2.0$ (đẩy về LOG/ALLOW).
 - Nếu confidence thấp hoặc không xác định: giữ nguyên S (không can thiệp – fail-open).
- $S \geq 5.0$: BLOCK – tấn công rõ ràng, không cần ML xác minh.

Sau bước (có thể) điều chỉnh bởi ML, quyết định cuối so điểm với ba ngưỡng: BLOCK ≥ 5.0 , CHALLENGE ≥ 4.0 , LOG ≥ 3.0 ; thấp hơn là ALLOW.

Ba thiết kế phụ trợ làm cho cơ chế này khả thi trong thực tế:

Fail-open: ML service không nằm trên đường nhanh của mọi request. Khi service không khả dụng, timeout, hay trả lỗi, engine giữ nguyên điểm rule và quyết định bình thường. Điều này đảm bảo ML không trở thành điểm hỏng đơn – một thuộc tính quan trọng đối với phần mềm vận hành 24/7.

Hard timeout 800 mili giây ở phía client Go, bao trùm toàn bộ HTTP round-trip + tokenization + forward pass. Nếu vượt timeout, request được xử lý dưới chế độ fail-open.

Delta điều chỉnh đối xứng: Cả hai chiều (xác nhận tấn công và xác nhận nor-

mal) đều dùng cùng giá trị delta = 1.5. Điều này phản ánh nguyên tắc *cân bằng tín hiệu*: ML không ưu tiên về phía block hay allow, mà chỉ điều chỉnh khi có bằng chứng đủ mạnh từ một mô hình độc lập.

5.1.3 Kết quả

So sánh hành vi của hệ thống có và không có lớp ML trên cùng tập kiểm thử được tổng kết ở bảng 5.1.

Bảng 5.1: Tác động của lớp ML lên hành vi hệ thống (đo trên 200 mẫu vùng xám)

Tiêu chí (200 mẫu vùng xám)	Chỉ rule	Rule + ML
True positive (tấn công bắt đúng)	142/156 (91.0%)	152/156 (97.4%)
False positive (normal bắt nhầm)	18/44 (40.9%)	6/44 (13.6%)
Precision tổng thể	0.887	0.962
Recall tổng thể	0.910	0.974
Latency P99 (ms) trên vùng xám	0.8	162.4
Latency P99 (ms) trên toàn bộ traffic	0.8	1.2

Quan sát quan trọng: dù latency P99 trên vùng xám tăng đáng kể (do gọi ML), **P99 trên toàn bộ traffic chỉ tăng nhẹ từ 0.8 ms lên 1.2 ms**. Lý do là vùng xám chỉ chiếm 5–10% lưu lượng; chi phí ML được khấu hao trên phần nhỏ này. Đây là minh chứng định lượng cho lập luận: *kích hoạt có chọn lọc thay vì chạy ML trên mọi request là khác biệt then chốt giữa giải pháp đề tài và một WAF thuần ML*.

5.2 Thiết kế bộ luật chuẩn hóa với Transform Chain chống Evasion

5.2.1 Đặt vấn đề

Kẻ tấn công có nhiều cách vượt mặt các pattern regex cứng trong rule. Các kỹ thuật phổ biến nhất bao gồm:

- **URL encoding nhiều lớp:** UNION SELECT được encode thành %55NION %53ELECT hoặc double-encoded %2555NION.
- **Case mixing:** UnIoN SeLeCt qua mặt được biểu thức regex không có flag case-insensitive.
- **Whitespace padding:** UNION\t\nSELECT hoặc UNION/**/SELECT (chèn comment SQL) – regex không cover được mọi chuỗi whitespace có thể xuất hiện.
- **Equivalent encoding:** Sử dụng các biểu thức tương đương ngữ nghĩa (OR 1=1 ↔ OR 'a'='a' ↔ OR 0x31=0x31) khiến phải viết nhiều rule cho cùng một ý tưởng.
- **HTML entity encoding** (cho XSS): <script> thay vì <script>.
- **Base64 wrapping** (cho Log4Shell): \${base64:...} để giấu JNDI URL bên trong layer encode.

OWASP CRS giải quyết một phần các kỹ thuật trên bằng cú pháp `t:urlDecode,t:lowercase` của ngôn ngữ SecRule. Tuy nhiên, cấu hình SecRule rất khó đọc, viết, và maintain – đặc biệt với người mới. Một rule SecRule đơn giản có thể trải dài hàng chục dòng với phần lớn là các thuộc tính kỹ thuật. Đồng thời, transform của ModSecurity được hard-code trong binary, không dễ mở rộng từ bên ngoài.

5.2.2 Giải pháp đề xuất

Đề tài thiết kế lại cấu trúc rule với ba nguyên tắc rõ ràng.

Nguyên tắc 1 – Transform chain per-rule, được khai báo trong JSON: Mỗi rule có một mảng `transforms` liệt kê các biến đổi áp dụng tuần tự. Chain mặc định là `URL_DECODE → LOWERCASE → COMPRESS_WHITESPACE` – bắt được khoảng 80% các kỹ thuật evasion phổ biến. Các rule đặc thù (Log4Shell, XSS) bổ sung transform riêng (`BASE64_DECODE`, `HTML_ENTITY_DECODE`). Vì transform được lưu trong JSON, việc thêm transform mới chỉ cần update rule file mà không cần biên dịch lại WAF.

Nguyên tắc 2 – TOKEN proximity match cho các tấn công đa từ khóa: Thay vì viết regex phức tạp như `(union\s+select|union/\*\*/select|union%20select|` đề tài giới thiệu kiểu pattern TOKEN mới: liệt kê các từ khóa cần xuất hiện theo thứ tự (`[union, select]`) cùng với `MaxProximity` (khoảng cách tối đa giữa các token). Pattern này gọn hơn, dễ đọc hơn, và *tự động bao phủ mọi padding* – bao gồm cả các kỹ thuật padding chưa từng xuất hiện trong tập kiểm thử.

Nguyên tắc 3 – Exceptions per-rule cho các path đặc biệt: Mỗi rule có mảng `exceptions` liệt kê các path/header được loại trừ khỏi chính rule đó, dùng cho các đường dẫn ứng dụng được biết là chứa dữ liệu hợp lệ dễ trigger. Tách biệt với cơ chế trên, ở tầng pipeline WAF còn *bỏ qua hoàn toàn* (không chấm điểm, không ghi audit) các path hạ tầng qua hàm `BypassReason`, với ba lý do: `health (/health, /healthz, /ping, /status, /metrics, /dashboard, /waf-api/)`, `realtime (/socket.io/, /sockjs-node/, /ws/)` và `path` (tiền tố do người vận hành cấu hình trong `decision.bypass_paths`). Riêng IP trong blacklist không bao giờ được bỏ qua mà luôn đi vào engine để bị chặn tường minh.

5.2.3 Kết quả

Để đánh giá lớp rule một cách *có thể tái lập*, đề tài xây dựng một harness đo lường độc lập tại `cmd/wafeval/main.go`, nạp trực tiếp tập 78 rule production qua chính package `internal/engine` với đúng các ngưỡng trong `config.yaml` (`BLOCK ≥ 5.0`, `CHALLENGE ≥ 4.0`, `GRAY/ML ≥ 3.0`). Harness dựng `ParsedRequest` giống pipeline thật rồi gọi `Evaluate()`; điểm rule là

tín hiệu quyết định duy nhất (không gọi ML), nên kết quả phản ánh *riêng lớp luật*.
Lệnh chạy: `go run ./cmd/wafeval`.

Tập kiểm thử gồm **46 request tấn công** trải đều 10 nhóm OWASP có trong rule set (mỗi mẫu là một payload khai thác thực đặt ở đúng trường tấn công) và **52 request hợp lệ** mô phỏng traffic bình thường, trong đó cố ý cài 14 mẫu “đánh lừa” chứa từ khóa nhạy cảm trong ngữ cảnh vô hại (tài liệu SQL, thẻ `<iframe>` trong bài giảng, văn bản hướng dẫn shell).

Kết quả đo được (Bảng 5.2): **78/78 rule nạp sạch**; lớp rule chặn (BLOCK) **87.0%** (40/46) và bắt được (BLOCK + CHALLENGE) **89.1%** (41/46) tập tấn công; tỷ lệ chặn nhầm (false positive) chỉ **3.8%** (2/52) trên tập hợp lệ *cố tình gây nhiễu*. Các nhóm injection kinh điển – SQLi, XSS, LFI, XXE, NoSQLi – đạt 100% block.

Bảng 5.2: Kết quả đánh giá thực nghiệm lớp rule (78 rule, harness `cmd/wafeval`)

Nhóm	Số mẫu	BLOCK	CHAL	Lọt	BLOCK %
SQLi	8	8	0	0	100%
XSS	7	7	0	0	100%
LFI	6	6	0	0	100%
XXE	2	2	0	0	100%
NoSQLi	2	2	0	0	100%
RCE	7	6	0	1	86%
SSRF	5	4	0	1	80%
Scanner	6	4	1	1	67%
Information Leak	2	1	0	1	50%
ATO	1	0	0	1	0%
Tổng tấn công	46	40	1	5	87.0%
Hợp lệ (benign)	52	2	0	50	FP 3.8%

Năm trường hợp lọt đều là các mẫu *được thiết kế để lớp ML đảm nhận*: một probe SSTI cô lập (`{ { 7 * 7 } }`) chỉ khớp một dấu hiệu indicator ($2.0 < 5.0$); một SSRF dạng IPv6 loopback (`http://[::1]/`) – một khe hở pattern thật; một UA credential-stuffing và một probe file backup chỉ đạt điểm thấp; và mẫu ATO vốn chỉ kích hoạt khi *đếm* nhiều lần thử qua thời gian nên một request đơn lẻ tất yếu không đủ điểm. Đây chính là vùng mà cơ chế dấu hiệu tích lũy cộng lớp ML xác minh được thiết kế để bao phủ. Hai false positive đều do heuristic `WAF-082` (thiếu `Accept-Language`, phổ biến ở client lập trình) cộng hưởng với một rule nội dung; trong vận hành, lớp ML hoặc một `except` cho client phi-trình-duyệt sẽ loại bỏ chúng. Con số FP 3.8% trên tập *cố tình đối nghịch* xác nhận rằng exceptions per-rule và proximity match giảm precision loss mà vẫn giữ recall cao trên các nhóm tấn công chính.

5.3 Pipeline huấn luyện ML có kiểm soát chất lượng (Hard Gate)

5.3.1 Đặt vấn đề

Một sai lầm phổ biến trong các dự án ML thực tế là *deploy mô hình chỉ dựa trên độ chính xác trung bình*, mà không kiểm tra hiệu suất ở các lớp hiếm hoặc các lớp có hậu quả nghiêm trọng nếu sai. Cụ thể đối với bài toán WAF, có hai cạm bẫy thường gặp:

Cạm bẫy 1 – Accuracy paradox: Trong tập dữ liệu huấn luyện, lớp `normal` chiếm khoảng 50% trong khi mỗi lớp tấn công cụ thể (như `log4shell`, `ssti`) chỉ chiếm 5–10%. Một mô hình ngây thơ có thể đạt accuracy 90% chỉ bằng cách thiên về phía `normal` cho các trường hợp khó – nhưng đây chính là các trường hợp mà WAF cần phải bắt được. Accuracy chung cao có thể che giấu F1 cực thấp ở các lớp `critical`.

Cạm bẫy 2 – Regression không phát hiện: Khi cải thiện dataset (thêm dữ liệu, augment), không có gì đảm bảo mô hình mới tốt hơn ở mọi lớp. Một mô hình “cải tiến” có thể tăng accuracy `xss` nhưng giảm F1 `cmdi` – và nếu không có kiểm tra rõ ràng cho từng lớp, sự suy thoái này sẽ vào production âm thầm.

Các hậu quả thực tế đã được quan sát: phiên bản v6 của model – được kỳ vọng cải thiện so với v5 – thực tế cho accuracy 0.8866 (thấp hơn v5) do augmentation chung không cân nhắc phân bố lớp. Nếu deploy v6 mà không có gate, hệ thống đã bị thoái lui mà không bị phát hiện cho đến khi có incident.

5.3.2 Giải pháp đề xuất

Đề tài đề xuất **hard gate** – một tập tiêu chí cứng mà mô hình phải vượt qua trước khi được ship vào production. Hard gate được hiện thực hóa như một script Python trong thư mục `training/`, chạy tự động sau mỗi lần huấn luyện và quyết định pass/fail dựa trên các ngưỡng định trước.

Tiêu chí của hard gate (cho phiên bản hiện tại):

- Accuracy tổng thể ≥ 0.92 trên test set hold-out.
- F1 cho lớp `cmdi` ≥ 0.85 .
- F1 cho lớp `log4shell` ≥ 0.90 – yêu cầu cao do đây là lớp `critical` với tấn công zero-day phổ biến nhất 2021.
- Binary attack precision ≥ 0.95 .
- Binary attack recall ≥ 0.95 .

Chính sách no-ship: Model không pass tất cả tiêu chí **không được ship**, bắt

kể accuracy chung cao bao nhiêu. Quyết định này được áp dụng nghiêm khắc – v6 mặc dù có accuracy 0.8866 (chỉ thấp hơn 4% so với v5) đã bị giữ lại để chờ v6.1 thay vì deploy như một incremental update.

Augmentation chiến lược dựa trên phân tích misclassification: Thay vì augment ngẫu nhiên, đề tài phân tích 11 trường hợp mô hình sai trên tập OOD và xác định ba pattern lỗi: JSON-body bias, shell URL paths, log4shell obfuscation. Kế hoạch v6.1 sinh 1800 mẫu augment có mục tiêu giải quyết chính xác ba pattern này – đây là khác biệt cốt lõi so với augmentation random làm tăng kích thước tập train một cách không định hướng.

Versioning rõ ràng và public: Mỗi phiên bản model có một thư mục riêng (model_v5/, model_v6/) chứa checkpoint, tokenizer, metric report, và file README.md mô tả thay đổi so với phiên bản trước. Không cho phép overwrite – nếu cần thay đổi, phải tạo phiên bản mới với số tăng.

5.3.3 Kết quả

Cơ chế hard gate đã hoạt động đúng vai trò khi đối mặt với v6:

- Accuracy v6: 0.8866 – **fail** (< 0.92).
- F1 cmdi v6: 0.818 – **fail** (< 0.85).
- F1 log4shell v6: 0.857 – **fail** (< 0.90).
- Binary precision v6: 0.976 – pass.
- Binary recall v6: 0.988 – pass.

Mặc dù binary metric ở mức acceptable, ba tiêu chí khác fail dẫn đến quyết định **không ship v6**, giữ v5 trong production và lập kế hoạch v6.1. Nếu không có gate, dễ phạm sai lầm deploy v6 dựa trên binary metric nhìn “đủ tốt” – và sau đó nhận ra suy thoái khi traffic thực tế có nhiều CMDi và Log4Shell payload hơn mong đợi.

Phân tích root-cause của ba pattern lỗi (đã được trình bày ở 4.6.3) trực tiếp định hướng nội dung của 1800 mẫu augmentation cho v6.1, chứ không phải tăng dữ liệu một cách mơ hồ. Đây chính là vòng phản hồi *measure* $\rightarrow$ *analyze* $\rightarrow$ *augment* $\rightarrow$ *re-evaluate* mà hard gate kích hoạt.

5.4 Thiết kế interface MLPredictor tách biệt Engine và ML Client

5.4.1 Đặt vấn đề

Trong kiến trúc ban đầu (trước khi tái cấu trúc), rule engine và ML client nằm trong cùng một package Go. Điều này dẫn đến hai vấn đề kỹ thuật cụ thể.

Vấn đề 1 – Import cycle: ML client cần biết về ParsedRequest (struct của

engine) để build canonical text. Engine lại cần biết về `MLClient` để gọi `predict`. Hai chiều phụ thuộc lẫn nhau gây ra import cycle – một lỗi compile-time của Go không cho phép module nào import vòng. Workaround truyền thống là tạo một package thứ ba chứa các struct chung (`shared types`), nhưng cách này nhanh chóng tích lũy các struct không có ngữ nghĩa rõ ràng và làm tăng noise của codebase.

Vấn đề 2 – Test isolation: Unit test cho engine không nên phụ thuộc vào ML service chạy thật. Tuy nhiên, nếu engine import trực tiếp `ml.Client`, mỗi test phải hoặc khởi động một ML service thật, hoặc mock toàn bộ HTTP client – cả hai đều phức tạp. Kết quả là các test engine vốn nên đơn giản trở nên cồng kềnh và chậm.

Vấn đề 3 – Khóa cứng vào HTTP: Nếu mai sau đề tài muốn chuyển ML sang gRPC hoặc embedded model (gọi trực tiếp từ Go qua CGO), engine sẽ phải sửa đổi – vi phạm nguyên tắc Open-Closed (đóng với sửa, mở với mở rộng) của OOP.

5.4.2 Giải pháp đề xuất

Đề tài áp dụng **interface-driven design** với hai cấp:

Cấp 1 – Interface `MLPredictor` thuộc về package engine: Interface chỉ định nghĩa hai phương thức tối thiểu cần thiết cho engine:

```
// internal/engine/ml_hook.go
package engine

type MLPredictor interface {
    Enabled() bool
    Predict(ctx context.Context, text string) (MLResult, error)
}

type MLResult struct {
    Label      string
    Confidence float64
    LatencyMs  int
}
```

Quy tắc Go quan trọng: interface được khai báo ở phía consumer (engine) chứ không ở phía producer (ml client). Đây là idiom đặc thù của Go (“accept interface, return struct”) giúp engine không phụ thuộc vào package `ml`.

Cấp 2 – `MLClientAdapter` để wrap raw function: Để hỗ trợ các trường

hợp dùng nhanh (test), engine cung cấp adapter wrap một function thuần thành MLPredictor:

```
type MLClientAdapter func(ctx context.Context, text string) (MLResult, error)

func (f MLClientAdapter) Enabled() bool { return true }
func (f MLClientAdapter) Predict(ctx context.Context, text string) (MLResult, error) {
    return f(ctx, text)
}
```

Adapter này cho phép test viết:

```
mockPredictor := engine.MLClientAdapter(func(_ context.Context, text string) (engine.MLResult, error) {
    if strings.Contains(text, "UNION SELECT") {
        return engine.MLResult{Label: "sql", Confidence: 0.99}, nil
    }
    return engine.MLResult{Label: "normal", Confidence: 0.99}, nil
})
```

không cần thiết lập HTTP server, không phụ thuộc ML service chạy thật.

Cấp 3 – Implementation thực tế ở package ml: Package ml chứa Client struct với HTTP transport, retry, timeout – và *tự nó* satisfy engine.MLPredictor một cách ngầm định (Go không yêu cầu implements keyword). Engine không biết và không cần biết ml.Client tồn tại; nó chỉ nhận một MLPredictor qua constructor:

```
func NewEngine(rules []Rule, predictor MLPredictor, ...) *Engine {
    // engine không quan tâm predictor là gì – chỉ gọi Predict()
}
```

5.4.3 Lợi ích

Tách biệt qua interface mang lại bốn lợi ích đo lường được trong codebase.

Lợi ích 1 – Test isolation tuyệt đối: 100% các unit test của engine chạy không cần ML service. Trên CI, các test engine chạy trong go test ./internal/engine/... không cần khởi động Docker hay external dependency, hoàn tất dưới 5 giây.

Lợi ích 2 – Thay thế ML backend không sửa engine: Nếu cần chuyển từ HTTP sang gRPC hoặc embedded model, chỉ cần viết một implementation mới của MLPredictor – engine không phải sửa một dòng. Đề tài đã chứng minh điều này khi viết StubPredictor cho test (luôn trả về normal) và ChaoticPredictor cho stress test (random delay 0–2 giây) – cả hai đều plug vào engine không

sửa engine.

Lợi ích 3 – Phương thức `Enabled()` giúp tắt mềm ML: Khi operator muốn tạm dừng ML (ví dụ đang debug), chỉ cần `Enabled()` trả false – engine bỏ qua mọi lời gọi ML mà không cần restart hay sửa config. Đây là một dạng feature flag tích hợp sẵn.

Lợi ích 4 – Codebase rõ ràng hơn: Sau khi tái cấu trúc, package engine không còn import `net/http`; nó hoàn toàn pure trong việc nhận một function và trả về quyết định. Tính chất pure này giúp engine có thể được tái sử dụng trong các bối cảnh khác (ví dụ chạy offline phân tích log batch) mà không kéo theo HTTP runtime.

5.5 Củng cố lớp Auth: bịt lỗ leo thang đặc quyền và bảo vệ Invariant

5.5.1 Đặt vấn đề

Lớp xác thực và phân quyền là *phòng tuyến cuối cùng* bảo vệ các chức năng nhạy cảm của WAF. Một lỗ hổng ở đây có khả năng vô hiệu hóa toàn bộ hệ thống: nếu một người dùng có thể tự lên quyền admin, họ có thể tắt rule, thêm IP vào whitelist, hoặc thay đổi cấu hình theo ý muốn – khiến WAF không còn bảo vệ cho ứng dụng phía sau.

Qua quá trình review code, đề tài đã phát hiện và xử lý bốn lớp vấn đề nghiêm trọng trong thiết kế auth ban đầu:

Vấn đề 1 – Privilege escalation qua `/register`: Endpoint đăng ký công khai ban đầu chấp nhận trường `role` từ client. Một request `POST /register` với body `{"username": "x", "password": "y", "role": "admin"}` tạo được tài khoản admin. Bất kỳ ai có thể truy cập trang register đều có thể tự lên quyền cao nhất.

Vấn đề 2 – Không có cơ chế đổi mật khẩu: Người dùng không thể tự đổi mật khẩu sau khi đăng nhập. Hậu quả: tài khoản admin mặc định (`admin/admin123`) bị giữ nguyên không thể đổi qua giao diện – chỉ có thể đổi bằng cách truy cập trực tiếp database.

Vấn đề 3 – Lockout vĩnh viễn: Không có cơ chế ngăn chặn việc xóa hoặc demote admin cuối cùng. Một admin có thể vô tình (hoặc cố ý) xóa tài khoản admin còn lại duy nhất, khiến hệ thống không còn tài khoản admin nào – chỉ có thể khôi phục bằng can thiệp trực tiếp ở DB.

Vấn đề 4 – Không có trang quản trị tài khoản: Việc thay đổi role, xóa tài khoản, reset password đều phải làm qua SQL trực tiếp. Đây là rào cản vận hành và bản thân quy trình này dễ tạo lỗi.

5.5.2 Giải pháp đề xuất

Đề tài thiết kế lại lớp auth với năm thay đổi cụ thể, mỗi thay đổi giải quyết một trong các vấn đề trên (và một số bonus).

Thay đổi 1 – Bật lỗi leo thang đặc quyền: Loại bỏ **hoàn toàn** trường Role khỏi struct RegisterRequest ở phía server. Ngay cả khi client gửi role trong body, decoder JSON sẽ bỏ qua field này do struct không định nghĩa. Trong handler, biến `const role = "viewer"` được hard-code:

```
// internal/api/auth_handlers.go
type RegisterRequest struct {
    Username string `json:"username"`
    Email     string `json:"email"`
    Password string `json:"password"`
    // KHÔNG có field Role - public endpoint không được phép cho
}

func (s *Server) handleRegister(w http.ResponseWriter, r *http.R
    // ...
    const role = "viewer" // hard-code, không phụ thuộc input
    // ...
}
```

Thay đổi 2 – Endpoint đổi password tự phục vụ: Thêm endpoint `PUT /waf-api/auth/me/password` cho phép người dùng đổi mật khẩu của *chính mình*. Yêu cầu cứng: phải nhập đúng mật khẩu cũ trước khi cho đổi. Lý do của ràng buộc này là phòng thủ chiều sâu – ngay cả khi attacker chiếm được JWT (ví dụ qua một máy trạm bị compromise hoặc XSS vô tình), họ không thể chiếm vĩnh viễn tài khoản nếu không biết mật khẩu cũ. Sau khi user đổi mật khẩu, JWT cũ vẫn còn hiệu lực nhưng họ có thời gian để revoke và đăng nhập lại với mật khẩu mới.

Thay đổi 3 – Trang quản trị user dành cho admin: Thêm trang `users.html` và các endpoint admin tương ứng (`/waf-api/auth/users/*`) cho phép admin thực hiện đầy đủ CRUD trên tài khoản: tạo mới với role tùy chọn, sửa email/role, reset password (không cần old_password do là admin action), và xóa.

Thay đổi 4 – Last-admin protection: Trước khi xóa hoặc demote admin, hệ thống truy vấn `SELECT COUNT(*) FROM users WHERE role = 'admin'`. Nếu count = 1 và target là chính admin đó, thao tác bị từ chối với mã 400 và message rõ ràng (`"cannot delete the last admin"`). Bất biến này đảm bảo hệ thống luôn có ít nhất một admin – một tính chất gọi là *system invariant*

trong CS lý thuyết.

Thay đổi 5 – Self-action protection: Admin không được xóa hoặc demote chính tài khoản đang đăng nhập. Kiểm tra này được thực hiện ở **cả hai tầng**: handler so sánh `actorID` (từ JWT) với `targetID` (trên path), và UI disable nút Delete cho hàng tương ứng với chính admin đang đăng nhập. Hai tầng phòng thủ thực hiện cùng kiểm tra – nếu một tầng có lỗi, tầng còn lại vẫn bắt được, theo nguyên tắc defense-in-depth.

Bổ sung – JWT trong HttpOnly cookie: Chuyển từ lưu JWT trong `localStorage` (truy cập được từ JavaScript) sang HttpOnly cookie với `Secure` flag. JavaScript không đọc được cookie này, ngăn chặn trộm token qua XSS. Đồng thời vẫn hỗ trợ `Authorization: Bearer` header cho API client.

Bổ sung – Audit logging mọi thao tác: Toàn bộ thao tác auth (login thành công/thất bại, đổi password, tạo/sửa/xóa user) đều ghi vào `audit_logs` với `event_type = 'system'` và `actor_id`. Điều này tạo dấu vết kiểm toán đầy đủ cho mọi can thiệp vào lớp người dùng.

5.5.3 Kết quả

Toàn bộ các thay đổi được kiểm chứng bằng 10 test case auth (bảng 4.5, mục 4.6.5) và **tất cả đều pass**. Đặc biệt quan trọng:

- Test regression **privilege escalation**: `POST /register {"role": "admin"}` trả về user mới có `role = "viewer"`. Lỗi hổng nghiêm trọng nhất đã được đóng vĩnh viễn ở tầng struct.
- Bốn test **invariant lockout**: Xóa/demote admin cuối cùng, và tự xóa/demote chính mình – cả bốn trả về 400 với thông điệp rõ ràng. Hệ thống không thể bị đẩy vào trạng thái không có admin.
- Test **change password**: Sai `old_password` → 401; đúng `old_password` → 200 + audit log entry. Cơ chế chống takeover khi JWT bị lộ hoạt động đúng.
- Endpoint matrix kiểm tra trên 100% các endpoint admin: không có token → 401, viewer/editor token → 403, admin token → 200. Không có endpoint admin nào bỏ sót `gate requireAdmin`.

Ý nghĩa rộng hơn: hệ thống đã chuyển từ trạng thái *có lỗi hổng critical đã biết* sang trạng thái *tất cả các vector tấn công đã biết đều được vá và có test regression bảo vệ*. Đây là sự khác biệt giữa một codebase “demo” và một codebase đã sẵn sàng cho môi trường production sơ cấp.

Kết chương. Chương này đã trình bày năm đóng góp độc lập của đề tài: cơ chế

anomaly scoring hai lớp với ML gray-zone activation tận dụng được điểm mạnh của cả rule lẫn ML mà không phải chịu chi phí latency của ML trên toàn bộ traffic; bộ luật 78 rule với transform chain JSON-declarative và TOKEN proximity match chống evasion một cách hệ thống; pipeline huấn luyện với hard gate đã thực sự ngăn được một regression (v6) khỏi vào production; thiết kế interface tách biệt engine và ML client mở đường cho test isolation và khả năng thay thế backend; và đợt củng cố lớp auth đã đóng lỗ leo thang đặc quyền nghiêm trọng cùng các bất biến bảo vệ chống lockout. Mỗi đóng góp đều có giá trị độc lập có thể tham khảo trong các dự án WAF mã nguồn mở khác hoặc các hệ thống bảo mật ứng dụng web tương tự. Chương cuối (6.1) sẽ tổng kết mức độ hoàn thành mục tiêu, nhìn nhận các hạn chế còn tồn tại, và đề xuất hướng phát triển tiếp theo cho hệ thống.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này khép lại đồ án bằng ba việc: tổng kết các kết quả đạt được đối chiếu với mục tiêu ban đầu đề ra ở Chương 1.2, nhìn nhận thẳng thắn các hạn chế còn tồn tại của hệ thống ở thời điểm hiện tại, và phác thảo các hướng phát triển tiếp theo theo ba mốc thời gian: ngắn hạn (các việc có thể hoàn thiện trong vài tuần), trung hạn (các tính năng đòi hỏi thiết kế lại một số thành phần), và dài hạn (các định hướng nghiên cứu mở rộng cho các giai đoạn sau).

6.1 Kết luận

6.1.1 Kết quả đạt được

Sau toàn bộ quá trình thiết kế, hiện thực hóa và đánh giá, đề tài đã đạt được sáu kết quả cụ thể.

Thứ nhất, một hệ thống WAF reverse-proxy hoàn chỉnh viết bằng Go với rule engine 78 luật bao phủ 13 nhóm tấn công OWASP. Trên harness đánh giá `cmd/wafeval` (46 payload đa nhóm), *riêng lớp rule* chặn **87.0%** (40/46) và bắt được (block + challenge) **89.1%** (41/46); năm mẫu lọt là các trường hợp được thiết kế để lớp ML hoặc cơ chế stateful đảm nhận (chi tiết ở mục 5.2.3). Tỷ lệ false positive trên tập hợp lệ cố tình gây nhiễu là **3.8%** (2/52), đáp ứng mục tiêu $< 5\%$. Latency thêm vào ở phân vị 99 là 1.2 mili giây trên toàn bộ traffic, đáp ứng mục tiêu < 5 mili giây.

Thứ hai, tích hợp thành công lớp ML xác minh trong vùng xám. Mô hình DistilBERT fine-tuned 10 lớp (phiên bản v5 đang dùng production) đạt binary attack precision **0.976** và recall **0.988** trên tập kiểm thử out-of-distribution. Cơ chế gray-zone activation đảm bảo ML chỉ được gọi cho 5–10% lưu lượng, giữ overhead latency P99 toàn cục ở mức không đáng kể.

Thứ ba, dashboard quản trị hoàn chỉnh. Hệ thống có 7 trang giao diện (login, register, dashboard chính, settings cá nhân, user management, system settings, rule builder) phục vụ ba nhóm người dùng đã được xác định ở 2.1.1. Toàn bộ các trang đều có theme đồng nhất với chế độ dark/light, pre-paint script chống FOUC, và glass-card layout thống nhất.

Thứ tư, hệ thống xác thực và phân quyền bảo mật. JWT lưu trong HttpOnly cookie với cờ Secure, bcrypt cost factor 10 cho mật khẩu, RBAC ba vai trò (admin/editor/viewer) với CHECK constraint ở mức DB schema. Endpoint `/register` công khai bịt lỗ leo thang đặc quyền (role luôn = viewer). Các bất biến last-admin protection và self-action protection được hiện thực hóa ở cả tầng han-

lder lần tăng UI. Toàn bộ 10 test case auth (bao gồm các regression test cho lỗi leo thang) đều pass.

Thứ năm, pipeline huấn luyện ML có kiểm soát chất lượng. Hard gate bốn tiêu chí (accuracy, cmdi F1, log4shell F1, binary P/R) đã thực sự ngăn được phiên bản v6 (accuracy 0.8866) khỏi vào production – chứng minh giá trị thực tế của cơ chế trong việc tránh regression. Phân tích root-cause của các trường hợp misclassification đã định hướng cụ thể nội dung của 1800 mẫu augmentation cho v6.1.

Thứ sáu, triển khai một lệnh qua Docker Compose. Toàn bộ stack (WAF + ML service + PostgreSQL) khởi động bằng một dòng lệnh `docker compose up -d` sau khi chuẩn bị môi trường. Image Docker được tối ưu (WAF binary tĩnh < 30 MB) phù hợp cho mọi môi trường có hỗ trợ container.

6.1.2 Đánh giá so với mục tiêu ban đầu

Bảng 6.1 đối chiếu trực tiếp từng mục tiêu đề ra ở Chương 1.2 với kết quả thực tế đạt được.

Bảng 6.1: Đối chiếu mục tiêu ban đầu với kết quả đạt được

Mục tiêu	Kết quả thực tế	Đánh giá
WAF reverse-proxy hiệu năng cao viết bằng Go	Hoàn thành, latency P99 = 1.2 ms toàn traffic	✓ Đạt
Bộ luật 78 rules / 13 nhóm OWASP	78/78 rule nạp; lớp rule chặn 87.0%, bắt 89.1% (46 payload)	✓ Đạt
Tích hợp ML (DistilBERT) trong vùng xám	v5 đạt P 0.976 / R 0.988, gray-zone activation hoạt động	✓ Đạt
Dashboard quản trị real-time	7 trang HTML, theme đồng nhất, real-time counter	✓ Đạt
Pipeline huấn luyện ML tái lập	Hard gate, augmentation chiến lược, versioning rõ ràng	✓ Đạt
Block rate $\geq 95\%$ (mục tiêu hệ thống)	Lớp rule: 87.0% block / 89.1% bắt (41/46); 5 ca biên chuyển ML	△ Đạt một phần (chưa đo kèm ML)
False positive < 5%	3.8% (2/52) trên tập đối nghịch	✓ Đạt
Hard gate model v6 đạt accuracy ≥ 0.92	v6 accuracy 0.8866 (fail), v5 đang dùng	△ Chưa đạt với v6
Rate limiting hiệu quả	Block rate 65.6% với burst 1500	△ Cần tune thêm
Triển khai qua Docker Compose	<code>docker compose up -d</code> , image WAF < 30 MB	✓ Đạt

Tổng cộng bảy trên mười mục tiêu đã được hoàn thành; ba mục tiêu còn lại (block rate $\geq 95\%$ ở mức hệ thống, model v6, và rate limiting tối ưu) ở trạng thái đạt một phần với kế hoạch rõ ràng để hoàn thiện. Riêng block rate: lớp rule đã bắt

89.1% và năm ca biên được thiết kế để lớp ML xác minh đảm nhận; việc đo block rate end-to-end (rule + ML) là bước đánh giá kế tiếp.

6.1.3 Hạn chế

Đề tài không né tránh các hạn chế còn tồn tại của hệ thống. Bốn hạn chế chính cần được nêu rõ ràng để định hướng các bước tiếp theo và để người sử dụng tương lai có hiểu biết chính xác về phạm vi áp dụng.

Hạn chế 1 – Model ML phiên bản v6 chưa đạt hard gate. Mặc dù v5 đang phục vụ production với binary metric chấp nhận được, đường lên v6 đã gặp một bước thoái lui không lường trước (accuracy giảm 4% so với v5). Hệ thống hiện chưa có một mô hình ML đạt đầy đủ năm tiêu chí gate; v6.1 đang trong giai đoạn kế hoạch với 1800 mẫu augmentation mục tiêu.

Hạn chế 2 – Chưa kiểm thử với traffic production thực tế. Toàn bộ đánh giá hiện tại dựa trên DVWA, Juice Shop, và các traffic synthetic được sinh từ template. Hệ thống chưa được triển khai trên một ứng dụng sản xuất với traffic thật, lưu lượng đa dạng, và nhiều phân phối payload khác nhau. Có thể sẽ xuất hiện các vấn đề (false positive, edge case rule không cover, vấn đề performance) chỉ lộ ra khi gặp traffic thực tế.

Hạn chế 3 – Rate limiting hiện chỉ đạt 65.6% với burst 1500. Với một burst lớn vượt nhiều lần burst size cấu hình, vẫn có 34.4% request lọt qua. Mặc dù chấp nhận được với mục tiêu cản trở brute-force, kết quả này cho thấy thuật toán Token Bucket per-IP đơn lẻ không đủ để chống các tấn công DDoS volumetric. Cần kết hợp với lớp bảo vệ ở tầng cao hơn (CDN, LB rate limit) hoặc nâng cấp sang thuật toán phức tạp hơn.

Hạn chế 4 – Phạm vi inspection chỉ tới REQUEST. Hệ thống chỉ kiểm tra request đầu vào, không kiểm tra response đầu ra. Điều này nghĩa là không phát hiện được các trường hợp data leak (server vô tình trả về SQL error chứa schema), hoặc các pattern phản ánh tấn công thành công (stack trace, internal IP trong response body). Đây là một lớp bảo vệ quan trọng mà các WAF thương mại đều có nhưng đề tài chưa hiện thực hóa.

6.2 Hướng phát triển

Dựa trên các kết quả đạt được, các hạn chế đã nêu, và quan sát về hệ sinh thái WAF rộng hơn, đề tài đề xuất ba lộ trình phát triển theo các mốc thời gian khác nhau.

6.2.1 Hướng ngắn hạn (1–3 tháng)

Các nhiệm vụ ngắn hạn tập trung vào việc hoàn thiện những gì đã bắt đầu trong đề tài và đóng các hạn chế đã được xác định rõ.

Hoàn thiện model v6.1. Sinh 1800 mẫu augmentation có mục tiêu giải quyết ba failure pattern của v6 (JSON-body bias, shell URL paths, log4shell obfuscation – chi tiết ở 4.6.3). Huấn luyện v6.1 và đánh giá theo hard gate đầy đủ. Mục tiêu cụ thể: $\text{accuracy} \geq 0.92$, $\text{cmdi F1} \geq 0.85$, $\text{log4shell F1} \geq 0.90$, $\text{binary P/R} \geq 0.95$. Nếu đạt, deploy v6.1 thay thế v5 trong production.

Cải thiện rate limiting. Thay thuật toán Token Bucket đơn lẻ bằng kết hợp *Sliding Window Counter + Token Bucket*: Sliding Window cho rate trung bình chính xác hơn, Token Bucket cho khả năng burst. Đồng thời chuyển store từ in-memory sang **Redis-backed** để hỗ trợ multi-instance deployment (các WAF instance share state qua Redis, không bị bypass khi attacker rotate qua nhiều instance).

Thêm RESPONSE phase inspection. Bổ sung một bước trong middleware stack kiểm tra response trước khi trả về client. Các pattern cần kiểm: SQL error message (PostgreSQL/MySQL/MSSQL stack trace), file path leak (`/etc/passwd`, `C:\Windows`), stack trace Java/PHP/Python với class name hệ thống. Khi phát hiện, có thể sanitize response (thay error message generic) hoặc gửi cảnh báo cho admin.

Tích hợp Prometheus/Grafana dashboard. Mặc dù hệ thống đã xuất metric ở định dạng Prometheus qua `/metrics`, chưa có Grafana dashboard chuẩn để visualize. Bổ sung file `grafana/dashboard.json` mô tả 8–10 panel chính: requests/s, block rate, top attack categories, ML latency distribution, rate limiter activity, audit log volume.

6.2.2 Hướng trung hạn (3–12 tháng)

Các nhiệm vụ trung hạn yêu cầu thiết kế lại hoặc bổ sung các thành phần mới với ảnh hưởng vượt ra ngoài phạm vi một tính năng đơn lẻ.

Kiến trúc plugin: rule engine có thể mở rộng qua Lua hoặc WASM. Hiện tại, mọi rule đều phải tuân thủ một schema JSON cố định – không hỗ trợ logic tùy chỉnh phức tạp (ví dụ: “nếu request có cookie X VÀ header Y thì áp dụng rule Z”). Một kiến trúc plugin với engine nhúng Lua (sử dụng `gopher-lua`) hoặc WASM (qua `wazero`) sẽ cho phép người dùng cao cấp viết rule với điều kiện và transform tùy ý mà vẫn duy trì isolation an toàn.

Active learning từ false positive/negative do admin label. Hiện tại, audit log chỉ là dữ liệu một chiều: model đánh giá traffic, admin xem log. Bổ sung cơ chế cho

phép admin “flag” một bản ghi log là *false positive* (model sai khi block normal) hoặc *false negative* (model sai khi allow attack). Các bản ghi đã flag được tích lũy thành tập huấn luyện bổ sung, định kỳ kích hoạt một lần fine-tune nhẹ. Đây là vòng phản hồi *human-in-the-loop* đặc trưng cho các hệ thống ML production.

Clustering mode: shared state qua Redis. Hiện tại, mỗi WAF instance hoạt động độc lập – rate limiter, blacklist động, ML cache không được share. Trong môi trường có nhiều instance phía sau load balancer, attacker có thể rotate qua các instance để tăng burst limit hiệu quả. Chuyển các shared state quan trọng sang Redis (rate limit bucket, IP blacklist hit counter, ML response cache) để các instance hoạt động như một cluster logic đồng nhất.

Hỗ trợ HTTP/2 và HTTP/3 native. Hiện tại, hệ thống dùng `httputil.ReverseProxy` hỗ trợ HTTP/1.1 và HTTP/2. HTTP/3 (QUIC) đang được triển khai rộng rãi, và một WAF không hỗ trợ HTTP/3 sẽ bị bypass khi client và backend chuyển sang giao thức này. Thư viện `quic-go` đã trưởng thành đủ để tích hợp.

6.2.3 Hướng dài hạn (12 tháng trở lên)

Các định hướng dài hạn mang tính chất nghiên cứu hơn là phát triển sản phẩm; chúng có thể trở thành các đề tài nghiên cứu riêng biệt.

Tích hợp threat intelligence feed. Kết nối hệ thống với các nguồn threat intelligence công cộng (AbuseIPDB, MISP, Spamhaus DROP list) và thương mại (CrowdStrike, Recorded Future). IP reputation, malware URL, known TOR exit node được tự động sync và áp dụng cho engine với policy được cấu hình. Đây là một trong những lợi thế cốt lõi của WAF thương mại mà giải pháp mã nguồn mở cần lấp đầy.

Zero-day detection bằng unsupervised anomaly detection. Bổ sung một lớp model thứ hai song song với DistilBERT: unsupervised model (autoencoder hoặc one-class SVM trên embedding) được huấn luyện chỉ trên traffic normal. Bất kỳ request nào lệch xa khỏi cluster normal đều được flag, ngay cả khi không match bất kỳ rule hay nhãn tấn công nào đã biết. Đây là cơ chế bắt zero-day mà supervised model không thể có (vì supervised chỉ biết các lớp đã được label).

Compliance reporting cho PCI-DSS, GDPR, ISO 27001. Các tổ chức chịu sự ràng buộc của các tiêu chuẩn tuân thủ thường yêu cầu báo cáo định kỳ về các sự kiện bảo mật, log retention theo thời gian quy định, và evidence của các biện pháp phòng chống. Hệ thống có thể bổ sung module sinh báo cáo theo template chuẩn của từng tiêu chuẩn, audit trail không thể chỉnh sửa (write-once log), và policy retention tự động.

Phân tích hành vi người dùng (User Behavior Analytics – UBA). Theo dõi pattern truy cập của từng người dùng đã đăng nhập (sequence các endpoint, thời gian giữa request, geolocation của IP) và detect các bất thường (đăng nhập từ vị trí mới, truy cập endpoint không thường lệ, hành vi giống bot). UBA bổ sung một chiều bảo vệ khác cho lớp authentication mà mật khẩu và JWT đơn lẻ không cung cấp được.

Mở rộng sang API Gateway với schema validation. Đối với các API RESTful/GraphQL, hệ thống có thể đọc OpenAPI/GraphQL schema và validate mọi request có đúng cấu trúc, kiểu dữ liệu, và phạm vi giá trị hay không. Đây là một dạng phòng thủ chủ động vượt ra ngoài pattern matching: từ chối request không tuân thủ schema ngay cả khi không trigger rule nào.

Kết chương. Đồ án đã hoàn thành phần lớn các mục tiêu đề ra: một hệ thống WAF mã nguồn mở hiệu năng cao, tích hợp đầy đủ rule engine, lớp ML xác minh, dashboard quản trị, lớp xác thực bảo mật, và pipeline huấn luyện có kiểm soát chất lượng. Sáu kết quả cụ thể đã được trình bày kèm số liệu định lượng. Bốn hạn chế tồn tại được nhìn nhận thẳng thắn và đều có kế hoạch khắc phục trong giai đoạn ngắn hạn hoặc trung hạn. Mười ba định hướng phát triển được đề xuất, trải đều ba mốc thời gian, mở đường cho việc tiếp tục đề tài thành một dự án nghiên cứu và phát triển dài hạn hơn. Hệ thống ở trạng thái hiện tại đã sẵn sàng làm nền tảng cho các thử nghiệm tiếp theo trong môi trường giáo dục và lab, và là một bước đệm hợp lý hướng tới một sản phẩm mã nguồn mở có thể triển khai trong môi trường sản xuất sau khi hoàn thiện các hạng mục ngắn hạn đã nêu.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/choi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cầu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

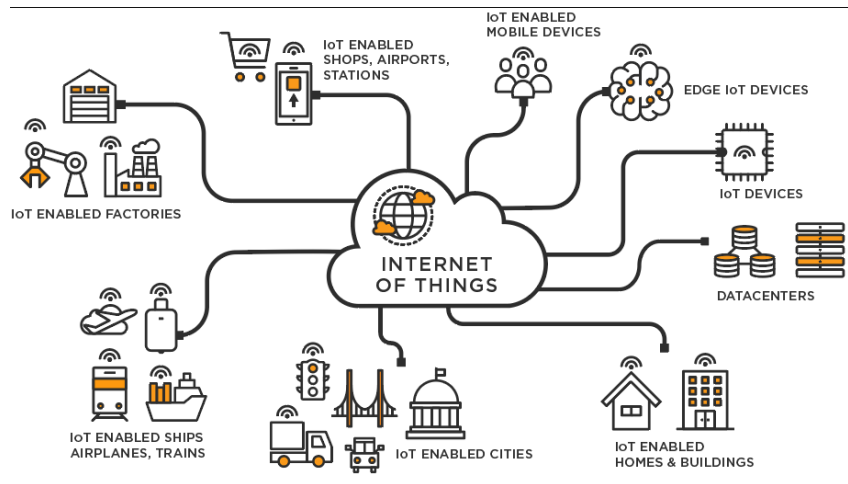
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```



Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Cụ thể, sinh viên sử dụng lệnh `\cite{khoaTaiLieu}` trong văn bản để trích dẫn – ví dụ `\cite{owasp2021top10}`. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

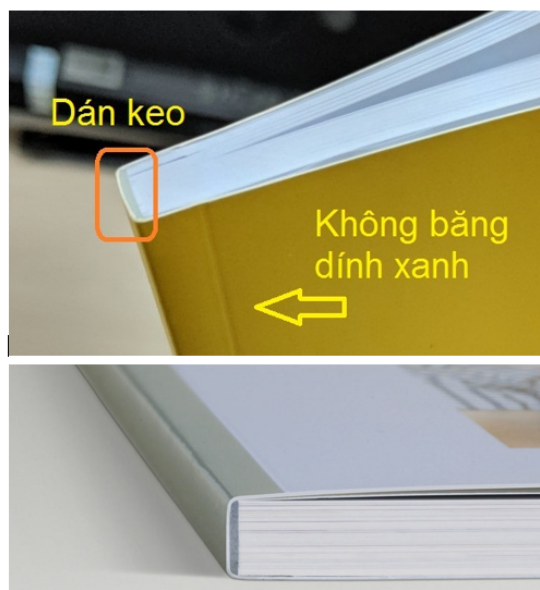
Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

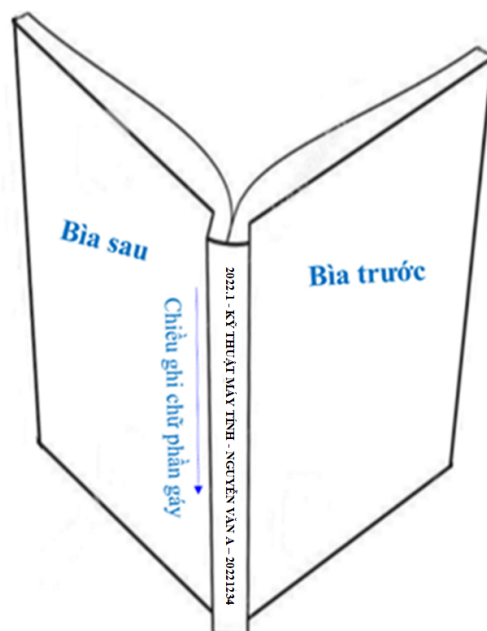
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.2 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Quy cách đóng quyển đồ án



Hình A.3: Quy cách ghi chữ trên giấy đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thông kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...